

FUJINET ENGINEERING SERIES

The RS-232 FujiNet User's Manual

Every FujiNet device, driven through a plain serial port

fnconfig.ini · the FujiBus packet · a command reference with real byte streams · CONFIG, NETCAT and 5
CARD STUD in Z80 for the Altair 8800

About this manual

This manual is for anyone who wants to drive a FujiNet from a computer through an RS-232 serial port, whatever that computer is. It is not tied to one platform: it describes the wire itself. It shows how to set the adapter up with its `fnconfig.ini` file, how the FujiBus packet is built byte by byte, and what every device on the bus accepts and answers, each command with the exact byte streams that go out and come back. Three appendices then build complete programs in Z80 assembly language for an Altair 8800 with a MITS 88-2SIO serial board: a CONFIG that manages host and disk slots, a NETCAT terminal, and a client for FujiNet 5 Card Stud. Their hardware-dependent code is marked off so that it can be moved to any other serial card.

It is typeset in the FujiNet engineering series style, alongside *The FujiNet Network Protocol Handbook*, which covers the network protocols themselves in depth. Every byte stream printed here was captured from a running adapter, and every program was assembled and run on a simulated Altair against it.

Canonical sources

<code>fujinet-firmware</code>	<code>lib/bus/rs232</code> , <code>lib/device/rs232</code> , <code>lib/device/fujiDevice</code> , <code>lib/device/NDevice</code> , <code>lib/config</code> (<code>670b2ed55</code>)
<code>fujinet-pc-rs232</code>	the RESEND extension, branch <code>rs232-resend</code> (<code>1c5dcb807</code>)
<code>fujinet-5cardstud</code>	the 5 Card Stud clients (<code>7333b1b</code>)
<code>fujinet-game-system</code>	the 5 Card Stud server and its binary state format (<code>19770c3</code>)
<code>simh</code>	the AltairZ80 simulator and its 88-2SIO model (<code>5cfa8662</code>)
<code>z88dk</code>	<code>z88dk-z80asm</code> , which assembles every listing (<code>3bd06cad56</code>)

Contents

1	Introduction	1
1.1	What this manual covers	1
1.2	Conventions	1
2	The RS-232 FujiNet	3
2.1	Hardware variants	3
2.2	The serial line	3
2.3	Powering up	4
2.4	fujinet-pc	4
2.5	The web interface	4
3	Configuring fnconfig.ini	6
3.1	Where the file lives	6
3.2	The format	6
3.3	[General]	6
3.4	[WiFi] and [WiFiStored1] to [WiFiStored8]	7
3.5	[HostN]: the eight host slots	7
3.6	[MountN]: the eight disk slots	7
3.7	[PrinterN]	8
3.8	[Serial] and [BOIP]	8
3.9	[Modem] and [Phonebook1] to [Phonebook16]	8
3.10	[Network]	9
3.11	Cloud credentials: [S3], [GoogleDrive], [OneDrive]	9
3.12	Sections the RS-232 build ignores	9
3.13	Three working files	9
4	The FujiBus Packet	12
4.1	A first look	12
4.2	SLIP framing	12
4.3	The header	13
4.4	The checksum	13
4.5	Parameters and descriptors	14
4.6	The payload	15
4.7	The reply	15
4.8	Building a packet, step by step	15
5	Transactions	16
5.1	One command, one reply	16
5.2	Timing	16
5.3	When no reply comes	17
5.4	Retrying	17
5.5	Stale bytes	18
5.6	Sharing the line with the modem	18
5.7	The rules, in short	18
6	The Devices on the Bus	19
6.1	A first session	19
7	The Fuji Device (70h)	22
7.1	WiFi	22
7.2	Adapter information	24
7.3	Host slots	25
7.4	Browsing a host	27
7.5	Disk slots	30
7.6	Bootng	32

7.7	Copying files	33
7.8	Application keys	33
7.9	Base64	35
7.10	Hashing	36
7.11	QR codes	37
7.12	Random numbers and GUIDs	39
7.13	Commands not available on RS-232	39
8	The Disk Drives (31h–38h)	40
8.1	What an image is	40
8.2	Commands	40
9	The Network Units (71h–78h)	43
9.1	The life of a connection	43
9.2	The devicespec	43
9.3	Opening and closing	44
9.4	Reading and writing	45
9.5	A web request, start to finish	46
9.6	Posting data	46
9.7	Parsing JSON	47
9.8	Line endings and translation	49
9.9	Files and directories	49
9.10	Serving: TCP servers and UDP	51
9.11	Interrupt rate	53
9.12	Status codes	54
9.13	Commands not available on RS-232	54
10	The Clock (45h)	55
10.1	Reading the time	55
10.2	Time zones	57
11	The Printer (40h)	59
11.1	Printer types	60
12	The Modem (50h)	61
12.1	Talking to the modem	61
12.2	A call	61
12.3	AT commands	62
12.4	Modem commands	62
13	The CP/M Device (5Ah)	63
A	Quick Reference	64
A.1	The packet	64
A.2	Devices	64
A.3	Fuji device (70h)	64
A.4	Disk drives (31h–38h)	65
A.5	Network units (71h–78h)	65
A.6	Clock (45h), printer (40h), modem (50h)	66
B	A FujiBus Library in Z80	67
B.1	The platform layer: ALTAIR.INC	67
B.2	Moving to other hardware	68
B.3	The packet layer: FUJIBUS.INC	69
B.4	The console layer: CONIO.INC	71
B.5	A first program: HELLO	72
B.6	Building	72
B.7	Listings	72
C	CONFIG in Z80	82
C.1	What it does	82
C.2	Talking to the Fuji device	84

C.3 The display	84
C.4 Naming a host	85
C.5 Browsing	86
C.6 Joining a network	88
C.7 Listing	88
D NETCAT in Z80	97
D.1 Opening	97
D.2 The loop	98
D.3 Listing	99
E 5 Card Stud in Z80	102
E.1 The server	103
E.2 Fetching	103
E.3 The game loop	104
E.4 Reporting for a Teletype	105
E.5 Asking for a move	106
E.6 Listing	106
F Running the Examples in SIMH	116
F.1 fujinet-pc	116
F.2 The simulator	116
F.3 Running a program	117
F.4 On real hardware	117
G Firmware Behaviour and Errata	118

CHAPTER 1

Introduction

FujiNet began as a peripheral for the Atari 8-bit computers and grew into a family: one ESP32 board and one firmware, wearing a different bus interface on each machine. The RS-232 FujiNet is the plainest member of that family. It has no cartridge edge and no proprietary connector, just a nine-pin serial port. Anything that can send and receive bytes on a serial line can use it: an IBM PC, a CP/M machine, an S-100 system, a terminal server, a microcontroller, or a Python script on a laptop.

That plainness is what this manual is about. Every FujiNet feature — WiFi, the eight host slots and eight disk slots, the `N`: network device with its two dozen protocols, the real-time clock, the printer and the modem — is reached the same way: by sending a small, self-describing packet down the serial line and reading the one packet that comes back. Learn that packet and you can drive all of FujiNet from any machine.

1.1 What this manual covers

1. **Getting connected.** Chapter 2 describes the hardware, the serial line and its control signals, and *fujinet-pc*, the desktop build of the firmware that speaks the same protocol over a serial port or a TCP socket. Chapter 3 is a complete guide to `fnconfig.ini`, the file that holds every setting the adapter keeps.
2. **The protocol.** Chapter 4 takes the FujiBus packet apart byte by byte: SLIP framing, the six-byte header, the checksum, the parameter descriptors and the payload. Chapter 5 covers the rules of a transaction, including timeouts, retries and the mistakes the firmware does not forgive. Chapter 6 lists the devices on the bus.
3. **The devices.** Chapters 7 to 13 are the command reference, one chapter per device: the Fuji device that manages the adapter itself, the eight disk drives, the eight network units, the clock, the printer, the modem and the CP/M device. Every command is shown with the byte stream a program sends and the byte stream FujiNet sends back.
4. **Working programs.** The appendices hold a quick reference, a small FujiBus library in Z80 assembly language, and three programs built on it for an Altair 8800 with an 88-2SIO serial board: CONFIG, NETCAT and a client for FujiNet 5 Card Stud. A final appendix runs them all on a simulated Altair, and another lists the firmware behaviour that may surprise you.

1.2 Conventions

Numbers written with an `h` suffix are hexadecimal, as Z80 assemblers write them: `70h` is 112. Multi-byte values on the wire are always **little-endian**, low byte first; a 16-bit length of 262 travels as `06 01`. Device and command names follow the firmware source (`FUJI_GET_WIFISTATUS` is written `GET_WIFISTATUS`). Parameters are numbered from zero, so **p0** is the first parameter of a command and **p1** the second. Slot numbers on the wire count from zero even though people count host and disk slots from one: disk slot 1 is `p0 = 0`.

The byte streams in this book are drawn as rows of cells, coloured by what each byte is (the legend is in Chapter 4). They are not illustrations: each one is a real exchange, captured from a FujiNet running the firmware this manual describes, and every checksum in them is right.

NOTE

This manual describes `fujinet-firmware` at commit `670b2ed55`. Where the firmware does something it plainly was not meant to do, the text says so where it matters, and Appendix G collects every such case with its source reference.

PART I

Getting Connected

The RS-232 FujiNet hardware, the serial line and its control signals, fujinet-pc, and the `fnconfig.ini` file that holds every setting the adapter keeps.

CHAPTER 2

The RS-232 FujiNet

The RS-232 FujiNet is an ESP32 with a serial line driver and a DB-9 connector. It is powered from its own USB-C socket, not from the serial port, so plug it into power first and then into the computer. Inside it runs the same firmware as every other FujiNet, built for the RS-232 bus.

2.1 Hardware variants

Three firmware targets drive the RS-232 bus. They differ only in how the bytes reach the ESP32; the packets are identical.

Target	Processor	Host link
<code>fujinet-rs232-rev0</code>	ESP32, 8 MB	UART 1 through a TRS3238E line driver: RX on GPIO 13, TX on GPIO 21.
<code>fujinet-rs232-s3</code>	ESP32-S3, 16 MB	UART 1 through a TRS3238E: RX on GPIO 41, TX on GPIO 42. UART 0 (GPIO 43/44) is the debug console on the USB port.
<code>fujiversal-rs232</code>	ESP32-S3	USB CDC-ACM host: the FujiBus packets travel over USB to a Fujiversal cartridge. There is no UART and the baud rate is ignored.

The two serial boards present the port as **data communications equipment** (DCE), the way a modem does: they carry a female DB-9 that plugs straight onto a PC's male COM port, with no null-modem cable. On a machine whose serial port is itself wired as DCE (many S-100 serial boards can be strapped either way), a null-modem adapter is needed.

DB-9 pin	Signal	Direction	Use on the RS-232 FujiNet
2	RXD	FujiNet → computer	replies
3	TXD	computer → FujiNet	commands
5	GND	—	signal ground
9	RI	FujiNet → computer	network attention (Section 2.2)
1	DCD	FujiNet → computer	driven only by the modem device
6	DSR	FujiNet → computer	driven only by the modem device
8	CTS	FujiNet → computer	held on (see the note below)
4	DTR	computer → FujiNet	read only by the modem device
7	RTS	computer → FujiNet	read only by the modem device

Three wires are all the bus needs: TXD, RXD and ground. Commands travel **entirely in-band**; there is no command line or attention line that must be asserted before a packet, as there was on the original Atari SIO bus.

NOTE

The firmware asks the ESP32 for RTS/CTS flow control, but it never routes those pins to the UART: CTS is set to **on** at start-up and stays there, and RTS is never looked at. In practice there is no hardware flow control, so do not wait for anything before sending. The line is 8 data bits, no parity, one stop bit.

2.2 The serial line

The default speed is **115,200 baud**, set by `[Serial] baud` in `fnconfig.ini` (Chapter 3). Any standard rate works; a computer whose UART cannot reach 115,200 simply needs the adapter set to match it. The Altair programs in this book run at 9600 baud, the top of the 88-2SIO's usual jumper settings.

The adapter answers only when spoken to. It never sends a packet on its own, with one exception: the modem device (Chapter 12) sends text whenever it has some. A program that never uses the modem can treat every byte that arrives as part of the reply to the command it just sent.

The RI “attention” line

FujiNet can tell a computer that a network unit wants attention without being asked. While any open network unit has data waiting, has lost its connection, or has an error to report, the firmware toggles the **RI** (ring indicator) line. Normally RI is held **on**. When a unit needs attention RI goes **off** for one interrupt period and on for the next, every `2 x rate` milliseconds, until the condition is cleared. The rate is 100 ms by default and can be changed per unit with `SET_INT_RATE` (Chapter 9). For 5 ms after each `READ` the line is left alone.

RI is a convenience. A program can always ask instead, by sending a network `STATUS` command, and the programs in this book do exactly that: an 88-2SIO has no RI input. On `fujinet-pc` the same signal is driven on the PC's own DTR or RTS line, chosen by `[Serial] proceed` (Chapter 3).

2.3 Powering up

At power-up the firmware reads `fnconfig.ini`, joins the WiFi network it names, and loads the host slots and disk slots it lists. Two settings decide what disk device 31h (disk slot 1) holds when the computer first asks:

- The **boot image**. `[General] boot_mode` names an image kept in the adapter's flash (mode 0 is `/autorun.img`) and the firmware places it in disk slot 1 at power-up. It is meant for computers that boot from the FujiNet, such as an IBM PC with the FujiNet BIOS driver.
- The **CONFIG switch**. When `[General] configenabled` is `0`, the firmware mounts every disk slot listed in `fnconfig.ini` as soon as WiFi is up, replacing the boot image with the slot 1 entry if there is one. When it is `1` (the default), the slots are loaded but not mounted until a program sends `MOUNT_ALL` or `MOUNT_IMAGE`.

A computer that does not boot from the FujiNet should normally set `configenabled=0`, so its disks are ready the moment it starts.

2.4 fujinet-pc

fujinet-pc is the FujiNet firmware built as an ordinary program for

Linux, macOS or Windows. Built for the RS-232 target it speaks exactly the same packets as the hardware, and it is the easiest way to develop for the FujiNet: everything in this manual was captured from it. It reaches the computer in one of two ways:

Link	How it is set up
Serial port	<code>[Serial] port</code> names the port (<code>/dev/ttyUSB0</code> , <code>COM3</code> , a pseudo-terminal) and <code>[Serial] baud</code> its speed. If <code>port</code> is empty, <code>/dev/ttyUSB0</code> and then <code>/dev/ttyS0</code> are tried.
TCP (Bus over IP)	With <code>[BOIP] enabled=1</code> , <i>fujinet-pc</i> listens on a TCP port (1985 by default for the RS-232 build) and the same SLIP frames travel over the socket. An emulator, a terminal server or a test script connects to it.

It is started from its `dist` directory:

```
./fujinet -c fnconfig.ini -s SD -u http://0.0.0.0:8000
```

`-c` names the configuration file, `-s` the directory that stands in for the SD card, and `-u` the address of the built-in web interface. On *fujinet-pc* the serial and Bus-over-IP settings are read only at start-up; restart the program after changing them. Appendix F uses *fujinet-pc* over TCP to run the Altair programs on a simulator.

2.5 The web interface

The adapter also serves a configuration page on port 80 (*fujinet-pc*: the `-u` address). It shows the network, the host and disk slots and the printer, and offers a firmware update. On the RS-232 build it can set the baud rate; the

other serial settings are made in `fnconfig.ini`. Anything the web page can change, a program on the computer can change too, with the commands in Chapter 7.

CHAPTER 3

Configuring fnconfig.ini

Everything the FujiNet remembers between power cycles lives in one text file, `fnconfig.ini`. The adapter rewrites it whenever a setting changes — when a program saves host slots, mounts a disk, or joins a network — and reads it at every start. Editing it by hand is the quickest way to set an adapter up before it is ever connected to a computer.

3.1 Where the file lives

On the hardware the file is kept in two places. At start-up the firmware looks for `/fnconfig.ini` on the SD card first and uses it if present; otherwise it reads the copy in its internal flash. When it saves, the `[General] fnconfig_on_spifs` setting decides where it writes: with `1` (the default) it writes the flash copy and then copies it to the SD card; with `0` it writes the SD card only.

So the simplest way to configure an adapter is to put a prepared `fnconfig.ini` in the root of an SD card, insert it, and power up.

On fujinet-pc the file is whatever `-c` names, `fnconfig.ini` in the current directory by default.

3.2 The format

The file is a series of sections. Each starts with a name in square brackets and holds `name=value` lines until the next section:

```
[General]
devicename=ALTAIR-FN
configenabled=0

[WiFi]
SSID=My Network
passphrase=correct horse battery staple
```

1. Section names are matched without regard to case, by their **beginning**: `[Host3]` is the third host section, `[Mount1]` the first disk slot. Numbered sections count from 1.
2. Key names are also case-blind. Spaces around the name and the value are trimmed; spaces inside a value are kept (an SSID may contain them).
3. A yes/no value is true when it begins with `1`, `T`, `t`, `Y` or `y`. Three keys are stricter and are true only for exactly `1`: `[WiFi] enabled`, `[WiFi] multi_ap` and `[Bluetooth] enabled`.
4. The firmware writes the file with CR LF line endings. It reads either.
5. Sections the firmware does not use on the RS-232 build are still read and written back, so one file can move between platforms.

The rest of this chapter lists every section and key. The **RS-232** column says whether the RS-232 firmware acts on the setting.

3.3 [General]

Key	Default	RS-232	Meaning
<code>devicename</code>	<code>FujiNet</code>	yes	The adapter's name, used as its network host name.
<code>timezone</code>	empty	yes	A POSIX time-zone string such as <code>CST6CDT,M3.2.0,M11.1.0</code> , used by the clock device. Empty means UTC.
<code>configenabled</code>	<code>1</code>	yes	<code>0</code> : mount every listed disk slot at start-up. <code>1</code> : leave them for a program to mount (Chapter 2).

Key	Default	RS-232	Meaning
<code>altconfigfile</code>	empty	partly	An alternate CONFIG image. When it is set, slots are not mounted at start-up even if <code>configenabled=0</code> .
<code>boot_mode</code>	<code>0</code>	yes	The image placed in disk slot 1 at power-up: <code>0</code> <code>/autorun.img</code> , <code>1</code> <code>/mount-and-boot.img</code> , <code>2</code> the lobby image on <code>tnfs.fujinet.online</code> , <code>3</code> <code>/hisioboot-fujinet.img</code> .
<code>fnconfig_on_spifs</code>	<code>1</code>	hardware	Where saves go (see above).
<code>printer_enabled</code>	<code>1</code>	yes	<code>0</code> makes the printer device ignore every command, without even a reply.
<code>encrypt_passphrase</code>	<code>0</code>	yes	<code>1</code> stores the WiFi passphrase obfuscated rather than in clear text.
<code>hsioindex</code>	<code>-1</code>	no	Atari high-speed SIO index.
<code>rotationsounds</code>	<code>1</code>	no	Atari disk-rotation sounds.
<code>config_ng</code>	<code>0</code>	no	Atari CONFIG selection.
<code>status_wait_enabled</code>	<code>1</code>	no	Atari and Commander X16 only.

3.4 [WiFi] and [WiFiStored1] to [WiFiStored8]

Key	Default	Meaning
<code>enabled</code>	<code>1</code>	<code>1</code> to use WiFi at all.
<code>SSID</code>	empty	The network to join.
<code>passphrase</code>	empty	Its password (obfuscated if <code>encrypt_passphrase=1</code>).
<code>multi_ap</code>	<code>0</code>	<code>1</code> to try the stored networks below as well.

Each `[WiFiStoredN]` section holds another `SSID` and `passphrase` for the adapter to try; a section that exists is in use. A program sets the primary network with `SET_SSID` (Chapter 7), which also saves it here.

3.5 [HostN]: the eight host slots

A host slot names a place disk images are kept. Sections `[Host1]` to `[Host8]` each hold:

Key	Meaning
<code>type</code>	<code>SD</code> for the adapter's own SD card, <code>TNFS</code> for a TNFS server.
<code>name</code>	<code>SD</code> for the SD card; otherwise the server's host name, such as <code>apps.irata.online</code> .

```
[Host1]
type=SD
name=SD

[Host2]
type=TNFS
name=tnfs.fujinet.online
```

3.6 [MountN]: the eight disk slots

Sections `[Mount1]` to `[Mount8]` say what image each disk slot holds. Disk slot N is disk device `30h + N`.

Key	Meaning
<code>hostslot</code>	The host slot the image lives on, counted from 1 .
<code>path</code>	The image's path on that host.
<code>mode</code>	<code>r</code> for read-only, <code>w</code> for read/write.

```
[Mount1]
hostslot=2
```

```
path=/cpm/altair/cpm22.dsk
mode=r
```

3.7 [PrinterN]

Key	Meaning
<code>type</code>	The printer emulation, a number from the table in Chapter 11. An unknown number falls back to <code>1</code> , a text file with trailing blanks trimmed.
<code>port</code>	The printer port, 1 to 4.

CAUTION

Only the first printer is ever used on the RS-232 build, and a bug in the section-name parser makes every `[PrinterN]` section load into that first slot. If you have more than one, the last in the file wins. Keep just `[Printer1]`.

3.8 [Serial] and [BOIP]

These two sections choose the link to the computer.

Key	Default	Applies to	Meaning
<code>[Serial] baud</code>	115200	hardware, fujinet-pc	Line speed. Ignored on the USB <code>fujiversal-rs232</code> target.
<code>[Serial] port</code>	empty	fujinet-pc	The serial device to open.
<code>[Serial] proceed</code>	DTR	fujinet-pc	Which of the PC's output lines carries the RI attention signal: <code>DTR</code> , <code>RTS</code> or <code>none</code> .
<code>[Serial] command</code>	DSR	—	An Atari setting, read but not used by the RS-232 bus.
<code>[BOIP] enabled</code>	0	hardware, fujinet-pc	<code>1</code> to carry the bus over TCP instead of the serial port.
<code>[BOIP] host</code>	hardware: empty; fujinet-pc: <code>localhost</code>	hardware, fujinet-pc	The address to listen on; empty or <code>*</code> means every address.
<code>[BOIP] port</code>	1985	hardware, fujinet-pc	The TCP port. An empty or out-of-range value means 1985.

CAUTION

fujinet-pc writes the `[Serial]` section twice when it saves (once with `port`, once with `baud`). Both are read correctly, so the duplicate is harmless. But it writes `proceed` back only on the Atari build: on the RS-232 build a hand-edited `proceed=RTS` is lost the next time the adapter saves its settings, and RI returns to DTR.

3.9 [Modem] and [Phonebook1] to [Phonebook16]

Key	Default	Meaning
<code>modem_enabled</code>	1	<code>0</code> silences the modem device entirely.
<code>sniffer_enabled</code>	0	<code>1</code> records modem traffic to a file.
<code>connect_delay_ms</code>	2000	Atari only.

Each `[PhonebookN]` section maps a number the modem can dial to a host: `number`, `host` and `port`. Chapter 12 shows them in use.

3.10 [Network]

Key	Default	Meaning
sntpserver	pool.ntp.org	The time server the clock uses.
log_network_json	0	1 logs every JSON document the parser reads (debug builds).
netstream_*	—	Five keys for the Atari NetStream feature (host , port , mode , register , rx_depth); not used on RS-232.

3.11 Cloud credentials: [S3], [GoogleDrive], [OneDrive]

These hold the credentials for the S3: , GDRIVE: , GCAL: , GMAIL: and ONEDRIVE: protocols of the network device. [S3] has endpoint , region , access_key , secret_key and use_ssl (default 1); [GoogleDrive] and [OneDrive] have refresh_token , access_token and token_expiry . *The FujiNet Network Protocol Handbook* explains how to obtain them.

3.12 Sections the RS-232 build ignores

[Bluetooth] , [Cassette] , [CPM] and [ENABLE] are read and written back, but nothing on the RS-232 bus uses them. [Tape1] is read too, and because of a bug it overwrites the first disk slot when present; leave it out.

3.13 Three working files

An Altair at 9600 baud

The configuration the Altair programs in this book were written for: the 88-2SIO's port B strapped for 9600 baud, two TNFS hosts and the SD card, CP/M in disk slot 1, mounted at power-up.

```
[General]
devicename=ALTAIR-FN
configenabled=0
timezone=CST6CDT,M3.2.0,M11.1.0

[WiFi]
enabled=1
SSID=My Network
passphrase=my password

[Serial]
baud=9600

[Host1]
type=SD
name=SD

[Host2]
type=TNFS
name=tnfs.fujinet.online

[Host3]
type=TNFS
name=apps.irata.online

[Mount1]
hosts1slot=2
path=/cpm/altair/cpm22.dsk
mode=r

[Printer1]
type=1
port=1
```

fujinet-pc on a serial port

```
[Serial]
port=/dev/ttyUSB0
baud=115200
proceed=DTR

[BOIP]
enabled=0
```

fujinet-pc for a simulator

The configuration Appendix F uses: the bus is carried over TCP port 1985 on the local machine, where the simulated Altair's serial port connects.

```
[BOIP]
enabled=1
host=localhost
port=1985
```

TIP

Settings that a program can change, the adapter saves for you. A program that writes host slots, mounts disks or joins a network does not need to touch `fnconfig.ini` at all: the firmware rewrites it with the new values.

PART II

The FujiBus Protocol

How a command is framed, checked and answered: the SLIP envelope, the six-byte header, the checksum, the parameter descriptors and the payload; the rules of a transaction; and the devices that answer on the bus.

CHAPTER 4

The FujiBus Packet

Every exchange on the RS-232 bus is one packet from the computer and one packet back. The firmware calls the format *FEP-004*; this manual calls it the **FujiBus packet**. It is self-describing: the packet says how long it is, checks itself, and says how many parameters it carries and how wide each one is. This chapter builds one from nothing.

4.1 A first look

Here is the smallest useful exchange: GET_WIFISTATUS, which asks the adapter whether it is on the network. The computer sends eight bytes and the adapter answers with nine.

■ SLIP END ■ device / command ■ length ■ checksum ■ descriptor ■ parameters ■ payload, ASCII below ■ escape pair

REQUEST → FUJINET

C0 70 FA 06 00 71 00 C0

REPLY · ACK · 1 PAYLOAD BYTE

C0 70 06 07 00 80 00 03 C0

Read the request from the left:

Byte	Value	Meaning
C0	END	SLIP: a frame starts.
70	device	The Fuji device, which manages the adapter.
FA	command	GET_WIFISTATUS.
06 00	length	The packet is 6 bytes long, low byte first.
71	checksum	Of all six bytes, computed with this byte as zero.
00	descriptor	No parameters follow.
C0	END	SLIP: the frame ends.

The reply has the same shape. Its device byte is again 70, the device that answered; its command byte is 06, ACK, meaning success; its length is 7 and its descriptor 0. The one byte after the header, 03, is the answer: connected. The rest of this chapter takes each part in turn.

4.2 SLIP framing

A packet travels inside a **SLIP** frame (RFC 1055). The frame begins and ends with the byte C0, called END.

Because C0 marks the edges, a C0 inside the packet must be disguised, and so must the escape byte itself:

Packet byte	Sent on the wire as
C0	DB DC
DB	DB DD
anything else	itself

The receiver reverses this: DB DC becomes C0, DB DD becomes DB. Escaping happens **after** the packet is built and checksummed, and is undone **before** it is checked; the length and the checksum always describe the packet itself, never its escaped form. This WRITE carries the two awkward bytes as its payload; on the wire each becomes a pair, and the frame grows from 12 bytes to 14 while its length field still says 0A, ten:

REQUEST → FUJINET · NOT SENT, COMPUTED

C0 71 57 0A 00 76 05 02 00 DB DC DB DD C0

REPLY · ACK · 0 PAYLOAD BYTES

C0 71 06 06 00 7D 00 C0

Four rules govern the frame:

1. **Send exactly one END before a packet and one after it.** RFC 1055 suggests opening a frame with an extra END to flush line noise. Do not: the firmware takes two ENDS in a row as an empty frame, discards it, and then reads the real packet as stray bytes (Chapter 5).
2. **Bytes before the first END are not part of the packet.** The firmware hands them to the modem device as if typed at a modem.
3. **A frame ends at the next END.** There is no other length limit on the wire; the length field inside is what is checked.
4. **An escape followed by anything but `DC` or `DD` is dropped.** The packet then fails its length or checksum test.

4.3 The header

After SLIP decoding, every packet starts with the same six bytes:

device	command	length (lo)	length (hi)	checksum	descriptor	params ...	payload ...
--------	---------	----------------	----------------	----------	------------	------------	-------------

Offset	Field	Meaning
0	device	The device the packet is for; in a reply, the device that answers. Chapter 6 lists them.
1	command	What to do. In a reply: <code>06</code> ACK or <code>15</code> NAK.
2–3	length	The total length of the packet in bytes, header included , low byte first. It counts the packet before SLIP escaping and without the END bytes. A packet whose length field disagrees with what arrived is thrown away.
4	checksum	See the next section.
5	descriptor	The first parameter descriptor (Section 4.5). <code>00</code> means no parameters.

4.4 The checksum

The checksum is an 8-bit sum of every byte of the packet — header, parameters and payload — with an **end-around carry**: each time the running sum overflows 255, the carry is added back into the low byte. It is computed with the checksum byte itself set to zero, and it is stored as it comes out, not inverted. In C:

```
uint8_t fb_checksum(const uint8_t *buf, size_t len)
{
    uint16_t chk = 0;
    for (size_t i = 0; i < len; i++) {
        chk += buf[i];
        chk = (chk >> 8) + (chk & 0xFF);    /* fold the carry back in */
    }
    return (uint8_t) chk;
}
```

In Z80 the fold is a single instruction: `ADD` sets the carry flag on overflow, and `ADC A,0` adds it back. This is the routine the programs in this book use:

```
CKSUM:  XOR    A                ; checksum of BC bytes at HL
CKS1:   ADD    A,(HL)
        ADC    A,0                ; end-around carry
        INC    HL
        DEC    BC
        LD     D,A
        LD     A,B
        OR     C
        LD     A,D
        JR     NZ,CKS1
        RET
```

The GET_WIFISTATUS request works out like this, with the checksum byte taken as zero:

Byte	Running sum	Carry out	After the fold
70	070	0	70
FA	16A	1	6B
06	071	0	71
00	071	0	71
00	071	0	71
00	071	0	71

The result, 71, is what goes in byte 4. To check a packet that arrives, save its checksum byte, set it to zero, compute, and compare.

4.5 Parameters and descriptors

Most commands take a few small numbers: a slot, a sector, a length, a mode. These travel as **parameters**, straight after the header. Each parameter is 1, 2 or 4 bytes wide, little-endian, and the descriptor byte says how many there are and how wide:

Descriptor (low 3 bits)	Parameters	Each	Bytes
0	none	—	0
1	1	8-bit	1
2	2	8-bit	2
3	3	8-bit	3
4	4	8-bit	4
5	1	16-bit	2
6	2	16-bit	4
7	1	32-bit	4

SET_DEVICE_FULLPATH, for instance, takes three 8-bit parameters (disk slot, host slot, mode), so its descriptor is 03 and the three bytes follow the header; after them comes the path as payload:

REQUEST → FUJINET

C0	70	E2	09	01	9B	03	01	00	01	2F	74	65	73	74	2F	64	69	73	6B	2E	...	245 more bytes ...	C0
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	--------------------	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	70	06	06	00	7C	00	C0
----	----	----	----	----	----	----	----

A disk READ takes one 32-bit sector number, descriptor 07 :

REQUEST → FUJINET · READ SECTOR 0 FROM DISK SLOT 1

C0	31	52	0A	00	94	07	00	00	00	00	C0
----	----	----	----	----	----	----	----	----	----	----	----

One descriptor describes at most four bytes of parameters, all the same width. When a command needs more, or mixed widths, bit 7 of the descriptor is set to say *another descriptor follows*; the extra descriptor bytes come right after the header, **before** any parameter values. One 8-bit parameter followed by one 16-bit parameter would be descriptors 81 05 and then the three value bytes. Bits 3 to 6 are not used. No command on the RS-232 bus needs more than one descriptor, but the firmware decodes chains correctly.

NOTE

The firmware reads a parameter's **value**, not its width: a slot number sent as a 16-bit parameter works as well as one sent as 8 bits. This manual uses the natural widths — 8 bits for slots, modes and flags, 16 bits for byte counts, 32 bits for sector numbers and offsets — and so do the example programs.

CAUTION

Send **every** parameter a command uses, even the ones it ignores. A handler that reads a parameter the packet does not carry throws an exception the firmware does not catch: fujinet-pc aborts, and the hardware restarts. A disk STATUS sent with no parameter crashes the adapter this way.

4.6 The payload

Everything after the parameters, up to the end of the packet, is the **payload**. Its length is not sent separately: it is the packet length minus the header, the extra descriptors and the parameters. Commands use the payload two ways:

- **Fixed-size structures.** A host-slot table is always 256 bytes, an SSID-and-password block 97, a path 256. Send them **at full size**, padding unused bytes with zeroes. The firmware copies whatever arrives into a buffer of the right size: a short payload leaves the rest of the buffer holding old or uninitialised memory (and some builds reject it outright).
- **Variable data.** A network devicespec, bytes written to a network connection, a time-zone string: these are sent at their natural length. Strings the firmware treats as C strings should end with a NUL byte; the reference says which.

4.7 The reply

The adapter answers every packet it can read with exactly one packet. Its **device** byte is the device that answered and its **command**

byte is the result:

Command byte	Name	Meaning
06	ACK	Success. Any data the command returns is the payload of this same packet.
15	NAK	Failure. A NAK never carries data.

A reply has no parameters, so its descriptor is always 00 and its payload starts at offset 6. Its length is 6 plus the number of data bytes. A command that returns nothing is answered with an empty ACK:

REQUEST → FUJINET

C0 70 F9 07 00 72 01 00 C0

REPLY · ACK · 0 PAYLOAD BYTES

C0 70 06 06 00 7C 00 C0

And a command the device does not know, or cannot carry out, with a bare NAK:

REQUEST → FUJINET

C0 70 01 06 00 77 00 C0

REPLY · NAK

C0 70 15 06 00 8B 00 C0

4.8 Building a packet, step by step

1. Write the device, the command, two zero bytes for the length, a zero for the checksum, and the descriptor.
2. Append any extra descriptors, then the parameter values, each little-endian.
3. Append the payload, padding fixed-size structures with zeroes.
4. Store the total length at offsets 2 and 3, low byte first.
5. Compute the checksum over the whole packet, with offset 4 still zero, and store it at offset 4.
6. Send C0, then each byte with C0 and DB escaped, then C0.

Receiving is the same in reverse: collect bytes from one END to the next, undo the escapes, check that the length field matches the number of bytes, check the checksum, then read the command byte.

CHAPTER 5

Transactions

A transaction is one command and its reply. The adapter handles one at a time, completely, before it reads the next packet, so the computer is always the one in charge of the pace. This chapter covers timing, what happens when something goes wrong, and how to recover.

5.1 One command, one reply

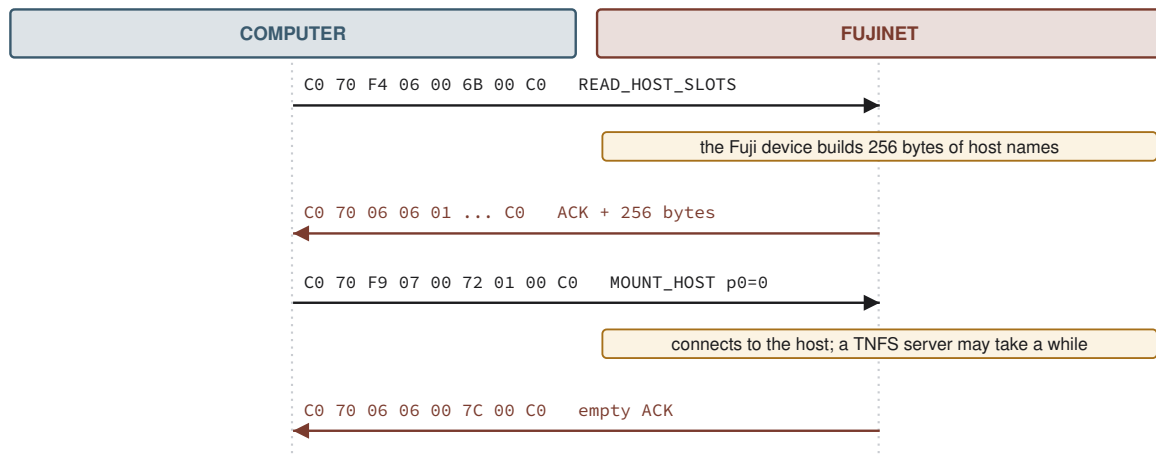


Figure 1. Two transactions. Nothing overlaps: the next command is sent only after the reply to the last one is in.

Write data travels in the command itself: a program that writes a sector puts the 512 bytes in the payload of the WRITE packet, and the reply is an empty ACK. Read data travels in the reply. No command needs a second packet in either direction.

5.2 Timing

Within a packet, the firmware waits at most 200 ms for each next byte on a serial port (500 ms over TCP). If the line goes quiet for longer than that in the middle of a frame, it gives up on the frame and discards what it had. A computer must therefore send each packet in one go, without pausing in the middle — an easy rule for any program that builds the packet first and sends it from memory.

Before the reply: most commands are answered within a few

milliseconds. Some take much longer, because the adapter has to do something slow on the network before it can answer:

Command	What it waits for
network OPEN	a DNS lookup, a TCP connection, a TLS handshake for <code>HTTPS: ;</code> seconds
SET_SSID	joining a WiFi network; up to tens of seconds
MOUNT_HOST, OPEN_DIRECTORY	a TNFS server
MOUNT_IMAGE	opening an image on a remote host
COPY_FILE	the whole copy

So a program needs two timeouts: a long one (ten seconds or more) for the first byte of a reply, and a short one between the bytes of a reply once it has begun. The Z80 library in Appendix B waits ten seconds for the first byte

by default, and a quarter of a second between bytes; CONFIG and NETCAT raise the first to 20 or 30 seconds around the slow commands.

After a reset or power-up, the firmware throws away anything that arrives until the line has been quiet for 100 ms.

5.3 When no reply comes

The adapter stays silent, rather than sending a NAK, in these cases:

Cause	What the firmware does
Bad checksum, wrong length, or a malformed frame	Logs “packet fail” and discards it. No reply at all.
A frame that stalled more than 200 ms	Discarded as above.
RESET (<code>FF</code>) to the Fuji device	Restarts immediately, without an ACK.
Any command to the printer when <code>printer_enabled=0</code>	Ignored.
A missing parameter	The firmware crashes (Chapter 4).
Any command to the modem device	The modem’s replies are never framed; see Chapter 12.

A packet for a device that does not exist is answered with a NAK whose device byte is `00` :

REQUEST → FUJINET

`C0 60 00 06 00 66 00 C0`

REPLY · NAK

`C0 00 15 06 00 1B 00 C0`

5.4 Retrying

When the reply does not come, or comes damaged, the computer cannot know whether the adapter ever saw the command. That matters for commands that change something:

Kind of command	Safe to send again?
Reads of state: GET_WIFISTATUS, READ_HOST_SLOTS, STATUS, disk READ	Yes. Running them twice changes nothing.
Settings: WRITE_HOST_SLOTS, SET_DEVICE_FULLPATH, MOUNT_IMAGE	Yes. The second run sets the same values again.
Network READ	No. The first READ took the bytes out of the adapter’s buffer; reading again returns the <i>next</i> bytes, and the lost ones are gone.
Network WRITE, printer WRITE	No. The data would be written twice.
Hash, Base64 and QR <i>input</i> commands	No. They append to a buffer.

For the first two kinds, the cure for a lost or damaged reply is to send the command again. For the others there are two choices: accept the rare loss, or use RESEND where the firmware has it.

RESEND: WHERE SUPPORTED

A FujiNet build that implements RESEND (fujinet-pc, branch `rs232-resend`) keeps a copy of the last reply it sent. Command `05` (ASCII ENQ), sent to any device with no parameters and no payload, makes it send that reply again, byte for byte, *without* running the command a second time. If it has sent nothing yet it answers NAK.

REQUEST → FUJINET · NOT SENT, COMPUTED

`C0 70 05 06 00 7B 00 C0`

REPLY · ACK · 1 PAYLOAD BYTE

`C0 70 06 07 00 80 00 03 C0`

Use RESEND only when part of a reply arrived and it was damaged — a wrong length or checksum, or a frame that stopped part-way. If *nothing* arrived, the command itself may never have reached the adapter, and RESEND would fetch the reply to the command before it. Firmware without RESEND (including the `670b2ed55` firmware this manual describes) treats `05` as an unknown command and answers NAK:

REQUEST → FUJINET

C0	70	05	06	00	7B	00	C0
----	----	----	----	----	----	----	----

REPLY · NAK

C0	70	15	06	00	8B	00	C0
----	----	----	----	----	----	----	----

The Z80 library in Appendix B asks for RESEND when it is assembled with `USE_RESEND` set, and sends the whole command again otherwise.

5.5 Stale bytes

A reply should only ever answer the command just sent. Two things can leave extra bytes waiting on the line and confuse the next exchange:

- A reply that arrived after the computer had given up on it.
- A command that answers twice: SET_SSID sends **two** NAKs when the adapter cannot join the network.

The defence is simple: before sending a command, read and discard anything already waiting. The Z80 library's FBCALL does this on every call.

5.6 Sharing the line with the modem

The modem device (Chapter 12) shares the serial line with the FujiBus packets, which leads to two traps:

- Any bytes that arrive **outside** a frame are given to the modem as if typed into it. A packet that loses its opening END is therefore not an error the adapter reports: its bytes go to the modem and the command is silently lost.
- While the modem has a connection open, it sends the remote end's data down the line **unframed**, interleaved with any FujiBus replies. A program that uses the modem must be ready to separate the two; a program that does not use the modem should leave it alone (or set `modem_enabled=0`).

5.7 The rules, in short

1. One END, the packet with `C0` and `DB` escaped, one END.
2. Length and checksum describe the unescaped packet, header included, with the checksum byte taken as zero.
3. Send every parameter the command reads; pad fixed-size payloads to full size.
4. Send a packet without pausing in the middle.
5. Drain stale input, send, then wait long for the first reply byte and briefly for the rest.
6. ACK carries the data; NAK carries nothing; silence means the packet was not read.
7. Retry reads freely; think before retrying anything that consumes or appends.

CHAPTER 6

The Devices on the Bus

Each packet names a device. The RS-232 firmware answers to these numbers:

Device	Name	What it is	Chapter
70	Fuji	The adapter itself: WiFi, host slots, disk slots, directories, application keys, and helpers for hashing, Base64 and QR codes.	7
31 – 38	Disk	The eight disk drives, one per disk slot: 31 is disk slot 1.	8
71 – 78	Network	The eight N: units: each opens one URL — HTTP, TCP, TNFS, SSH and two dozen more.	9
45	Clock	The real-time clock, in a dozen formats.	10
40	Printer	A printer that renders to a file on the adapter.	11
50	Modem	A Hayes-style modem that dials TCP hosts.	12
5A	CP/M	Defined, but not present on the RS-232 bus.	13

The disk drives and network units are numbered from one in conversation (disk slot 1, unit N1:) and from zero in parameters: the Fuji command that mounts an image into disk slot 3 takes p0 = 2 , and that drive then answers as device 33 .

6.1 A first session

The commands below, sent in order, take a freshly powered adapter from nothing to reading a disk sector. Every one is described in Part III; the streams here are the real ones.

1. Is the adapter on the network? GET_WIFISTATUS answers 03 for connected.

REQUEST → FUJINET

C0 70 FA 06 00 71 00 C0

REPLY · ACK · 1 PAYLOAD BYTE

C0 70 06 07 00 80 00 03 C0

2. Which hosts does it know? READ_HOST_SLOTS returns 8 names of 32 bytes each.

REQUEST → FUJINET

C0 70 F4 06 00 6B 00 C0

REPLY · ACK · 256 PAYLOAD BYTES

C0 70 06 06 01 B4 00 53 44 00 ... 242 more bytes ... C0

3. Connect to host slot 1 (the SD card) with MOUNT_HOST, p0 = 0 .

REQUEST → FUJINET

C0 70 F9 07 00 72 01 00 C0

REPLY · ACK · 0 PAYLOAD BYTES

C0 70 06 06 00 7C 00 C0

4. Put /test/disk.img from host slot 1 into disk slot 2, read-only: SET_DEVICE_FULLPATH with p0 = 1 (disk slot 2), p1 = 0 (host slot 1), p2 = 1 (read), then MOUNT_IMAGE with p0 = 1 , p1 = 1 .

REQUEST → FUJINET

C0 70 E2 09 01 9B 03 01 00 01 2F 74 65 73 74 2F 64 69 73 6B 2E ... 245 more bytes ... C0

REPLY · ACK · 0 PAYLOAD BYTES

C0 70 06 06 00 7C 00 C0

REQUEST → FUJINET

C0	70	F8	08	00	75	02	01	01	C0
----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	70	06	06	00	7C	00	C0
----	----	----	----	----	----	----	----

5. Read sector 0 from disk slot 2, which is device 32 .

REQUEST → FUJINET

C0	32	52	0A	00	95	07	00	00	00	00	C0
----	----	----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 512 PAYLOAD BYTES

C0	32	06	06	02	40	00	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	... 502 more bytes ...	C0
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	------------------------	----

That is the whole pattern. Everything else in this manual is the same exchange with a different device, command, parameters and payload.

PART III

The Devices

A command reference for every device on the RS-232 bus: the Fuji device, the disk drives, the network units, the clock, the printer, the modem and the CP/M device. Every command is shown with the bytes a program sends and the bytes FujiNet sends back.

CHAPTER 7

The Fuji Device (70h)

Device `70` is the adapter itself. It joins WiFi networks, keeps the eight host slots and eight disk slots, browses the hosts, mounts disk images, stores small files for programs, and offers a few helpers — hashing, Base64, QR codes, random numbers — that are hard to do on a small computer. Its commands fall into groups, and this chapter takes them group by group.

Each command below is shown as a card: its name and command byte, the parameters it reads, the payload it expects, and what it answers, followed by a real exchange.

7.1 WiFi

The adapter joins the network named in `fnconfig.ini` at power-up. These commands report on that connection and choose another network.

GET_WIFISTATUS

FA

PARAMS none
PAYLOAD none
REPLY 1 byte: `03` connected, `06` not connected

The quick question every program should ask first. The two values are borrowed from the ESP32's own status codes.

REQUEST → FUJINET

`C0 70 FA 06 00 71 00 C0`

REPLY · ACK · 1 PAYLOAD BYTE

`C0 70 06 07 00 80 00 03 C0`

GET_WIFI_ENABLED

EA

PARAMS none
PAYLOAD none
REPLY 1 byte: `01` if WiFi is enabled in `fnconfig.ini`, `00` if not

REQUEST → FUJINET

`C0 70 EA 06 00 61 00 C0`

REPLY · ACK · 1 PAYLOAD BYTE

`C0 70 06 07 00 7E 00 01 C0`

SCAN_NETWORKS

FD

PARAMS none
PAYLOAD none
REPLY 1 byte: the number of networks found

Scans for networks and keeps the list for `GET_SCAN_RESULT`. A scan takes a second or two; the reply waits for it. (fujinet-pc, which has no radio, always reports sixteen networks named after its configured one.)

CAUTION

GET_SSID returns the **stored WiFi password** in clear text in its last 64 bytes. Use GET_ADAPTERCONFIG to show the current network instead.

7.2 Adapter information

GET_ADAPTERCONFIG

E8

PARAMS

none

PAYLOAD

none

REPLY

140 bytes: the structure below

Everything about the adapter's network connection, in one reply. The addresses are binary, most significant byte first, the way they are written: 127.0.0.1 is 7F 00 00 01 .

REQUEST → FUJINET

C0

70

E8

06

00

5F

00

C0

REPLY · ACK · 140 PAYLOAD BYTES

C0

70

06

92

00

24

00

44

75

6D

6D

79

20

43

61

66

65

00

00

00

00

... 128 more bytes ...

C0

Offset	Size	Field
0	33	SSID of the current network, NUL-terminated; NOT CONNECTED when there is none
33	64	host name
97	4	IP address
101	4	gateway
105	4	netmask
109	4	DNS server
113	6	MAC address
119	6	BSSID (the access point's MAC address)
125	15	firmware version, NUL-terminated (e.g. v1.6-670b2ed55)

The IP and DNS addresses are filled in only while the adapter is connected.

GET_ADAPTERCONFIG_EXTENDED

C4

PARAMS

none

PAYLOAD

none

REPLY

240 bytes: the 140 above, then the addresses as text

The same 140 bytes, followed by the addresses already formatted as NUL-terminated strings, so a small program can print them without converting:

REQUEST → FUJINET

C0

70

C4

06

00

3B

00

C0

REPLY · ACK · 240 PAYLOAD BYTES

C0

70

06

F6

00

D1

00

44

75

6D

6D

79

20

43

61

66

65

00

00

00

00

... 228 more bytes ...

C0

Offset	Size	Field
140	16	IP address as text, "192.168.1.42"
156	16	gateway as text
172	16	netmask as text
188	16	DNS server as text
204	18	MAC address as text, "A0:B7:65:12:34:56"
222	18	BSSID as text

STATUS

53

PARAMS	none
PAYLOAD	none
REPLY	4 bytes, all zero

Present for compatibility. On the RS-232 bus it always answers four zero bytes.

REQUEST → FUJINET

C0	70	53	06	00	C9	00	C0
----	----	----	----	----	----	----	----

REPLY · ACK · 4 PAYLOAD BYTES

C0	70	06	0A	00	80	00	00	00	00	00	C0
----	----	----	----	----	----	----	----	----	----	----	----

DEVICE READY

00

PARAMS	none
PAYLOAD	none
REPLY	512 bytes, every one 41 (ASCII A)

A test of the link: a known 512-byte pattern that exercises a long reply. A program can use it at start-up to check that its receive code keeps up.

REQUEST → FUJINET

C0	70	00	06	00	76	00	C0
----	----	----	----	----	----	----	----

REPLY · ACK · 512 PAYLOAD BYTES

C0	70	06	06	02	01	00	41 A	41 A	41 A	41 A	41 A	41 A	41 A	41 A	41 A	41 A	... 500 more bytes ...	C0
----	----	----	----	----	----	----	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	------------------------	----

7.3 Host slots

A **host slot** names a place that disk images live: the adapter's SD card, or a TNFS server on the network. There are eight. The table is held as eight fixed 32-byte fields:

host 1: 32 bytes	host 2: 32 bytes	...	host 8: 32 bytes
------------------	------------------	-----	------------------

Each field holds a NUL-terminated name: **SD** for the SD card, otherwise a server's host name. An empty field (first byte zero) is an unused slot.

READ_HOST_SLOTS**F4**

PARAMS none
 PAYLOAD none
 REPLY 256 bytes: 8 host names of 32 bytes

REQUEST → FUJINET

C0	70	F4	06	00	6B	00	C0
----	----	----	----	----	----	----	----

REPLY · ACK · 256 PAYLOAD BYTES

C0	70	06	06	01	B4	00	53	44	00	00	00	00	00	00	00	00	00	00	00	...	242 more bytes	...	C0
							S	D															

WRITE_HOST_SLOTS**F3**

PARAMS none
 PAYLOAD 256 bytes: 8 host names of 32 bytes
 REPLY empty ACK

Replaces the whole table and saves it to `fnconfig.ini`. All eight hosts are marked as not connected; mount a host again before browsing it. To change one slot, read the table, change that field, and write all 256 bytes back.

REQUEST → FUJINET

C0	70	F3	06	01	A2	00	53	44	00	00	00	00	00	00	00	00	00	00	00	...	242 more bytes	...	C0
							S	D															

REPLY · ACK · 0 PAYLOAD BYTES

C0	70	06	06	00	7C	00	C0
----	----	----	----	----	----	----	----

MOUNT_HOST**F9**

PARAMS p0: host slot, 0 to 7
 PAYLOAD none
 REPLY empty ACK

Connects to the host: for a TNFS server this opens a session, which may take a moment. Mount a host before opening a directory on it or mounting an image from it.

REQUEST → FUJINET

C0	70	F9	07	00	72	01	00	C0
----	----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	70	06	06	00	7C	00	C0
----	----	----	----	----	----	----	----

UNMOUNT_HOST**E6**

PARAMS p0: host slot
 PAYLOAD none
 REPLY empty ACK

Closes the host's session and unmounts every disk slot that uses it. It is answered with NAK if the host was never mounted — and, on fujinet-pc, for the SD card, which cannot be unmounted:

REQUEST → FUJINET

C0	70	E6	07	00	66	01	07	C0
----	----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	70	06	06	00	7C	00	C0
----	----	----	----	----	----	----	----

REQUEST → FUJINET

C0	70	E6	07	00	5F	01	00	C0
----	----	----	----	----	----	----	----	----

REPLY · NAK

C0	70	15	06	00	8B	00	C0
----	----	----	----	----	----	----	----

SET_HOST_PREFIX**E1**

PARAMS p0: host slot
PAYLOAD 256 bytes: a directory path, NUL-terminated
REPLY empty ACK

Sets a working directory on the host. Later paths on that host that do not begin with `/` are taken relative to it.

REQUEST → FUJINET

C0	70	E1	07	01	4C	01	00	2F	74	65	73	74	00	00	00	00	00	00	00	...	243 more bytes	...	C0
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	----------------	-----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	70	06	06	00	7C	00	C0
----	----	----	----	----	----	----	----

GET_HOST_PREFIX**E0**

PARAMS p0: host slot
PAYLOAD none
REPLY 256 bytes: the prefix, NUL-terminated

Read only up to the first NUL. The rest of the 256 bytes is whatever was in the firmware's buffer, and if no prefix was ever set the reply is **entirely** leftover memory: set a prefix (even an empty one) before relying on this.

REQUEST → FUJINET

C0	70	E0	07	00	59	01	00	C0
----	----	----	----	----	----	----	----	----

REPLY · ACK · 256 PAYLOAD BYTES

C0	70	06	06	01	78	00	2F	74	65	73	74	00	00	00	35	00	00	00	00	00	...	242 more bytes	...	C0
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	----------------	-----	----

7.4 Browsing a host

Directories are read one entry at a time, like a file. The adapter keeps one directory open at a time, across all hosts.

OPEN_DIRECTORY**F7**

PARAMS p0: host slot
PAYLOAD 256 bytes: path, NUL, then an optional pattern and NUL
REPLY empty ACK

Opens a directory on a mounted host. A trailing `/` on the path is removed. To list only some files, put a wildcard pattern after the path's NUL: `/games` NUL `*.img` NUL. Opening a directory closes any that was open.

REQUEST → FUJINET

C0	70	F7	07	01	91	01	00	2F	74	65	73	74	2F	00	00	00	00	00	00	00	...	243 more bytes	...	C0
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	----------------	-----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	70	06	06	00	7C	00	C0
----	----	----	----	----	----	----	----

READ DIR ENTRY

F6

PARAMS	p0: length wanted (1 to 255) · p1: flags
PAYLOAD	none
REPLY	exactly p0 bytes

Reads the next entry. The reply is always exactly `p0` bytes: the file name, a NUL, and NUL padding. A directory's name ends with `/`. A name too long for the space is shortened with an ellipsis in the middle; ask for more bytes to avoid it. When the directory is used up, the reply begins with `7F 7F`:

REQUEST → FUJINET

C0	70	F6	08	00	95	02	24	00	C0
----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 36 PAYLOAD BYTES

[illegible]

REQUEST → FUJINET

C0	70	F6	08	00	95	02	24	00	C0
----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 36 PAYLOAD BYTES

[illegible]

Setting bit 7 of `p1` (`80`) puts 12 bytes of details in front of the name:

Offset	Size	Field
0	1	year – 1970 of the last modification
1	1	month, 1 to 12
2	1	day
3	1	hour
4	1	minute
5	1	second
6	4	size in bytes, little-endian
10	1	flags: 01 directory, 02 name was shortened
11	1	media type: 00 unknown, 01 disk image, 02 ROM, 03 IMD

REQUEST → FUJINET

C0	70	F6	08	00	22	02	30	80	C0
----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 48 PAYLOAD BYTES

C0	70	06	36	00	03	00	FF	0C	1F	12	00	00	00	00	00	00	00	64	69	...	34 more bytes ...	C0
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	-------------------	----

A host that keeps no modification times (fujinet-pc's SD directory is one) reports 31 December 1969, 18:00 — the year byte `FF` is `-1` — and a size of zero. The 12 bytes count against `p0`, so ask for 12 more.

Block mode

With the top two bits of `p1` both set (`C0` to `FF`), `READ_DIR_ENTRY` returns many entries at once, packed into pages. `p0` is the number of 256-byte pages wanted and the low six bits of `p1` the number of entries per group. The reply is exactly `p0 × 256` bytes:

Part	Layout
Page header	<code>4D 46 (MF)</code> , <code>04</code> , number of groups
Group header	5 bytes: flags (<code>80</code> = last group), entry count, data size (16 bits), group index
Each entry	8 bytes: year – 1970; directory flag in bit 7 with the month in the low nibble; day in bits 7–3 with hour bits 4–2 in bits 2–0; hour bits 1–0 in bits 7–6 with the minute in bits 5–0; size (24 bits); media type — then the name and a NUL

Directory names carry no trailing `/` in block mode; the flag bit says what they are. One request for page 1 of `/test` with up to 8 entries per group returned:

BLOCK-MODE REPLY (FIRST 80 OF 256 BYTES)

```

4D 46 04 01      'MF', version 4, 1 group
80 04 47 00 00    last group, 4 entries, 71 bytes, group 0
FF 0C FC 80 00 00 00 00 1969-12-31 18:00, size 0, type 0
62 6C 61 6E 6B 2E 69 6D 67 00 blank.img
FF 0C FC 80 00 00 00 00 ... copy.txt
...

```

GET_DIRECTORY_POSITION

E5

PARAMS none
PAYLOAD none
REPLY 2 bytes: the index of the next entry

REQUEST → FUJINET

`C0 70 E5 06 00 5C 00 C0`

REPLY · ACK · 2 PAYLOAD BYTES

`C0 70 06 08 00 7F 00 01 00 C0`

SET_DIRECTORY_POSITION

E4

PARAMS `p0`: entry index (16 bits)
PAYLOAD none
REPLY empty ACK

Moves to an entry; the next `READ_DIR_ENTRY` returns it. Together with `GET_DIRECTORY_POSITION` this lets a program page backwards through a long directory without reading it from the start.

REQUEST → FUJINET

`C0 70 E4 08 00 62 05 00 00 C0`

REPLY · ACK · 0 PAYLOAD BYTES

`C0 70 06 06 00 7C 00 C0`

CLOSE_DIRECTORY

F5

PARAMS

none

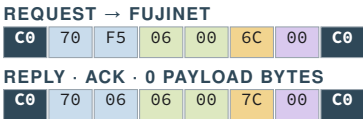
PAYLOAD

none

REPLY

empty ACK

Always answered with ACK.



7.5 Disk slots

A **disk slot** connects a disk image on a host to one of the eight disk devices: disk slot N is device `30h + N`. The slot table holds eight 38-byte records:

Offset	Size	Field
0	1	host slot, 0 to 7, or <code>FF</code> for an empty slot
1	1	mode: bit 0 read, bit 1 write, bit 6 (<code>40</code>) the image is mounted
2	36	the image's file name, NUL-terminated

The full path of an image can be up to 256 bytes, but the slot table holds only the **file name**, without its directory; GET_DEVICE_FULLPATH returns the whole path.

READ_DEVICE_SLOTS

F2

PARAMS

none

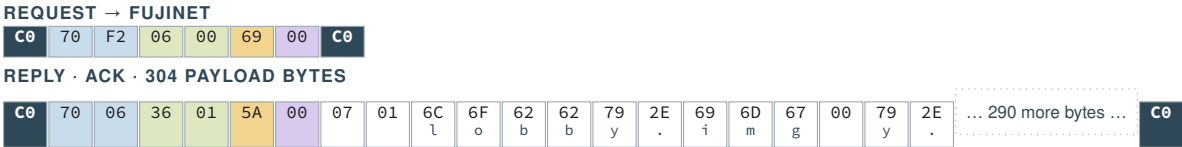
PAYLOAD

none

REPLY

304 bytes: 8 records of 38

In this reply, slot 1 is `07 01 lobby.img` : host slot 8, read-only, not mounted. Slot 2 is `00 41 disk.img` : host slot 1, read-only (`01`) and mounted (`40`). The bytes after a name's NUL are left over from longer names and mean nothing.



WRITE_DEVICE_SLOTS

F1

PARAMS

none

PAYLOAD

304 bytes: 8 records of 38

REPLY

empty ACK

Replaces the whole slot table and saves it. It does not mount anything.



CAUTION

Because READ_DEVICE_SLOTS returns only file names, writing its reply straight back with WRITE_DEVICE_SLOTS replaces every image's full path with its bare file name, and the images can no longer be found. To change one slot, use SET_DEVICE_FULLPATH instead.

SET_DEVICE_FULLPATH**E2**

PARAMS p0: disk slot · p1: host slot · p2: mode (01 read, 02 read/write)
PAYLOAD 256 bytes: the image's path, NUL-terminated
REPLY empty ACK

Puts an image in a disk slot, without mounting it, and saves the slot to `fnconfig.ini`. An empty path empties the slot.

REQUEST → FUJINET

C0	70	E2	09	01	9B	03	01	00	01	2F	74	65	73	74	2F	64	69	73	6B	2E	...	245 more bytes ...	C0
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	--------------------	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	70	06	06	00	7C	00	C0
----	----	----	----	----	----	----	----

GET_DEVICE_FULLPATH**DA**

PARAMS p0: disk slot
PAYLOAD none
REPLY 256 bytes: the full path, NUL-padded

REQUEST → FUJINET

C0	70	DA	07	00	54	01	01	C0
----	----	----	----	----	----	----	----	----

REPLY · ACK · 256 PAYLOAD BYTES

C0	70	06	06	01	B6	00	2F	74	65	73	74	2F	64	69	73	6B	2E	69	6D	67	...	242 more bytes ...	C0
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	--------------------	----

MOUNT_IMAGE**F8**

PARAMS p0: disk slot · p1: mode (01 read, 02 read/write)
PAYLOAD none
REPLY empty ACK

Opens the image named in the slot so the disk device can read and write it. The host must be mounted first. The adapter decides how to treat the file from its extension (Chapter 8); a file it cannot classify is refused with NAK.

REQUEST → FUJINET

C0	70	F8	08	00	75	02	01	01	C0
----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	70	06	06	00	7C	00	C0
----	----	----	----	----	----	----	----

UNMOUNT_IMAGE**E9**

PARAMS p0: disk slot
PAYLOAD none
REPLY empty ACK

Closes the image and empties the slot.

REQUEST → FUJINET

C0	70	E9	07	00	63	01	01	C0
----	----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	70	06	06	00	7C	00	C0
----	----	----	----	----	----	----	----

MOUNT_ALL**D7**

PARAMS none
 PAYLOAD none
 REPLY empty ACK

Mounts every host that a slot uses, then every slot that holds an image — what the firmware does by itself at power-up when `configenabled=0`. It stops at the first failure and answers NAK.

REQUEST → FUJINET

C0	70	D7	06	00	4E	00	C0
----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	70	06	06	00	7C	00	C0
----	----	----	----	----	----	----	----

NEW_DISK**E7**

PARAMS none
 PAYLOAD 262 bytes: sectors (16 bits), sector size (16 bits), host slot, disk slot, path (256)
 REPLY NAK

Meant to create a blank image of the given geometry and put it in the disk slot.

REQUEST → FUJINET

C0	70	E7	0C	01	D2	00	D0	02	00	02	00	02	2F	74	65	73	74	2F	62	6C	...	248 more bytes ...	C0
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	--------------------	----

REPLY · NAK

C0	70	15	06	00	8B	00	C0
----	----	----	----	----	----	----	----

CAUTION

On the RS-232 build **NEW_DISK** **always** answers NAK: the image-creation routine it calls is not implemented for this platform. It does create (or truncate!) the named file, empty, and it does change the disk slot's host and file name. Create blank images some other way.

7.6 Booting

CONFIG_BOOT**D9**

PARAMS p0: 01 keep the boot image, 00 drop it
 PAYLOAD none
 REPLY empty ACK

With `p0 = 0`, removes the boot image from disk slot 1 if no real image has been put there. Always answered with ACK.

REQUEST → FUJINET

C0	70	D9	07	00	53	01	01	C0
----	----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	70	06	06	00	7C	00	C0
----	----	----	----	----	----	----	----

SET_BOOT_MODE**D6**

PARAMS p0: boot mode, as [General] boot_mode
PAYLOAD none
REPLY empty ACK

Places the boot image for the given mode in disk slot 1 at once: 0 /autorun.img, 1 /mount-and-boot.img, 2 the lobby image on tnfs.fujinet.online, 3 /hisioboot-fujinet.img. Always answered with ACK, even if the image does not exist.

REQUEST → FUJINET · NOT SENT, COMPUTED

C0	70	D6	07	00	4F	01	00	C0
----	----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	70	06	06	00	7C	00	C0
----	----	----	----	----	----	----	----

RESET**FF**

PARAMS none
PAYLOAD none
REPLY none

Restarts the adapter at once, **without** replying. Wait a few seconds, and expect the firmware to discard input until the line has been quiet for 100 ms after it comes back.

REQUEST → FUJINET · NOT SENT, COMPUTED

C0	70	FF	06	00	76	00	C0
----	----	----	----	----	----	----	----

NO REPLY

7.7 Copying files

COPY_FILE**D8**

PARAMS p0: source host slot · p1: destination host slot — both counted from 1
PAYLOAD source|destination, NUL-terminated
REPLY empty ACK

Copies a file between hosts (or within one) on the adapter, without the bytes crossing the serial line. A destination ending in / gets the source file's name. Unlike every other command, the host slots here count from 1: p0 = 1 is host slot 1. The reply waits until the copy is done.

REQUEST → FUJINET

C0	70	D8	28	00	7E	02	01	01	2F	74	65	73	74	2F	72	65	61	64	6D	65	2E	74	78	74	7C	2F
74	65	73	74	2F	63	6F	70	79	2E	74	78	74	00	C0												

REPLY · ACK · 0 PAYLOAD BYTES

C0	70	06	06	00	7C	00	C0
----	----	----	----	----	----	----	----

7.8 Application keys

Application keys are small files the adapter keeps on its SD card for programs: a high-score table, a player name, a server address. A key is named by a creator number, an application number and a key number, and holds up to 64 bytes. It is stored on the SD card as /FujiNet/ccccaakk.key, the three numbers in hexadecimal, so keys need an SD card.

Every key operation starts with OPEN_APPKEY and a 6-byte description:

Offset	Size	Field
0	2	creator, little-endian (assign your own; 0 is not allowed)
2	1	application
3	1	key
4	1	mode: 00 read, 01 write
5	1	reserved, 0

OPEN_APPKEY

DC

PARAMS none
 PAYLOAD 6 bytes, above
 REPLY empty ACK

Answered with NAK if there is no SD card or the creator is 0.

REQUEST → FUJINET

C0	70	DC	0C	00	A2	00	34	12	01	01	01	00	C0
----	----	----	----	----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	70	06	06	00	7C	00	C0
----	----	----	----	----	----	----	----

WRITE_APPKEY

DE

PARAMS none
 PAYLOAD the key's contents, up to 64 bytes
 REPLY empty ACK

Needs an OPEN_APPKEY in mode 01 first, and closes the key afterwards.

REQUEST → FUJINET

C0	70	DE	16	00	46	00	48	49	47	48	20	53	43	4F	52	45	20	33	31	33	33	37	C0
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	70	06	06	00	7C	00	C0
----	----	----	----	----	----	----	----

READ_APPKEY

DD

PARAMS none
 PAYLOAD none
 REPLY 2-byte length, little-endian, then that many bytes

Needs an OPEN_APPKEY in mode 00 first. The RS-232 build puts a 2-byte length in front of the data, here 10 00, sixteen bytes.

REQUEST → FUJINET

C0	70	DD	06	00	54	00	C0
----	----	----	----	----	----	----	----

REPLY · ACK · 18 PAYLOAD BYTES

C0	70	06	18	00	7F	00	10	00	48	49	47	48	20	53	43	4F	52	45	20	33	31	33	33	37	C0
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

CAUTION

READ_APPKEY answers ACK even when it fails — no key, no SD card, not opened for reading — and then returns a length of 64 and 64 zero bytes. A key of 64 zeroes is therefore indistinguishable from no key; store something recognisable in your keys.

CLOSE_APPKEY**DB**

PARAMS none
 PAYLOAD none
 REPLY empty ACK

REQUEST → FUJINET

C0	70	DB	DD	06	00	52	00	C0
----	----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	70	06	06	00	7C	00	C0
----	----	----	----	----	----	----	----

7.9 Base64

The adapter can encode to and decode from Base64, a job too heavy for many small machines. Each direction is four steps: feed the input in pieces, compute, ask the length, and read the result in pieces. Encoding and decoding share **one** buffer, so finish one job before starting the next.

BASE64_ENCODE_INPUT**D0**

PARAMS p0: byte count (16 bits)
 PAYLOAD that many bytes
 REPLY empty ACK

Appends to the buffer. Send it as many times as needed.

REQUEST → FUJINET

C0	70	D0	0F	00	14	05	07	00	46	75	6A	69	4E	65	74	C0
									F	u	j	i	N	e	t	

REPLY · ACK · 0 PAYLOAD BYTES

C0	70	06	06	00	7C	00	C0
----	----	----	----	----	----	----	----

BASE64_ENCODE_COMPUTE**CF**

PARAMS none
 PAYLOAD none
 REPLY empty ACK

Replaces the buffer's contents with their Base64 encoding.

REQUEST → FUJINET

C0	70	CF	06	00	46	00	C0
----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	70	06	06	00	7C	00	C0
----	----	----	----	----	----	----	----

BASE64_ENCODE_LENGTH**CE**

PARAMS none
 PAYLOAD none
 REPLY 4 bytes: the length of the result, little-endian

The encoder leaves a NUL at the end of its result and this length counts it: `FujiNet` encodes to the 12 characters `RnVqaU5ldA==`, and the length is `0D`, 13.

REQUEST → FUJINET

C0	70	CE	06	00	45	00	C0
----	----	----	----	----	----	----	----

REPLY · ACK · 4 PAYLOAD BYTES

C0	70	06	0A	00	0D	00	0D	00	00	00	00	C0
----	----	----	----	----	----	----	----	----	----	----	----	----

BASE64_ENCODE_OUTPUT

PARAMS

p0: byte count (16 bits)

PAYLOAD

none

REPLY

that many bytes, taken from the front of the buffer

CD

Answered with NAK if `p0` is 0 or more than the buffer holds.

REQUEST → FUJINET

C0

70

CD

08

00

57

05

0C

00

C0

REPLY · ACK · 12 PAYLOAD BYTES

C0

70

06

12

00

89

00

52

6E

56

71

61

55

35

6C

64

41

3D

3D

C0

R

n

V

q

a

U

S

l

d

A

=

=

The decoding commands mirror them exactly:

Command	Code	Same as
BASE64_DECODE_INPUT	CC	ENCODE_INPUT, with Base64 text as input
BASE64_DECODE_COMPUTE	CB	ENCODE_COMPUTE
BASE64_DECODE_LENGTH	CA	ENCODE_LENGTH (no NUL is counted)
BASE64_DECODE_OUTPUT	C9	ENCODE_OUTPUT

REQUEST → FUJINET

C0

70

CC

14

00

63

05

0C

00

52

6E

56

71

61

55

35

6C

64

41

3D

3D

C0

R

n

V

q

a

U

S

l

d

A

=

=

REPLY · ACK · 0 PAYLOAD BYTES

C0

70

06

06

00

7C

00

C0

REQUEST → FUJINET

C0

70

C9

08

00

4E

05

07

00

C0

REPLY · ACK · 7 PAYLOAD BYTES

C0

70

06

0D

00

3B

00

46

75

6A

69

4E

65

74

C0

F

u

j

i

N

e

t

7.10 Hashing

The adapter computes MD5, SHA-1, SHA-256 and SHA-512 digests.

HASH_INPUT

PARAMS

p0: byte count (16 bits)

PAYLOAD

that many bytes

REPLY

empty ACK

C8

Appends to the data to be hashed.

REQUEST → FUJINET

C0

70

C8

0B

00

73

05

03

00

61

62

63

C0

a

b

c

REPLY · ACK · 0 PAYLOAD BYTES

C0

70

06

06

00

7C

00

C0

HASH_COMPUTE

PARAMS

p0: algorithm: 00 MD5, 01 SHA-1, 02 SHA-256, 03 SHA-512

PAYLOAD

none

REPLY

empty ACK

C7

Computes the digest and clears the input. HASH_COMPUTE_NO_CLEAR (`c3`) does the same but keeps the input, to compute a second digest of the same data.

REQUEST → FUJINET

C0	70	C7	07	00	41	01	01	C0
----	----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	70	06	06	00	7C	00	C0
----	----	----	----	----	----	----	----

HASH_LENGTH**C6**

PARAMS p0: 01 for hexadecimal text, 00 for binary
PAYLOAD none
REPLY 1 byte: the digest's length in that form

REQUEST → FUJINET

C0	70	C6	07	00	40	01	01	C0
----	----	----	----	----	----	----	----	----

REPLY · ACK · 1 PAYLOAD BYTE

C0	70	06	07	00	A5	00	28	C0
----	----	----	----	----	----	----	----	----

HASH_OUTPUT**C5**

PARAMS p0: 01 for hexadecimal text, 00 for binary
PAYLOAD none
REPLY the digest

SHA-1 of `abc`, as hexadecimal text. In binary the same digest is 20 bytes; here is the MD5 of `abc`, binary:

REQUEST → FUJINET

C0	70	C5	07	00	3F	01	01	C0
----	----	----	----	----	----	----	----	----

REPLY · ACK · 40 PAYLOAD BYTES

C0	70	06	2E	00	22	00	61	39	39	39	33	65	33	36	34	37	30	36	38	31	36	61	62	61	33	65
32	35	37	31	37	38	35	30	63	32	36	63	39	63	64	30	64	38	39	64	C0						

REQUEST → FUJINET

C0	70	C5	07	00	3E	01	00	C0
----	----	----	----	----	----	----	----	----

REPLY · ACK · 16 PAYLOAD BYTES

C0	70	06	16	00	3C	00	90	01	50	98	3C	D2	4F	B0	D6	96	3F	7D	28	E1	7F	72	C0
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

HASH_CLEAR**C2**

PARAMS none
PAYLOAD none
REPLY empty ACK

Clears the input without computing.

REQUEST → FUJINET

C0	70	C2	06	00	39	00	C0
----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	70	06	06	00	7C	00	C0
----	----	----	----	----	----	----	----

7.11 QR codes

The adapter turns text into a QR code and renders it in one of several forms, one of which is ready to print on an ANSI terminal.

QRCODE_INPUT**BC**

PARAMS p0: byte count (16 bits)
PAYLOAD that many bytes of text
REPLY empty ACK

REQUEST → FUJINET

C0	70	BC	1F	00	13	05	17	00	68	74	74	70	73	3A	2F	2F	66	75	6A	69	6E	65	74	2E	6F	6E
									h	t	t	p	s	:	/	/	f	u	j	i	n	e	t	.	o	n
6C	69	6E	65	2F	C0																					
l	i	n	e	/																						

REPLY · ACK · 0 PAYLOAD BYTES

C0	70	06	06	00	7C	00	C0
----	----	----	----	----	----	----	----

QRCODE_ENCODE**BD**

PARAMS p0: QR version (1 to 40) · p1: error correction: 00 L, 01 M, 02 Q, 03 H · p2: 01 to shorten a URL first
PAYLOAD none
REPLY empty ACK

Encodes the input and clears it. The version sets the size, and so the capacity: a 23-character URL needs version 3 at level L, and is refused with NAK at version 1.

REQUEST → FUJINET

C0	70	BD	09	00	3D	03	03	00	00	C0
----	----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	70	06	06	00	7C	00	C0
----	----	----	----	----	----	----	----

QRCODE_LENGTH**BE**

PARAMS p0: output form, below
PAYLOAD none
REPLY 4 bytes: the length of the rendered code

Renders the code in the chosen form and returns its length:

REQUEST → FUJINET

C0	70	BE	07	00	37	01	00	C0
----	----	----	----	----	----	----	----	----

REPLY · ACK · 4 PAYLOAD BYTES

C0	70	06	0A	00	EB	00	6B	00	00	00	C0
							k				

p0	Form	Layout
00	binary	the code's size in modules, then every module row by row, packed 8 to a byte, least significant bit first
01	ANSI	text with ANSI colour codes, two spaces per module, a newline per row: print it on an ANSI terminal
02	bitmap	the size, then each row packed into whole bytes, most significant bit leftmost
03	SVG	an SVG image as text
04	ATASCII	Atari text-mode graphics
05	PETSCII	Commodore text-mode graphics

QRCODE_OUTPUT**BF**

PARAMS p0: byte count (16 bits)
PAYLOAD none
REPLY that many bytes, taken from the front of the rendered code

A version 3 code is 29 modules square; in binary form that is one size byte (**1D**) and 106 bytes of modules.

REQUEST → FUJINET

C0	70	BF	08	00	A8	05	6B	00	C0
----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 107 PAYLOAD BYTES

C0	70	06	71	00	28	00	1D	7F	44	DE	3F	48	84	0B	76	25	02	DD	2E	99	...	94 more bytes ...	C0
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	-------------------	----

7.12 Random numbers and GUIDs**RANDOM_NUMBER****D3**

PARAMS none
PAYLOAD none
REPLY 4 bytes: a random 32-bit number, little-endian

REQUEST → FUJINET

C0	70	D3	06	00	4A	00	C0
----	----	----	----	----	----	----	----

REPLY · ACK · 4 PAYLOAD BYTES

C0	70	06	0A	00	CE	00	92	AB	DB	DC	4F	0	C0
----	----	----	----	----	----	----	----	----	----	----	----	---	----

GENERATE_GUID**BB**

PARAMS none
PAYLOAD none
REPLY 37 bytes: a version 4 UUID as text, and a NUL

REQUEST → FUJINET

C0	70	BB	06	00	32	00	C0
----	----	----	----	----	----	----	----

REPLY · ACK · 37 PAYLOAD BYTES

C0	70	06	2B	00	EE	00	32	66	34	63	32	37	66	33	2D	32	32	37	32	2D	34	66	66	32	2D	62	
62	32	31	2D	35	31	35	34	37	65	39	37	61	61	38	64	00	C0										

7.13 Commands not available on RS-232

These Fuji commands exist on other FujiNet platforms but are answered with NAK on the RS-232 bus: **ENABLE_UDPSTREAM** (**F0**), **SET_BAUDRATE** (**EB**), **SET_HSIO_INDEX** (**E3**), **SET_SIO_EXTERNAL_CLOCK** (**DF**), **ENABLE_DEVICE** (**D5**), **DISABLE_DEVICE** (**D4**), **DEVICE_ENABLE_STATUS** (**D1**), **GET_TIME** (**D2** ; use the clock device), **GET_HEAP** (**C1**), **GET_DEVICE1_FULLPATH** to **GET_DEVICE10_FULLPATH** (**A0** – **A9** ; use **GET_DEVICE_FULLPATH**), **UPDATE_FIRMWARE** (**90**) and **HSIO_INDEX** (**3F**).

CHAPTER 8

The Disk Drives (31h–38h)

Each of the eight disk slots appears on the bus as a disk drive: disk slot 1 is device 31, disk slot 8 is device 38. Once the Fuji device has mounted an image into a slot (Chapter 7), the drive reads and writes it sector by sector. A drive whose slot is empty answers every command with NAK.

8.1 What an image is

The firmware decides how to treat an image by the end of its file name:

Extension	Treated as	Sectors
.IMD	ImageDisk	The image’s own sector sizes, in logical order, sector 0 first. Sectors the image records as bad or deleted are reported as such.
.ROM, .BIN, .INT, .ITV, .CHF	cartridge ROM	Not readable as a disk: see below.
anything else	flat image	A plain sequence of 512-byte sectors, sector 0 at the start of the file.

Most S-100 and CP/M disk images are flat images of 128-byte sectors. The drive still serves them in 512-byte pieces: sector *n* is bytes 512*n* to 512*n* + 511 of the file, whatever the image’s own geometry. A program that wants 128-byte CP/M records reads the 512-byte sector that holds them and picks out the quarter it needs.

CAUTION

A ROM image is not a disk. Mounting one tells the adapter to send the whole ROM to a Fujiversal cartridge, so the firmware starts sending packets of its own — to device FF, the cartridge — and waits for each to be acknowledged. On an ordinary serial computer those packets arrive where the MOUNT_IMAGE reply was expected, nothing acknowledges them, and the mount fails. Do not mount .ROM, .BIN, .INT, .ITV or .CHF files on a computer that is not a Fujiversal cartridge.

8.2 Commands

READ

52

PARAMS

p0: sector number (32 bits), from 0

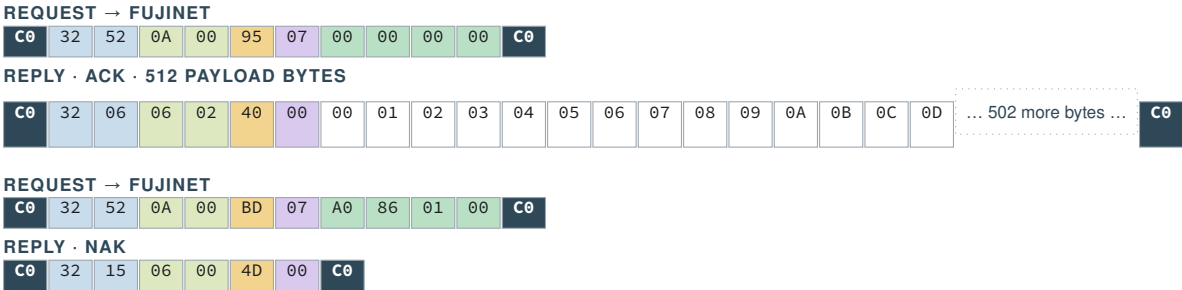
PAYLOAD

none

REPLY

the sector: 512 bytes for a flat image

Answered with NAK if the sector is past the end of the image or the slot is empty:



WRITE**57**

PARAMS p0: sector number (32 bits)
PAYLOAD the sector: 512 bytes for a flat image
REPLY empty ACK

Writes the sector. The image must be mounted read/write (MOUNT_IMAGE mode 02); a read-only mount answers NAK.

REQUEST → FUJINET

C0	32	57	0A	02	9D	07	01	00	00	00	00	01	02	03	04	05	06	07	08	09	...	506 more bytes ...	C0
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	--------------------	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	32	06	06	00	3E	00	C0
----	----	----	----	----	----	----	----

PUT**50**

PARAMS p0: sector number (32 bits)
PAYLOAD the sector
REPLY empty ACK

The same as WRITE. (On the Atari bus PUT skips the verify; on RS-232 the two are one routine.)

REQUEST → FUJINET

C0	32	50	0A	02	96	07	01	00	00	00	00	01	02	03	04	05	06	07	08	09	...	506 more bytes ...	C0
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	--------------------	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	32	06	06	00	3E	00	C0
----	----	----	----	----	----	----	----

STATUS**53**

PARAMS p0: sector number (32 bits)
PAYLOAD see below
REPLY NAK

On other FujiNet buses this returns the drive's four status bytes. On the RS-232 bus it is wired, by mistake, to **WRITE**: it takes a sector number and a payload and writes them. Sent without a payload it simply fails, as here; sent without a parameter it crashes the adapter (Chapter 4).

REQUEST → FUJINET

C0	32	53	0A	00	96	07	00	00	00	00	C0
----	----	----	----	----	----	----	----	----	----	----	----

REPLY · NAK

C0	32	15	06	00	4D	00	C0
----	----	----	----	----	----	----	----

CAUTION

Do not use disk STATUS on the RS-232 bus. There is nothing it can tell you that MOUNT_IMAGE and READ_DEVICE_SLOTS do not, and a stray payload would overwrite a sector.

FORMAT**21**

PARAMS none
 PAYLOAD none
 REPLY 128 bytes: FF FF , then zeroes

Returns an empty bad-sector list in the Atari style — the bytes FF FF padded to 128 — and **does not touch the image**. FORMAT_MEDIUM (22) is the same. An ImageDisk or ROM image answers NAK.

REQUEST → FUJINET

REPLY · ACK · 128 PAYLOAD BYTES

PERCOM_READ**4E**

PARAMS none
 PAYLOAD none
 REPLY 12 bytes: a PERCOM drive-geometry block

The Atari PERCOM block: tracks, step rate, sectors per track (16 bits, **big-endian**), sides, density, sector size (16 bits, big-endian), a “drive present” flag and three reserved bytes. A flat image reports all zeroes; an ImageDisk image reports its geometry.

REQUEST → FUJINET

REPLY · ACK · 12 PAYLOAD BYTES

PERCOM_WRITE**4F**

PARAMS none
 PAYLOAD 12 bytes
 REPLY empty ACK

Accepted and stored, with no effect on a flat image.

REQUEST → FUJINET

REPLY · ACK · 0 PAYLOAD BYTES

The Atari high-speed variants of these commands (D0 – D7 , A1 , A2 , 3F) are not recognised on RS-232 and are answered with NAK.

CHAPTER 9

The Network Units (71h–78h)

The eight network units are FujiNet's **N:** device: each opens one URL, and then reads and writes it like a file. Unit 1 is device **71**, unit 8 device **78**. One unit might hold a TCP connection to a bulletin board, another an HTTP request to a weather service, a third a file on a TNFS server — all at once, each independent.

The protocols themselves — HTTP and HTTPS, TCP and UDP, TNFS, FTP, SMB, NFS, SSH and SFTP, TELNET, WebSockets, S3, the Google and Microsoft cloud services, the SD card, and more — are described in depth in *The FujiNet Network Protocol Handbook*. This chapter covers the commands that drive them over the RS-232 bus.

9.1 The life of a connection

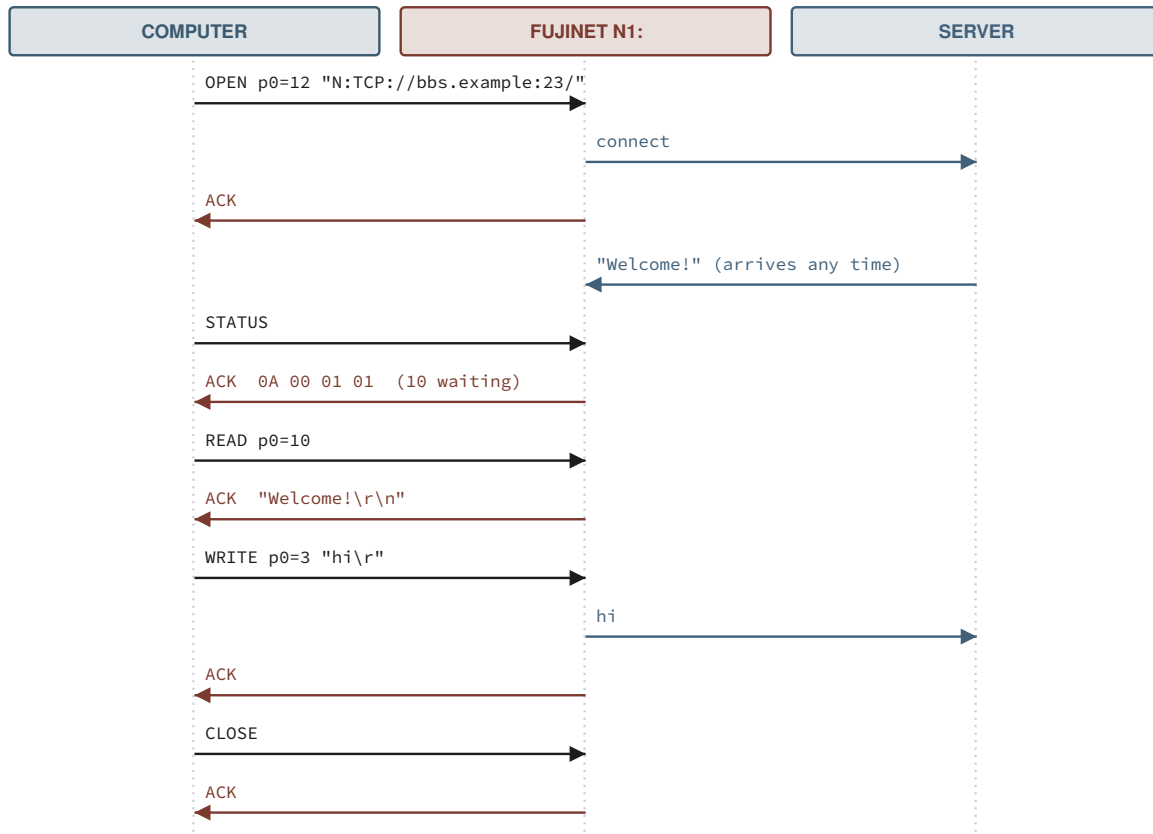


Figure 2. A unit's life: OPEN, then STATUS and READ to collect what has arrived, WRITE to send, CLOSE at the end.

Data from the far end collects in the unit's buffer as it arrives. The computer finds out how much is waiting with STATUS and collects it with READ, at whatever pace suits it; nothing is lost while it is busy.

9.2 The devicespec

A unit is opened on a **devicespec**, a URL with an optional **N:** prefix:

```
N:HTTPS://5card.carr-designs.com/tables?bin=1
```

The prefix may name a unit (**N2:**), but the unit that opens the URL is the device the packet is sent to, whatever the prefix says. The devicespec ends at its first NUL, or at a CR LF, and may be up to 255 characters.

9.3 Opening and closing

OPEN

PARAMS

payload: mode · p1: translation

PAYLOAD

the devicespec, NUL-terminated

REPLY

empty ACK

4F

Opens the URL. For most protocols the reply waits until the connection is made, so allow for DNS, TCP and TLS: several seconds. Opening a unit that is already open closes the old connection first. When OPEN fails it answers NAK, and STATUS then gives the reason in its error byte.

REQUEST → FUJINET

C0714F2A00E6020C004E3A485454503A2F2F3132372E302E302E31

3A373830322F66756A692E6A736F6E00C0

3A373830322F66756A692E6A736F6E00C0

REPLY · ACK · 0 PAYLOAD BYTES

C0710606007D00C0

The mode says what the program means to do:

p0	General meaning	For HTTP and HTTPS
04	read	GET
05	—	DELETE
06	read a directory listing	—
07	read a directory, alternate format	—
08	write	PUT
09	append	DELETE, headers allowed
0C	read and write	GET, headers allowed
0D	—	POST
0E	—	PUT, headers allowed

“Headers allowed” modes let a program use SET_HTTP_MODE (below) to send and collect HTTP headers; mode **0C** is the usual choice for a GET.

The **translation** in **p1** converts line endings as data passes through the unit: **00** none, **01** CR, **02** LF, **03** CR LF, **04** PETSCII. The RS-232 bus’s own line ending is CR LF. Programs that handle bytes themselves, like the ones in this book, use **00** .

CLOSE

PARAMS

none

PAYLOAD

none

REPLY

empty ACK

43

Closes the connection and frees the unit.

REQUEST → FUJINET

C071430600BA00C0

REPLY · ACK · 0 PAYLOAD BYTES

C0710606007D00C0

9.4 Reading and writing

STATUS

53

PARAMS	none
PAYLOAD	none
REPLY	4 bytes: bytes waiting (16 bits), connected, error

The heartbeat of every network program. The first two bytes are the number of bytes waiting to be read, up to 65,535. The third is `01` while the connection is up. The fourth is a status code from the table at the end of this chapter: `01` all is well, `88` (136) end of file.

REQUEST → FUJINET

C0	71	53	06	00	CA	00	C0
----	----	----	----	----	----	----	----

REPLY · ACK · 4 PAYLOAD BYTES

C0	71	06	0A	00	99	00	16	00	01	01	C0
----	----	----	----	----	----	----	----	----	----	----	----

IMPORTANT

How a program knows it has everything depends on the protocol. For TCP and TELNET, the connection is over when nothing is waiting and the **connected** byte is `00`. For HTTP, the connected byte is `00` from the start; the reply is complete when nothing is waiting and the **error** byte is `88`, end of file. A program that serves both, like NETCAT, should stop on either.

Here is the end of an HTTP reply — nothing waiting, not connected, end of file:

REQUEST → FUJINET

C0	71	53	06	00	CA	00	C0
----	----	----	----	----	----	----	----

REPLY · ACK · 4 PAYLOAD BYTES

C0	71	06	0A	00	0A	00	00	00	00	88	C0
----	----	----	----	----	----	----	----	----	----	----	----

READ

52

PARAMS	p0: most bytes wanted (16 bits)
PAYLOAD	none
REPLY	up to p0 bytes

Returns what is waiting, up to `p0` bytes; ask for the number STATUS reported, or less. A READ when nothing is waiting is answered with NAK, and `p0 = 0` is refused the same way. A READ cannot be repeated: the bytes it returned have left the unit's buffer.

REQUEST → FUJINET

C0	71	52	08	00	E6	05	16	00	C0
----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 22 PAYLOAD BYTES

C0	71	06	1C	00	4C	00	48	45	4C	4C	4F	20	46	52	4F	4D	20	31	32	37	2E	30	2E	30	2E	31
							H	E	L	L	O		F	R	O	M		1	2	7	.	0	.	0	.	1
0D	0A	C0																								

WRITE

57

PARAMS	p0: byte count (16 bits)
PAYLOAD	that many bytes
REPLY	empty ACK

Sends the bytes to the far end.

REQUEST → FUJINET

C0	71	57	0C	00	C6	05	04	00	68	69	0D	0A	C0
									h	i			

REPLY · ACK · 0 PAYLOAD BYTES

C0	71	06	06	00	7D	00	C0
----	----	----	----	----	----	----	----

9.5 A web request, start to finish

An HTTP GET of a small JSON document, as every byte goes:

OPEN mode 0C (GET), translation 00 :

REQUEST → FUJINET

C0	71	4F	2A	00	E6	02	0C	00	4E	3A	48	54	54	50	3A	2F	2F	31	32	37	2E	30	2E	30	2E	31
									N	:	H	T	T	P	:	/	/	1	2	7	.	0	.	0	.	1
3A	37	38	30	32	2F	66	75	6A	69	2E	6A	73	6F	6E	00	C0										
:	7	8	0	2	/	f	u	j	i	.	j	s	o	n												

REPLY · ACK · 0 PAYLOAD BYTES

C0	71	06	06	00	7D	00	C0
----	----	----	----	----	----	----	----

STATUS: 53 bytes waiting (35 00); not connected, status OK:

REQUEST → FUJINET

C0	71	53	06	00	CA	00	C0
----	----	----	----	----	----	----	----

REPLY · ACK · 4 PAYLOAD BYTES

C0	71	06	0A	00	B7	00	35	00	00	01	C0
							5				

READ all 53:

REQUEST → FUJINET

C0	71	52	08	00	06	05	35	00	C0
----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 53 PAYLOAD BYTES

C0	71	06	3B	00	DC	00	7B	22	6E	61	6D	65	22	3A	22	46	75	6A	69	4E	... 39 more bytes ...				C0
							{	"	n	a	m	e	"	:	"	F	u	j	i	N					

STATUS: nothing left, end of file (88):

REQUEST → FUJINET

C0	71	53	06	00	CA	00	C0
----	----	----	----	----	----	----	----

REPLY · ACK · 4 PAYLOAD BYTES

C0	71	06	0A	00	0A	00	00	00	00	88	C0
----	----	----	----	----	----	----	----	----	----	----	----

CLOSE:

REQUEST → FUJINET

C0	71	43	06	00	BA	00	C0
----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	71	06	06	00	7D	00	C0
----	----	----	----	----	----	----	----

9.6 Posting data

A POST sends a body to the server before reading its reply. Open in mode 0D, switch the unit to “post data” with SET_HTTP_MODE, WRITE the body, switch back to “body”, and read the reply as usual:

OPEN mode 0D (POST):

REQUEST → FUJINET

C0	71	4F	26	00	68	02	0D	00	4E	3A	48	54	54	50	3A	2F	2F	31	32	37	2E	30	2E	30	2E	31
									N	:	H	T	T	P	:	/	/	1	2	7	.	0	.	0	.	1
3A	37	38	30	32	2F	73	63	6F	72	65	00	C0														
:	7	8	0	2	/	s	c	o	r	e																

REPLY · ACK · 0 PAYLOAD BYTES

C0	71	06	06	00	7D	00	C0
----	----	----	----	----	----	----	----

SET_HTTP_MODE p1 = 4, post data:

REQUEST → FUJINET

C0	71	4D	08	00	CC	02	00	04	C0
----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	71	06	06	00	7D	00	C0
----	----	----	----	----	----	----	----

WRITE the body:

REQUEST → FUJINET

C0	71	57	11	00	4A	05	09	00	41	4C	54	41	49	52	3D	34	32	C0
									A	L	T	A	I	R	=	4	2	

REPLY · ACK · 0 PAYLOAD BYTES

C0	71	06	06	00	7D	00	C0
----	----	----	----	----	----	----	----

SET_HTTP_MODE p1 = 0, body — this sends the request:

REQUEST → FUJINET

C0	71	4D	08	00	C8	02	00	00	C0
----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	71	06	06	00	7D	00	C0
----	----	----	----	----	----	----	----

READ the server's answer:

REQUEST → FUJINET

C0	71	52	08	00	DD	05	0D	00	C0
----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 13 PAYLOAD BYTES

C0	71	06	13	00	F7	00	47	4F	54	20	41	4C	54	41	49	52	3D	34	32	C0
							G	O	T		A	L	T	A	I	R	=	4	2	

SET_HTTP_MODE

4D

PARAMS p0: unused, but must be sent · p1: channel mode
PAYLOAD none
REPLY empty ACK

Chooses what READ and WRITE mean on an HTTP unit: 00 the body (the default), 01 collect response headers, 02 read the collected headers, 03 write request headers, 04 write POST data. NAK on a unit that is not HTTP.

REQUEST → FUJINET

C0	71	4D	08	00	C8	02	00	00	C0
----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	71	06	06	00	7D	00	C0
----	----	----	----	----	----	----	----

9.7 Parsing JSON

A unit can parse what it has read as JSON (or HTML, or XML) and hand back single values by path, so a small computer never has to hold the whole document. After OPEN:

SET_PARSER p1 = 1, JSON:

REQUEST → FUJINET

C0	71	FC	08	00	79	02	00	01	C0
----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	71	06	06	00	7D	00	C0
----	----	----	----	----	----	----	----

PARSE — reads the whole reply and parses it:

REQUEST → FUJINET

C0	71	50	06	00	C7	00	C0
----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	71	06	06	00	7D	00	C0
----	----	----	----	----	----	----	----

QUERY /name :

REQUEST → FUJINET

C0	71	51	0B	00	9F	00	2F	6E	61	6D	65	C0
							/	n	a	m	e	

REPLY · ACK · 0 PAYLOAD BYTES

C0	71	06	06	00	7D	00	C0
----	----	----	----	----	----	----	----

STATUS: the value is 8 bytes long:

REQUEST → FUJINET

C0	71	53	06	00	CA	00	C0
----	----	----	----	----	----	----	----

REPLY · ACK · 4 PAYLOAD BYTES

C0	71	06	0A	00	8A	00	08	00	00	01	C0
----	----	----	----	----	----	----	----	----	----	----	----

READ it:

REQUEST → FUJINET

C0	71	52	08	00	D8	05	08	00	C0
----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 8 PAYLOAD BYTES

C0	71	06	0E	00	47	00	46	75	6A	69	4E	65	74	0A	C0
							F	u	j	i	N	e	t		

The value comes back with the parser’s line ending appended, a line feed by default. Array elements are addressed by index: `/ports/0` returned `1985` and a line feed.

SET_PARSER

FC

PARAMS

p0: unused, but must be sent · p1: 00 none, 01 JSON, 02 HTML, 03 XML

PAYLOAD

none

REPLY

empty ACK

REQUEST → FUJINET

C0	71	FC	08	00	79	02	00	01	C0
----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	71	06	06	00	7D	00	C0
----	----	----	----	----	----	----	----

PARSE

50

PARAMS

none

PAYLOAD

none

REPLY

empty ACK

Reads the rest of the unit’s data and parses it. NAK if nothing is open or the document will not parse.

REQUEST → FUJINET

C0	71	50	06	00	C7	00	C0
----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	71	06	06	00	7D	00	C0
----	----	----	----	----	----	----	----

QUERY

51

PARAMS

none

PAYLOAD

the path, such as `/0/account/display_name`

REPLY

empty ACK

Selects a value. STATUS then reports its length, and READ returns it.

REQUEST → FUJINET

C0	71	51	0B	00	9F	00	2F	6E	61	6D	65	C0
							/	n	a	m	e	

REPLY · ACK · 0 PAYLOAD BYTES

C0	71	06	06	00	7D	00	C0
----	----	----	----	----	----	----	----

SET_PARAMETER

FB

PARAMS

p0: 00 query flags, or 01 line ending · p1: the value

PAYLOAD

none

REPLY

empty ACK

With `p0 = 1`, `p1` becomes the line ending appended to query results. With `p0 = 0`, `p1` holds query flags: bits 0–2 remap characters, and bits 4–5 choose the output, `00` verbatim or `10` plain ASCII with entities decoded and everything else stripped. NAK if no parser is set.

REQUEST → FUJINET

C0	71	FB	08	00	78	02	01	00	C0
----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	71	06	06	00	7D	00	C0
----	----	----	----	----	----	----	----

9.8 Line endings and translation

TRANSLATION

54

PARAMS p0: unused, but must be sent · p1: translation, as for OPEN
PAYLOAD none
REPLY empty ACK

Sets a translation that is combined with OPEN's **p1** from the next OPEN on; **FF** turns translation off whatever OPEN asks.

REQUEST → FUJINET

C0	74	54	08	00	D5	02	00	03	C0
----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	74	06	06	00	80	00	C0
----	----	----	----	----	----	----	----

SET_EOL

4C

PARAMS none
PAYLOAD the line ending to use; empty for the default
REPLY empty ACK

Overrides the line ending translation converts to.

REQUEST → FUJINET

C0	74	4C	08	00	DF	00	0D	0A	C0
----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	74	06	06	00	80	00	C0
----	----	----	----	----	----	----	----

9.9 Files and directories

Units opened on file protocols (TNFS, FTP, SMB, NFS, SFTP, the SD card) support moving around and changing files. These commands take the full devicespec in the payload; the unit need not be open.

RENAME

20

PARAMS p0: mode · p1: unused — both must be sent
PAYLOAD N:proto://host/path/old,new , NUL-terminated
REPLY empty ACK

REQUEST → FUJINET

C0	74	20	25	00	E8	02	00	00	4E	3A	53	44	3A	2F	74	65	73	74	2F	6E	6F	74	65	2E	74	78
									N	:	S	D	:	/	t	e	s	t	/	n	o	t	e	.	t	x
74	2C	6D	65	6D	6F	2E	74	78	74	00																
t	,	m	e	m	o	.	t	x	t																	

REPLY · ACK · 0 PAYLOAD BYTES

C0	74	06	06	00	80	00	C0
----	----	----	----	----	----	----	----

The others have the same shape:

Command	Code	Payload
DELETE	21	the file's devicespec
LOCK	23	the file's devicespec (where the protocol supports it)
UNLOCK	24	the file's devicespec
MKDIR	2A	the new directory's devicespec
RMDIR	2B	the directory's devicespec

REQUEST → FUJINET	
C0	74 2A 17 00 7D 02 00 00 4E 3A 53 44 3A 2F 74 65 73 74 2F 73 75 62 00 C0
REPLY · ACK · 0 PAYLOAD BYTES	
C0	74 06 06 00 80 00 C0

A directory is read by opening it in mode 06 : READ then returns a text listing, one entry per line.

READ on N:SD:/test/ , opened in mode 06 :	
REQUEST → FUJINET	
C0	74 52 08 00 80 05 AC 00 C0
REPLY · ACK · 172 PAYLOAD BYTES	
C0	74 06 B2 00 3A 00 42 4C 41 4E 4B 20 20 20 20 49 4D 47 20 20 ... 158 more bytes ... C0

CHDIR	2C
PARAMS	none
PAYLOAD	a path, . . . , / , or a whole devicespec
REPLY	empty ACK

Sets the unit's working directory, against which relative devicespecs are taken.

REQUEST → FUJINET	
C0	74 2C 21 00 60 00 4E 3A 54 4E 46 53 3A 2F 2F 65 63 2E 74 6E 66 73 2E 69 6F 2F
	6D 73 64 6F 73 2F 00 C0
	m s d o s /
REPLY · ACK · 0 PAYLOAD BYTES	
C0	74 06 06 00 80 00 C0

GETCWD	30
PARAMS	none
PAYLOAD	none
REPLY	the working directory, with no NUL

REQUEST → FUJINET	
C0	74 30 06 00 AA 00 C0
REPLY · ACK · 24 PAYLOAD BYTES	
C0	74 06 1E 00 AE 00 54 4E 46 53 3A 2F 2F 65 63 2E 74 6E 66 73 2E 69 6F 2F 6D 73
	64 6F 73 2F C0
	d o s /

USERNAME**FD**

PARAMS none
PAYLOAD the user name, NUL-terminated
REPLY empty ACK

Sets the login for the next OPEN. PASSWORD (**FE**) sets the password the same way.

REQUEST → FUJINET

C0	74	FD	0D	00	FE	00	61	6C	74	61	69	72	00	C0
							a	l	t	a	i	r		

REPLY · ACK · 0 PAYLOAD BYTES

C0	74	06	06	00	80	00	C0
----	----	----	----	----	----	----	----

SEEK**25**

PARAMS p0: offset from the start (32 bits)
PAYLOAD none
REPLY empty ACK

REQUEST → FUJINET

C0	71	25	0A	00	B1	07	0A	00	00	00	C0
----	----	----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	71	06	06	00	7D	00	C0
----	----	----	----	----	----	----	----

TELL**26**

PARAMS none
PAYLOAD none
REPLY 4 bytes: the current offset

After reading all 53 bytes of a document, the offset is 53. On a unit that is not open, TELL returns four bytes of leftover memory.

REQUEST → FUJINET

C0	71	26	06	00	9D	00	C0
----	----	----	----	----	----	----	----

REPLY · ACK · 4 PAYLOAD BYTES

C0	71	06	0A	00	B6	00	35	00	00	00	C0
							5				

9.10 Serving: TCP servers and UDP

A TCP server

A unit opened on **N:TCP://:port/** — no host — listens. STATUS reports a client waiting as *connected*, ACCEPT takes it, and from then on READ and WRITE talk to that client:

OPEN N:TCP://:7803/ :**REQUEST → FUJINET**

C0	72	4F	17	00	2C	02	0C	00	4E	3A	54	43	50	3A	2F	2F	3A	37	38	30	33	2F	00	C0
									N	:	T	C	P	:	/	/	:	7	8	0	3	/		

REPLY · ACK · 0 PAYLOAD BYTES

C0	72	06	06	00	7E	00	C0
----	----	----	----	----	----	----	----

ACCEPT, once STATUS shows a caller:

REQUEST → FUJINET

C0	72	41	06	00	B9	00	C0
----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	72	06	06	00	7E	00	C0
----	----	----	----	----	----	----	----

READ what the client sent:

REQUEST → FUJINET

C0	72	52	08	00	D7	05	06	00	C0
----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 6 PAYLOAD BYTES

C0	72	06	0C	00	CA	00	50	49	4E	47	0D	0A	C0
							P	I	N	G			

CLOSE_CLIENT

63

PARAMS none
 PAYLOAD none
 REPLY empty ACK

Hangs up on the current client and keeps listening for the next.

REQUEST → FUJINET

C0	72	63	06	00	DB	DD	00	C0
----	----	----	----	----	----	----	----	----

REPLY · NAK

C0	72	15	06	00	8D	00	C0
----	----	----	----	----	----	----	----

CAUTION

CLOSE_CLIENT's answer is inverted. It disconnects the client and then answers **NAK** (as above); it answers **ACK** when there was no client to disconnect. Treat either reply as done, and check STATUS.

UDP

A unit opened on `N:UDP://host:port/` sends datagrams to that host; opened on `N:UDP://:port/` it listens on the port. Each READ returns one datagram.

OPEN `N:UDP://:7804/`, listening:

REQUEST → FUJINET

C0	73	4F	17	00	30	02	0C	00	4E	3A	55	44	50	3A	2F	2F	3A	37	38	30	34	2F	00	C0
									N	:	U	D	P	:	/	/	:	7	8	0	4	/		

REPLY · ACK · 0 PAYLOAD BYTES

C0	73	06	06	00	7F	00	C0
----	----	----	----	----	----	----	----

READ one datagram (STATUS shows 0 waiting, see below):

REQUEST → FUJINET

C0	73	52	08	00	13	05	40	00	C0
----	----	----	----	----	----	----	----	----	----

REPLY · ACK · 4 PAYLOAD BYTES

C0	73	06	0A	00	B2	00	50	49	4E	47	C0
							P	I	N	G	

GET_REMOTE — who sent it:

REQUEST → FUJINET

C0	73	72	06	00	EB	00	C0
----	----	----	----	----	----	----	----

REPLY · ACK · 256 PAYLOAD BYTES

C0	73	06	06	01	B2	00	31	32	37	2E	30	2E	30	2E	31	3A	37	38	30	35	...	243 more bytes ...	C0
							1	2	7	.	0	.	0	.	1	:	7	8	0	5			

SET_DESTINATION `N:127.0.0.1:7805` — answer them:

REQUEST → FUJINET

C0	73	44	17	00	1D	00	4E	3A	31	32	37	2E	30	2E	30	2E	31	3A	37	38	30	35	00	C0
							N	:	1	2	7	.	0	.	0	.	1	:	7	8	0	5		

REPLY · ACK · 0 PAYLOAD BYTES

C0	73	06	06	00	7F	00	C0
----	----	----	----	----	----	----	----

WRITE the answer:

REQUEST → FUJINET

C0	73	57	0C	00	15	05	04	00	50	4F	4E	47	C0
									P	O	N	G	

REPLY · ACK · 0 PAYLOAD BYTES

C0	73	06	06	00	7F	00	C0
----	----	----	----	----	----	----	----

IMPORTANT

STATUS does not count waiting datagrams on fujinet-pc: it reports 0 bytes even when a datagram has arrived. READ anyway, with a buffer big enough for the largest datagram expected; it returns the datagram if there is one. STATUS also points the unit's destination at whoever sent the last datagram, so a server can simply WRITE its reply.

GET_REMOTE**72**

PARAMS none
PAYLOAD none
REPLY 256 bytes: `address:port` , ended by `9B`

The sender of the last datagram, as text. The text ends with the byte `9B` (the Atari end-of-line), not a NUL; the rest of the 256 bytes is leftover memory. NAK on the ESP32 hardware, where it is not implemented.

REQUEST → FUJINET

C0	73	72	06	00	EB	00	C0
----	----	----	----	----	----	----	----

REPLY · ACK · 256 PAYLOAD BYTES

C0	73	06	06	01	B2	00	31	32	37	2E	30	2E	30	2E	31	3A	37	38	30	35	...	243 more bytes ...	C0
							1	2	7	.	0	.	0	.	1	:	7	8	0	5			

SET_DESTINATION**44**

PARAMS none
PAYLOAD `prefix:host:port` and a NUL
REPLY empty ACK

Where the next WRITE goes. The text must have a prefix before the host (`N:` will do), because the host is taken from between the first and last colons, and it must end with a NUL: without one, fujinet-pc reads past the end of the payload.

REQUEST → FUJINET

C0	73	44	17	00	1D	00	4E	3A	31	32	37	2E	30	2E	30	2E	31	3A	37	38	30	35	00	C0
							N	:	1	2	7	.	0	.	0	.	1	:	7	8	0	5		

REPLY · ACK · 0 PAYLOAD BYTES

C0	73	06	06	00	7F	00	C0
----	----	----	----	----	----	----	----

9.11 Interrupt rate**SET_INT_RATE****5A**

PARAMS p0: milliseconds (8 bits)
PAYLOAD none
REPLY empty ACK

How often the unit toggles the RI attention line while it needs attention (Chapter 2). The default is 100 ms on the hardware and 20 ms on fujinet-pc.

REQUEST → FUJINET

C0	74	5A	07	00	3B	01	64	C0
----	----	----	----	----	----	----	----	----

REPLY · ACK · 0 PAYLOAD BYTES

C0	74	06	06	00	80	00	C0
----	----	----	----	----	----	----	----

9.12 Status codes

The fourth byte of STATUS:

Code	Hex	Meaning
1	01	success
131	83	opened write-only
132	84	invalid command
135	87	opened read-only
136	88	end of file
138	8A	general timeout
144	90	general error
146	92	not implemented
151	97	file exists
162	A2	no space on device
165	A5	invalid devicespec
166	A6	invalid position
167	A7	access denied
170	AA	file not found
200	C8	connection refused
201	C9	network unreachable
202	CA	socket timeout
203	CB	network down
204	CC	connection reset
205	CD	connection already in progress
206	CE	address in use
207	CF	not connected
208	D0	server not running
209	D1	no connection waiting
210	D2	service not available
211	D3	connection aborted
212	D4	invalid user name or password
213	D5	could not parse JSON
214	D6	general client error
215	D7	general server error
255	FF	could not allocate buffers

An HTTP OPEN to a port where nothing listens is accepted — HTTP connects lazily — and the failure shows up in the first STATUS, as **CF** :

REQUEST → FUJINET

C0 74 53 06 00 CD 00 C0

REPLY · ACK · 4 PAYLOAD BYTES

C0 74 06 0A 00 54 00 00 00 00 CF C0

9.13 Commands not available on RS-232

GET_DSTATS_VALUE (**FF**), SET_UNIT (**FA**), SET_HSIO_INDEX (**E3**), PARSE_ALT (**80**), QUERY_ALT (**81**), GET_ERROR (**45**) and HSIO_INDEX (**3F**) are answered with NAK. Use STATUS for the error code.

CHAPTER 10

The Clock (45h)

The adapter sets its clock from the network at start-up (from the `[Network] sntpserver`, `pool.ntp.org` by default) and offers the time in a dozen formats, so that each computer can have it in the shape its own system software expects.

Every clock command takes one parameter. With `p0 = 1`, the commands that honour it give the time in the **alternate** time zone (set with `SETTZ`, below) instead of the adapter's own (`[General] timezone`). Send `p0 = 0` otherwise — but always send it (Chapter 4).

10.1 Reading the time

GETTIME

93

PARAMS p0: 00 system zone, 01 alternate zone
PAYLOAD none
REPLY 6 bytes: day, month, year – 2000, hour, minute, second

The APETime format, binary. Here: day 4, month 10, 2026, at 00:48:15 UTC.

REQUEST → FUJINET

C0 45 93 07 00 E0 01 00 C0

REPLY · ACK · 6 PAYLOAD BYTES

C0 45 06 0C 00 BE 00 04 0A 1A 00 30 0F C0

GETTZTIME

9A

PARAMS p0: unused
PAYLOAD none
REPLY 6 bytes, as GETTIME, always in the alternate zone

REQUEST → FUJINET

C0 45 9A 07 00 E7 01 00 C0

REPLY · ACK · 6 PAYLOAD BYTES

C0 45 06 0C 00 D0 00 03 0A 1A 13 30 0F C0

GET_SIMPLE

54

PARAMS p0: zone
PAYLOAD none
REPLY 7 bytes: century, year, month, day, hour, minute, second

The firmware calls this command `SETTZ_ALT2`: on buses where reading and writing use separate commands it sets the time zone. On RS-232 the read table claims it first, so it is a **read**: the time as plain binary, with a full century byte (14 is 20).

REQUEST → FUJINET

C0 45 54 07 00 A1 01 00 C0

REPLY · ACK · 7 PAYLOAD BYTES

C0 45 06 0D 00 D3 00 14 1A 0A 04 00 30 0F C0

GET_SIMPLE_HUNDREDTHS**4D**

PARAMS p0: zone
PAYLOAD none
REPLY 8 bytes: as GET_SIMPLE, then hundredths of a second, 0 to 99

REQUEST → FUJINET

C0	45	4D	07	00	9A	01	00	C0
----	----	----	----	----	----	----	----	----

REPLY · ACK · 8 PAYLOAD BYTES

C0	45	06	0E	00	F0	00	14	1A	0A	04	00	30	0F	1C	C0
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

GET_PRODOS**50**

PARAMS p0: zone
PAYLOAD none
REPLY 4 bytes: the ProDOS date and time

Byte 0 is the day plus the month times 32; byte 1 is the year (two digits) times 2 plus the month's top bit; byte 2 the minute; byte 3 the hour.

REQUEST → FUJINET

C0	45	50	07	00	9D	01	00	C0
----	----	----	----	----	----	----	----	----

REPLY · ACK · 4 PAYLOAD BYTES

C0	45	06	0A	00	FE	00	44	35	30	00	C0
----	----	----	----	----	----	----	----	----	----	----	----

GET_SOS**53**

PARAMS p0: zone
PAYLOAD none
REPLY 19 bytes: `YYYYMMDD0HHMSS000` and a NUL

The Apple III SOS format, as text. The 0 after the date is fixed.

REQUEST → FUJINET

C0	45	53	07	00	A0	01	00	C0
----	----	----	----	----	----	----	----	----

REPLY · ACK · 19 PAYLOAD BYTES

C0	45	06	19	00	E8	00	32	30	32	36	31	30	30	34	30	30	30	34	38	31	35	30	30	30	00	C0
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

GET_ISO_LOCAL**49**

PARAMS p0: zone
PAYLOAD none
REPLY 25 bytes: `YYYY-MM-DDTHH:MM:SS+HHMM` and a NUL

ISO 8601 with the zone's offset. In the alternate zone, after SETTZ set it to US Central time:

REQUEST → FUJINET

C0	45	49	07	00	96	01	00	C0
----	----	----	----	----	----	----	----	----

REPLY · ACK · 25 PAYLOAD BYTES

C0	45	06	1F	00	3D	00	32	30	32	36	2D	31	30	2D	30	34	54	30	30	3A	34	38	3A	31	35	2B
30	0	30	0	30	0	00	C0																			

REQUEST → FUJINET

C0	45	49	07	00	97	01	01	C0
----	----	----	----	----	----	----	----	----

REPLY · ACK · 25 PAYLOAD BYTES

C0	45	06	1F	00	4D	00	32	30	32	36	2D	31	30	2D	30	33	54	31	39	3A	34	38	3A	31	35	2D
30	0	5	0	0	0	C0																				

GET_ISO_UTC

5A

PARAMS p0: unused
 PAYLOAD none
 REPLY 25 bytes: ISO 8601 in UTC, offset +0000

REQUEST → FUJINET

C0	45	5A	07	00	A7	01	00	C0
----	----	----	----	----	----	----	----	----

REPLY · ACK · 25 PAYLOAD BYTES

C0	45	06	1F	00	3D	00	32	30	32	36	2D	31	30	2D	30	34	54	30	30	3A	34	38	3A	31	35	2B
30	0	0	0	0	0	C0																				

10.2 Time zones

A time zone is a POSIX TZ string: UTC0 , CST6CDT,M3.2.0,M11.1.0 , CET-1CEST,M3.5.0,M10.5.0/3 .

GET_GENERAL

47

PARAMS p0: unused
 PAYLOAD none
 REPLY the system time zone and a NUL

An empty or malformed zone reads as UTC .

REQUEST → FUJINET

C0	45	47	07	00	94	01	00	C0
----	----	----	----	----	----	----	----	----

REPLY · ACK · 4 PAYLOAD BYTES

C0	45	06	0A	00	42	00	55	54	43	00	C0
							U	T	C		

GETTZ_LEN

4C

PARAMS p0: unused
 PAYLOAD none
 REPLY 1 byte: the length of the system zone, plus 1 for the NUL

REQUEST → FUJINET

C0	45	4C	07	00	99	01	00	C0
----	----	----	----	----	----	----	----	----

REPLY · ACK · 1 PAYLOAD BYTE

C0	45	06	07	00	56	00	04	C0
----	----	----	----	----	----	----	----	----

SETTZ

PARAMS

p0: unused

PAYLOAD

a time-zone string, NUL-terminated

REPLY

empty ACK

99

Sets the **alternate** zone, used by commands sent with `p0 = 1` and by GETTZTIME. It is not saved: it lasts until the adapter restarts.

REQUEST → FUJINET

C0	45	99	1E	00	FF	01	00	43	53	54	36	43	44	54	2C	4D	33	2E	32	2E	30	2C	4D	31	31	2E
								C	S	T	6	C	D	T	,	M	3	.	2	.	0	,	M	1	1	.

31	2E	30	00	C0
1	.	0		

REPLY · ACK · 0 PAYLOAD BYTES

C0	45	06	06	00	51	00	C0
----	----	----	----	----	----	----	----

SETTZ_ALT

PARAMS

p0: unused

PAYLOAD

a time-zone string, NUL-terminated

REPLY

empty ACK

74

Sets the **system** zone and saves it to `fnconfig.ini` as `[General] timezone`.

REQUEST → FUJINET · NOT SENT, COMPUTED

C0	45	74	0C	00	E3	01	00	55	54	43	30	00	C0
								U	T	C	0		

REPLY · ACK · 0 PAYLOAD BYTES

C0	45	06	06	00	51	00	C0
----	----	----	----	----	----	----	----

NOTE

Both SETTZ commands answer ACK before they look at the string. A zone containing control characters or bytes above `7E` is then quietly ignored.

The Atari-format and “alternate spelling” reads — `41` , `61` , `69` , `70` , `73` , `7A` — are not available on RS-232 and answer NAK.

CHAPTER 11

The Printer (40h)

The printer device accepts text and printer control codes and renders them on the adapter, as a printer of the type chosen in `fnconfig.ini` would. The result is a file kept on the adapter's SD card (or in its flash when there is no card), which the web interface offers for download — as a PDF for the emulated graphics printers, as text for the plain types.

WRITE

57

PARAMS none
PAYLOAD up to 255 bytes to print
REPLY empty ACK

Prints the bytes. PUT (50) is the same command. An empty payload is refused with NAK.

REQUEST → FUJINET

C0	40	57	1C	00	6C	00	48 H	45 E	4C L	4C L	4F O	20	46 F	52 R	4F O	4D M	20	54 T	48 H	45 E	20	41 A	4C L	54 T	41 A	49 I
52 R	0D	C0																								

REPLY · ACK · 0 PAYLOAD BYTES

C0	40	06	06	00	4C	00	C0
----	----	----	----	----	----	----	----

CAUTION

Keep each WRITE to **255 bytes or fewer**. The renderer takes the length as a single byte, so a longer write is cut short, and its buffer holds 320 bytes with no check: more than that overwrites the firmware's memory. One line per WRITE is the natural size.

STATUS

53

PARAMS p0: unused, but must be sent
PAYLOAD none
REPLY 4 bytes: 00 , a byte with no meaning, 05 , 00

The Atari 820 status frame, kept for compatibility: the timeout byte 05 is the only constant. The second byte is meant to repeat the last write's parameter; on RS-232 it is never set, and holds leftover memory.

REQUEST → FUJINET

C0	40	53	07	00	9B	01	00	C0
----	----	----	----	----	----	----	----	----

REPLY · ACK · 4 PAYLOAD BYTES

C0	40	06	0A	00	C9	00	00	74	05	00	C0
								t			

NOTE

When `[General] printer_enabled=0` , the printer ignores every command without replying.

11.1 Printer types

[Printer1] type chooses the emulation:

Type	Printer	Type	Printer
0	file, raw bytes	9	Atari 1029
1	file, text, trailing blanks trimmed (default)	10	Atari XMM801
2	file, ASCII	11	Atari XDM121
3	Atari 820	12	Epson 80
4	Atari 822	13	Epson for The Print Shop
5	Atari 825	14	Okimate 10
6	Atari 1020	15	GRANTIC (PNG image)
7	Atari 1025	16	HTML
8	Atari 1027	17	HTML with ATASCII

For a CP/M or S-100 machine, type 1 (a text file) or type 12 (an Epson 80, which understands the usual ESC codes and renders a PDF) are the useful choices. The firmware treats CR as the end of a line.

CHAPTER 12

The Modem (50h)

The modem device is a Hayes-style modem that dials TCP hosts instead of telephone numbers: `ATDT bbs.example.com:23` connects, and from then on the bytes typed go to the host and the host's bytes come back. It is the oldest part of the RS-232 firmware, and it does not follow the packet rules of the rest of the bus.

12.1 Talking to the modem

There are two ways in:

- **Raw bytes.** Anything that arrives on the serial line outside a FujiBus frame — before an opening END — is given to the modem, as if typed at it. A terminal program can therefore talk to the modem directly, sending `ATDT...` and a carriage return as plain text.
- **WRITE packets.** A FujiBus WRITE (57) to device 50 hands its payload to the modem the same way.

And one way out: everything the modem says — its result messages, and in data mode the remote host's bytes — is written to the serial line **as raw bytes, never inside a frame**. That includes its answers to the FujiBus commands below: a modem STATUS comes back as two bare bytes, and an acknowledgement as nothing at all.

CAUTION

The modem's immediate answers to AT commands are suppressed on the RS-232 bus: `AT` does not answer `OK`, `ATIP` does not print the address. The commands still take effect, and messages the modem sends later — `CONNECT`, `NO CARRIER` — do appear. Write modem programs to watch for `CONNECT` and the remote host's data, not for `OK`.

12.2 A call

Captured on fujinet-pc, sending plain text on the serial line to a test server on port 7777:

COMPUTER SENDS (RAW)

```
ATDT127.0.0.1:7777<CR>
```

FUJINET SENDS BACK (RAW)

```
WELCOME TO THE ECHO SERVER<CR><LF>
TYPE A LINE, OR BYE.<CR><LF>
CONNECT 115200<CR><LF>
```

COMPUTER SENDS, THEN FUJINET ANSWERS

```
hello<CR>          -> YOU SAID: HELLO<CR><LF>
bye<CR>           -> SO LONG.<CR><LF> NO CARRIER<CR><LF>
```

While the call was up, a FujiBus GET_WIFISTATUS packet sent on the same line was answered normally, framed, in between the host's text. A program can use the modem and the rest of the bus together, provided it can tell raw modem bytes from framed replies: a FujiBus reply always begins with `c0`.

12.3 AT commands

Command	Effect
<code>ATDT host : port, ATDP , ATDI</code>	Dial. The port defaults to 23. A host of digits only is looked up in the phonebook.
<code>ATA</code>	Answer an incoming call (see <code>ATPORT</code>).
<code>ATH , +++ATH</code>	Hang up. <code>+++</code> returns to command mode during a call.
<code>ATO</code>	Return to a call from command mode.
<code>ATPORT n</code>	Listen for incoming connections on TCP port <i>n</i> .
<code>ATS0=1 , ATS0=0</code>	Auto-answer on or off.
<code>ATE0 , ATE1</code>	Echo off or on.
<code>ATV0 , ATV1</code>	Numeric or word result codes.
<code>ATNET0 , ATNET1</code>	TELNET negotiation off or on.
<code>AT+TERM= type</code>	Terminal type reported to TELNET hosts: <code>VT52</code> , <code>VT100</code> , <code>ANSI</code> , <code>DUMB</code> . (<code>ANSI</code> and <code>DUMB</code> are swapped in the firmware.)
<code>ATWIFILIST , ATWIFCONNECT ssid , key</code>	List networks; join one.
<code>ATGET url</code>	Fetch a web page and print it.
<code>ATPB number = host : port</code>	Add a phonebook entry.
<code>ATPBLIST , ATPBCLEAR</code>	List or empty the phonebook.
<code>AT+SNIFF , AT-SNIFF</code>	Record the traffic to a file, or stop.
<code>AT? , ATIP</code>	Help; the adapter's address. Their output is suppressed on RS-232.

`AT&F` , `AT&W` , `ATM0` , `ATX1` , `AT&C1` , `AT&D2` and `ATS` registers other than `S0` are accepted and ignored, so old dialling strings work. The phonebook is kept in `fnconfig.ini` (Chapter 3); dialling `5551234` pretends to connect, for old terminal programs that insist on dialling a number first.

CAUTION

`ATCPM` is listed by the modem's help text but the CP/M device it hands over to does not exist on the RS-232 bus (Chapter 13): the firmware dereferences a null pointer and crashes. Do not send it.

12.4 Modem commands

Command	Code	Params	Effect and reply
WRITE	57	—	Payload to the AT parser, or to the host during a call. No reply.
STATUS	53	p0 (unused)	2 raw bytes: <code>00</code> , then the modem lines — DSR in bits 7–6, CTS in bits 3–2, <code>11</code> each while connected — and bit 0 set when data is waiting.
LISTEN	4C	p0: port	Listen for calls on a TCP port.
UNLISTEN	4D	—	Stop listening.
AUTOANSWER	4F	p0: <code>01</code> on	Answer calls by itself.
BAUDRATELOCK	4E	p0: on/off, p1: baud	Fix the reported speed.
SET_DUMP	44	p0: on/off	The traffic sniffer.
CONFIGURE	42	—	Accepted; sets 9600 unless the rate is locked.
CONTROL	41	—	Accepted; does nothing on RS-232.
STREAM	58	—	Atari concurrent mode: 9 raw bytes of POKEY divisors, then the bus changes speed. Not useful on RS-232.

MODEM STATUS, CAPTURED

```
sent:      C0 50 53 07 00 AB 01 00 C0      (a normal FujiBus packet)
received:  00 00                          (two raw bytes, no frame)
```

CHAPTER 13

The CP/M Device (5Ah)

The FujiNet firmware contains a CP/M 2.2 emulator, RunCPM, and a device to start it: command `47` to device `5A` would switch the serial line over to a CP/M console running *on the adapter*. On the Atari and Coleco ADAM FujiNets this lets a terminal run CP/M programs from the SD card.

On the RS-232 build the CP/M device is compiled in but **never placed on the bus**, so device `5A` does not exist. A packet sent to it is answered like any packet to a missing device, with a NAK whose device byte is `00`. The `[CPM]` section of `fnconfig.ini` has no effect, and the modem's `ATCPM` command, which would hand the line to the same device, crashes the adapter (Chapter 12).

A computer that wants CP/M should run its own, from a disk image mounted in a disk slot (Chapter 8). That is what the 88-2SIO Altair in the appendices would do: boot CP/M from disk slot 1 and run CONFIG, NETCAT and 5 Card Stud as `.COM` files.

APPENDIX A

Quick Reference

A.1 The packet

Item	Rule
Frame	<code>C0</code> , the packet with <code>C0</code> sent as <code>DB DC</code> and <code>DB</code> as <code>DB DD</code> , <code>C0</code> . One END each side.
Header	device · command · length (16 bits, LE, whole packet) · checksum · descriptor
Checksum	8-bit sum of the whole packet with end-around carry, computed with the checksum byte zero.
Descriptor	low 3 bits: <code>0</code> none, <code>1 – 4</code> that many u8, <code>5</code> one u16, <code>6</code> two u16, <code>7</code> one u32. Bit 7: another descriptor follows.
Reply	device = who answered; command <code>06</code> ACK (data in the payload) or <code>15</code> NAK (never data); descriptor <code>00</code> .
Silence	bad checksum, bad length, stalled frame, RESET, printer disabled, missing parameter (crash).

A.2 Devices

Fuji	Disk	Printer	Clock	Modem	CP/M	Network
70	31 – 38	40	45	50	5A (absent)	71 – 78

A.3 Fuji device (70h)

Code	Command	Params / payload	Reply
FF	RESET	—	none (restarts)
FE	GET_SSID	—	97: SSID 33, password 64
FD	SCAN_NETWORKS	—	1: count
FC	GET_SCAN_RESULT	p0 index	34: SSID 33, RSSI
FB	SET_SSID	97: SSID 33, password 64	ACK when joined
FA	GET_WIFISTATUS	—	1: <code>03</code> up, <code>06</code> down
F9	MOUNT_HOST	p0 host	ACK
F8	MOUNT_IMAGE	p0 disk, p1 mode	ACK
F7	OPEN_DIRECTORY	p0 host; 256: path NUL [pattern NUL]	ACK
F6	READ_DIR_ENTRY	p0 length, p1 flags	p0 bytes; <code>7F 7F</code> at end
F5	CLOSE_DIRECTORY	—	ACK
F4	READ_HOST_SLOTS	—	256: 8 × 32
F3	WRITE_HOST_SLOTS	256: 8 × 32	ACK
F2	READ_DEVICE_SLOTS	—	304: 8 × 38
F1	WRITE_DEVICE_SLOTS	304: 8 × 38	ACK
EA	GET_WIFI_ENABLED	—	1
E9	UNMOUNT_IMAGE	p0 disk	ACK
E8	GET_ADAPTERCONFIG	—	140
E7	NEW_DISK	262	always NAK
E6	UNMOUNT_HOST	p0 host	ACK
E5	GET_DIRECTORY_POSITION	—	2
E4	SET_DIRECTORY_POSITION	p0 (u16)	ACK
E2	SET_DEVICE_FULLPATH	p0 disk, p1 host, p2 mode; 256: path	ACK
E1	SET_HOST_PREFIX	p0 host; 256: prefix	ACK
E0	GET_HOST_PREFIX	p0 host	256
DE	WRITE_APPKEY	up to 64 bytes	ACK
DD	READ_APPKEY	—	length (u16) + data
DC	OPEN_APPKEY	6: creator, app, key, mode, 0	ACK
DB	CLOSE_APPKEY	—	ACK
DA	GET_DEVICE_FULLPATH	p0 disk	256
D9	CONFIG_BOOT	p0	ACK
D8	COPY_FILE	p0, p1 hosts from 1 ; <code>src dst</code> NUL	ACK

Code	Command	Params / payload	Reply
D7	MOUNT_ALL	—	ACK
D6	SET_BOOT_MODE	p0 mode	ACK
D3	RANDOM_NUMBER	—	4
D0 / CC	BASE64 ENC/DEC INPUT	p0 count (u16); data	ACK
CF / CB	BASE64 ENC/DEC COMPUTE	—	ACK
CE / CA	BASE64 ENC/DEC LENGTH	—	4
CD / C9	BASE64 ENC/DEC OUTPUT	p0 count (u16)	data
C8	HASH_INPUT	p0 count (u16); data	ACK
C7 / C3	HASH_COMPUTE (/NO_CLEAR)	p0 algorithm 0–3	ACK
C6	HASH_LENGTH	p0 01 hex	1
C5	HASH_OUTPUT	p0 01 hex	digest
C4	GET_ADAPTERCONFIG_EXTENDED	—	240
C2	HASH_CLEAR	—	ACK
BF	QRCODE_OUTPUT	p0 count (u16)	data
BE	QRCODE_LENGTH	p0 form 0–5	4
BD	QRCODE_ENCODE	p0 version, p1 ECC, p2 shorten	ACK
BC	QRCODE_INPUT	p0 count (u16); text	ACK
BB	GENERATE_GUID	—	37
53	STATUS	—	4 zeroes
00	DEVICE_READY	—	512 × 41

A.4 Disk drives (31h–38h)

Code	Command	Params / payload	Reply
52	READ	p0 sector (u32)	512 (flat image)
57	WRITE	p0 sector (u32); sector	ACK
50	PUT	p0 sector (u32); sector	ACK
53	STATUS	do not use: it writes	—
21 / 22	FORMAT, FORMAT_MEDIUM	—	128: FF FF ...
4E	PERCOM_READ	—	12
4F	PERCOM_WRITE	12	ACK

A.5 Network units (71h–78h)

Code	Command	Params / payload	Reply
4F	OPEN	p0 mode, p1 translation; devicespec NUL	ACK
43	CLOSE	—	ACK
53	STATUS	—	4: waiting (u16), connected, status
52	READ	p0 count (u16)	up to count bytes
57	WRITE	p0 count (u16); data	ACK
FC	SET_PARSER	p0 0, p1 01 JSON / 02 HTML / 03 XML	ACK
50	PARSE	—	ACK
51	QUERY	path	ACK
FB	SET_PARAMETER	p0 00 flags / 01 EOL, p1 value	ACK
54	TRANSLATION	p0 0, p1 translation	ACK
4C	SET_EOL	EOL bytes	ACK
5A	SET_INT_RATE	p0 ms	ACK
25	SEEK	p0 offset (u32)	ACK
26	TELL	—	4
30	GETCWD	—	path, no NUL
2C	CHDIR	path	ACK
FD	USERNAME	name NUL	ACK
FE	PASSWORD	password NUL	ACK
20	RENAME	p0 mode, p1 0; spec, new NUL	ACK
21 / 23 / 24	DELETE / LOCK / UNLOCK	p0 mode, p1 0; spec NUL	ACK
2A / 2B	MKDIR / RMDIR	p0 mode, p1 0; spec NUL	ACK

Code	Command	Params / payload	Reply
41	ACCEPT	—	ACK
63	CLOSE_CLIENT	—	inverted: NAK on success
40	SET_HTTP_MODE	p0 0, p1 mode 0–4	ACK
72	GET_REMOTE	—	256: addr:port 9B
44	SET_DESTINATION	N:host:port NUL	ACK

OPEN modes: 04 read/GET, 05 DELETE, 06 directory, 07 directory (alternate), 08 write/PUT, 09 append/DELETE with headers, 0C read-write/GET with headers, 0D POST, 0E PUT with headers. Translation: 00 none, 01 CR, 02 LF, 03 CR LF, 04 PETSCII. Status: 01 OK, 88 end of file, C8 refused, CF not connected; the full list is in Chapter 9.

A.6 Clock (45h), printer (40h), modem (50h)

Device	Code	Command	Reply (all clock commands take p0)
45	93	GETTIME	6: D M Y-2000 h m s
45	9A	GETTZTIME	6, alternate zone
45	54	GET_SIMPLE	7: century Y M D h m s
45	4D	GET_SIMPLE_HUNDREDTHS	8
45	50	GET_PRODOS	4
45	53	GET_SOS	19: YYYYMMDD0HHMMSS000 NUL
45	49	GET_ISO_LOCAL	25: ISO 8601 NUL
45	5A	GET_ISO_UTC	25
45	47	GET_GENERAL	zone NUL
45	4C	GETTZ_LEN	1
45	99	SETTZ	ACK (alternate zone, not saved)
45	74	SETTZ_ALT	ACK (system zone, saved)
40	57 / 50	WRITE / PUT	ACK; at most 255 bytes
40	53	STATUS	4 (p0 required)
50	57	WRITE	none; text to the modem
50	53	STATUS	2 raw bytes, unframed

APPENDIX B

A FujiBus Library in Z80

The programs in the next three appendices share three include files: a **platform layer** that touches the hardware, a **console layer** of printing and input helpers, and the **FujiBus layer** that builds, sends and receives packets. They are written for the Z80 and assembled with `z88dk-z80asm`. An 8080 version is possible: the Z80-only instructions are `JR`, `DJNZ`, `LDIR`, `NEG`, `SBC HL` and the `ED`-prefixed 16-bit loads such as `LD DE,(nn)`, each of which has a short 8080 equivalent.



Figure 3. The three layers. Only ALTAIR.INC knows what the hardware is; CONIO.INC, used by the programs for the terminal, sits beside FUJIBUS.INC on top of it.

B.1 The platform layer: ALTAIR.INC

The MITS 88-2SIO carries two Motorola 6850 ACIAs. Port A, at I/O addresses `10h` (control and status) and `11h` (data), is the operator’s terminal or Teletype. Port B, at `12h` and `13h`, is wired to the FujiNet. A 6850 is programmed by writing `03` (master reset) and then a mode byte to its control register; `15h` selects a divide-by-16 clock, 8 data bits, no parity, one stop bit and no interrupts. The baud rate itself is set by jumpers on the board. Reading the status register gives `RDRF` (bit 0, a byte has arrived) and `TDRE` (bit 1, the transmitter can take a byte). Every routine that touches a port lives in ALTAIR.INC, between lines that read `PLATFORM BEGIN` and `PLATFORM END` (shaded in the listing at the end of this appendix). These are the routines to rewrite for other hardware, and their contracts are all the rest of the code relies on:

Routine	Contract (BC, DE, HL preserved unless stated)
<code>PINIT</code>	Set up both serial ports.
<code>CONST</code>	<code>A = 0</code> if no key is waiting; non-zero if one is.
<code>CONIN</code>	Wait for a key; return it in <code>A</code> with bit 7 clear.
<code>CONOUT</code>	Send <code>A</code> to the terminal.
<code>FNOUT</code>	Send <code>A</code> to the FujiNet.
<code>FNST</code>	<code>A = 0</code> if no byte is waiting from the FujiNet.
<code>FNIN</code>	Wait for a byte from the FujiNet, but not for ever: carry clear and the byte in <code>A</code> , or carry set after about 250 ms of silence.
<code>DELAY</code>	Wait about <code>A × 10</code> ms.
<code>PEXIT</code>	Leave the program: <code>HALT</code> on bare metal, <code>JP 0</code> (CP/M warm boot) when assembled with <code>-DCPM</code> .

The two timeouts are counted in instructions, so they depend on the clock. FNIN’s loop takes 51 T-states a pass, and `FNTMO = 9600` passes is about 250 ms at 2 MHz; DELAY’s inner loop is 26 T-states, and `DLY10MS = 769` is 10 ms. Double both constants for a 4 MHz Z80.

FNIN: RECEIVE A BYTE, WITH A TIMEOUT

```

109 FNIN:  PUSH    BC
110        LD      BC,FNTMO
111 FIN1:   IN       A,(FNSTAT)      ; 11
112        AND     RDRF              ; 7
113        JR      NZ,FIN2           ; 7
114        DEC     BC                ; 6
115        LD      A,B               ; 4
116        OR      C                 ; 4
117        JR      NZ,FIN1           ; 12 = 51 T-states a pass
118        POP     BC
119        SCF                      ; timed out
120        RET
121 FIN2:   IN       A,(FNDATA)
122        POP     BC
123        OR      A                 ; clear carry: byte received
124        RET

```

B.2 Moving to other hardware

Only the routines between the PLATFORM markers change. A few common cases:

A different 88-2SIO strapping, or two boards

Change the four EQU s at the top. A board strapped at 20h for the FujiNet needs only FNSTAT EQU 20h and FNDATA EQU 21h .

An Intel 8251 USART

Many S-100 and single-board machines (the Cromemco TU-ART, the SD Systems boards, the IMSAI SIO) use an 8251. Its status bits are in different places — TXRDY is bit 0, RXRDY bit 1 — and it needs a mode and a command byte after reset:

PLATFORM: 8251 USART FOR THE FUJINET (DATA 22H, CONTROL 23H)

```

FNDATA EQU 22h
FNCTL EQU 23h
TXRDY EQU 01h
RXRDY EQU 02h

FNINIT: XOR     A                ; the classic 8251 wake-up: three zeroes,
        OUT     (FNCTL),A        ; then an internal reset
        OUT     (FNCTL),A
        OUT     (FNCTL),A
        LD      A,40h
        OUT     (FNCTL),A
        LD      A,4Eh            ; mode: async x16, 8 data, no parity, 1 stop
        OUT     (FNCTL),A
        LD      A,37h            ; command: RTS, error reset, RX on, DTR, TX on
        OUT     (FNCTL),A
        RET

FNOUT:  PUSH    AF
FOUT1:  IN      A,(FNCTL)
        AND     TXRDY
        JR      Z,FOUT1
        POP     AF
        OUT     (FNDATA),A
        RET

FNST:   IN      A,(FNCTL)
        AND     RXRDY
        RET

```

FNIN keeps its shape: poll FNCTL for RXRDY instead of FNSTAT for RDRF .

A Zilog SIO or SCC

The Z80 SIO (Kaypro, Osborne Executive, many CP/M boards) reports “character available” in bit 0 and “transmit buffer empty” in bit 2 of read register 0, which is what a plain `IN` from its control port returns. Its set-up is a short table of register writes (channel reset, then registers 4, 3 and 5 for x16 clock, 8 bits, receiver and transmitter on), and the baud rate comes from a separate CTC or baud-rate generator.

A CP/M machine whose console is not a 6850

Under CP/M the console routines can call the BIOS or BDOS instead of touching hardware: `CONST` becomes BDOS function 11, `CONIN` function 1 (or the BIOS `CONIN` entry, which does not echo), `CONOUT` function 2. Only the FujiNet routines then need real port code, for the serial port the FujiNet is plugged into — the reader or punch port, or a second serial card.

IMPORTANT

Whatever the hardware, the receive side must keep up. The FujiNet sends a reply as fast as the line allows, with no flow control, and a single-byte UART like the 6850 holds only one byte: `FBRECV` stores each byte and goes straight back for the next. Do not add work to that loop, and do not let an interrupt routine hold the processor for longer than one character time (about 1 ms at 9600 baud). If you cannot avoid it, lower the baud rate, or use the `RESEND` command where the firmware has it.

B.3 The packet layer: FUJIBUS.INC

A program sends a command in three steps: start the packet with `FBNEW`, add parameters and payload with `FBYTE`, `FBWORD`, `FBMEM` and `FBSTR`, then call `FBCALL`. `GET_WIFISTATUS`, for example, is:

A COMPLETE COMMAND

```
LD      A,70h          ; device: Fuji
LD      C,0FAh         ; command: GET_WIFISTATUS
LD      B,00h          ; descriptor: no parameters
CALL    FBNEW
CALL    FBCALL          ; send it, collect the reply
JR      C,NOREPLY       ; carry: no good reply
JR      NZ,GOTNAK       ; NZ: NAK
LD      A,(HL)          ; HL -> payload, BC = its length
```

The descriptor is given by the caller, not worked out by the library: the programmer knows the command takes, say, two 8-bit parameters, and writes `LD B,02h`. `FBNEW` stores the device, the command and the descriptor, leaves room for the length and the checksum, and `FBCALL` fills both in when the packet is complete:

FBCALL: FINISHING THE PACKET

```
151 FBCALL: LD      HL,(TXPTR)      ; length = TXPTR - TXBUF
152          LD      DE,TXBUF
153          OR      A
154          SBC     HL,DE
155          LD      (TXLEN),HL
156          LD      (TXBUF+2),HL   ; +2/+3 length, little-endian
157          XOR     A
158          LD      (TXBUF+4),A     ; checksum is computed over a zero
159          LD      B,H
160          LD      C,L
161          LD      HL,TXBUF
162          CALL    CKSUM
163          LD      (TXBUF+4),A
164          LD      A,FBTRIES
165          LD      (TRIES),A
166          CALL    FBDRAIN        ; throw away anything stale first
167          CALL    FBSEND
```

FBSTR is the library's answer to the fixed-size payload rule of Chapter 4: it copies a NUL-terminated string into a field of exactly the size given, padding with zeroes. A path for OPEN_DIRECTORY is therefore one call:

A 256-BYTE PATH FIELD

```
LD    HL,PATH      ; the string
LD    BC,256       ; the field size
CALL  FBSTR
```

Sending

FBSEND writes the frame: one END, each byte with `C0` and `DB` replaced by their escape pairs, and one END.

FBSEND: SLIP FRAMING

```
211 FBSEND: LD    HL,TXBUF
212         LD    BC,(TXLEN)
213 FBTX:   LD    A,SLEND      ; exactly one END opens the frame
214         CALL  FNOUT
215 FBT1:   LD    A,(HL)
216         CP    SLEND
217         JR    Z,FBT2
218         CP    SLESC
219         JR    Z,FBT3
220         CALL  FNOUT      ; an ordinary byte
221         JR    FBT4
222 FBT2:   LD    A,SLESC      ; C0h -> DB DC
223         CALL  FNOUT
224         LD    A,SLEEND
225         CALL  FNOUT
226         JR    FBT4
227 FBT3:   CALL  FNOUT      ; DBh -> DB DD
228         LD    A,SLEESC
229         CALL  FNOUT
230 FBT4:   INC    HL
231         DEC    BC
232         LD    A,B
233         OR     C
234         JR    NZ,FBT1
235         LD    A,SLEND      ; and one END closes it
236         JP    FNOUT
```

Receiving

FBRECV is the delicate part. It waits up to FBWAIT timeouts (40, about ten seconds) for the opening END, skipping anything else; then it stores bytes, undoing escapes, until the closing END. Each byte must arrive within one FNIN timeout or the frame is abandoned. Finally it checks the length field and the checksum. On failure it sets `FBERR` to say why, which the programs print:

FBERR	Meaning
1	no reply at all
2	the reply stopped part-way
3	longer than RXMAX (600 bytes)
4	shorter than a header
5	the length field disagrees with the bytes received
6	bad checksum

FBRECV: THE RECEIVE LOOP

```
280 FBR3:   LD    HL,RXBUF      ; HL = store pointer
281         LD    BC,0          ; BC = bytes stored
282 FBR4:   CALL  FNIN
283         JR    C,FBSTAL      ; frame stalled part-way
284         CP    SLEND
```

```

285      JR      Z,FBR8          ; end of frame
286      CP      SLESC
287      JR      NZ,FBR6
288      CALL     FNIN           ; an escape: take the next byte
289      JR      C,FBSTAL
290      CP      SLEEND
291      LD      A,SLEND
292      JR      Z,FBR6
293      LD      A,SLESC
294 FBR6:  PUSH    HL             ; overlong? keep reading, stop storing
295      LD      HL,RXMAX
296      OR      A
297      SBC     HL,BC
298      POP     HL
299      JR      Z,FBR7          ; BC = RXMAX: buffer full
300      JR      C,FBR7          ; BC > RXMAX: still full
301      LD      (HL),A
302      INC     HL
303 FBR7:  INC     BC             ; count every byte, stored or not
304      JR      FBR4

```

Retrying

FBCALL tries up to three times. Before the first try it drains the port (FBDRAIN, Chapter 5), so that a stale byte cannot be taken for the reply. After a failure it sends the whole command again — or, when the library is assembled with `-DUSE_RESEND=1` and some of the reply did arrive, it sends the RESEND command instead:

FBCALL: THE RETRY LOOP

```

168 FBC1:  CALL    FBRECV
169      JR      NC,FBC3          ; a good frame
170      LD      A,(TRIES)
171      DEC     A
172      LD      (TRIES),A
173      SCF
174      RET     Z               ; out of tries: carry set
175      IF      USE_RESEND
176      LD      A,(RXGOT)        ; did any of the reply arrive?
177      OR      A
178      JR      Z,FBC2          ; no: the command itself was lost
179      CALL    FBRSND          ; yes: ask for the reply again
180      JR      FBC1
181      ENDIF
182 FBC2:  CALL    FBSEND          ; send the whole command again
183      JR      FBC1

```

The retry logic was tested by passing the simulated Altair's traffic through a proxy that damaged every second or third reply ([tools/faultproxy.py](https://github.com/fujinet/fujinet-firmware/blob/master/tools/faultproxy.py)): CONFIG ran correctly both ways, re-sending commands in the default build and asking for RESEND in the other.

B.4 The console layer: CONIO.INC

CONIO.INC holds the small routines every program needs: PRINT (print the string that follows the CALL), PRSTR and PRSTRN, CRLF, PRHEX, PRDEC (and PRDECW, right-aligned in a field), GETLN (a line editor with backspace and Ctrl-C), GETKEY and UCASE. Its one trick is PRINT, which finds its string through the return address and returns past it:

PRINT: AN INLINE STRING

```

28 PRINT: EX      (SP),HL        ; HL = the string after the CALL
29      CALL    PRSTR
30      EX      (SP),HL        ; return just past its NUL
31      RET

```

B.5 A first program: HELLO

HELLO asks for the WiFi status and prints the host slots: the smallest program that proves the hardware, the cable and the configuration are right. Run it first.

HELLO ON THE SIMULATED ALTAIR

```
FUJINET RS-232 HELLO
WIFI STATUS 03
1: SD
2: fujinet.online
3: apps.irata.online
4: fujinet.diller.org
5: FujiNet.Atari8bit.net
6: atarionline.eu
7: fujinet.pl
8: ec.tnfs.io
```

B.6 Building

```
cd listings/hello
z88dk-z80asm -b hello.asm           # -> hello.bin, to load at 0100h
z88dk-z80asm -b -DCPM=1 hello.asm   # the same, exiting to CP/M
z88dk-z80asm -b -DUSE_RESEND=1 ...  # ask for RESEND on a bad reply
```

Every program is `ORG 0100h` and uses no CP/M calls, so the same binary can be loaded at `0100h` and started from a monitor or the front panel, or renamed `.COM` and run from CP/M (assemble with `-DCPM=1` so that it returns to CP/M rather than halting). `make listings` at the top of the manual's directory builds them all.

B.7 Listings

ALTAIR.INC

```
1 ;=====
2 ; ALTAIR.INC -- platform layer for an Altair 8800 with a MITS 88-2SIO
3 ;
4 ; Everything that touches hardware lives in this one file. Port A of
5 ; the 88-2SIO is the operator's terminal (or Teletype); port B is the
6 ; FujiNet. To move the examples to another machine, rewrite only the
7 ; routines between the PLATFORM BEGIN / PLATFORM END markers below and
8 ; keep their register contracts.
9 ;
10 ; Contracts (all routines preserve BC, DE, HL unless stated):
11 ;   PINIT      initialise both serial ports
12 ;   CONST      A = 0 if no key waiting, NZ (A = FFh) if a key is
13 ;   CONIN      wait for a key; A = character, bit 7 stripped
14 ;   CONOUT     send the character in A to the terminal
15 ;   FNOUT      send the byte in A to the FujiNet
16 ;   FNST       A = 0 if no byte waiting from the FujiNet, NZ if one is
17 ;   FNIN       receive a byte from the FujiNet with a timeout:
18 ;               carry clear, A = byte    -- a byte arrived
19 ;               carry set                -- nothing within FNTMO
20 ;   DELAY      wait about 10 ms x A (A = 0 means 256)
21 ;   PEXIT      return to the monitor / CP/M
22 ;=====
23
24 ;-----
25 ; PLATFORM BEGIN: 88-2SIO port assignments
26 ;-----
27 CONSTAT EQU    10h          ; port A control/status (terminal)
28 CONDATA EQU    11h          ; port A data
29 FNSTAT EQU     12h          ; port B control/status (FujiNet)
30 FNDATA EQU     13h          ; port B data
31
32 RDRF EQU       01h          ; 6850 status: receive data register full
33 TDRE EQU       02h          ; 6850 status: transmit data register empty
34
35 ACIARST EQU     03h          ; 6850 control: master reset
36 ACIAMOD EQU     15h          ; 6850 control: /16 clock, 8 data, no
37 ;                               ; parity, 1 stop, interrupts off.
38 ;                               ; The baud rate itself is set by the
39 ;                               ; jumpers on the 88-2SIO board.
40
41 ; FNIN gives up after FNTMO passes through its polling loop. One pass
42 ; is 51 T-states, so at 2 MHz 9600 passes is about 250 ms -- longer
43 ; than the 200 ms inter-byte gap the FujiNet itself allows. Scale it
```

```

44 ; with your CPU clock (double it at 4 MHz).
45 FNTMO EQU 9600
46
47 ; DELAY's inner loop: 10 ms at 2 MHz. Again, scale with the clock.
48 DLY10MS EQU 769
49 ;-----
50 ; PLATFORM END
51 ;-----
52
53 ;-----
54 ; PLATFORM BEGIN: PINIT -- reset and program both 6850 ACIAs
55 ;-----
56 PINIT: LD A,ACIARST
57 OUT (CONSTAT),A
58 OUT (FNSTAT),A
59 LD A,ACIAMOD
60 OUT (CONSTAT),A
61 OUT (FNSTAT),A
62 RET
63 ;-----
64 ; PLATFORM END
65 ;-----
66
67 ;-----
68 ; PLATFORM BEGIN: terminal I/O on port A
69 ;-----
70 CONST: IN A,(CONSTAT)
71 AND RDRF
72 RET Z ; A = 0: nothing waiting
73 LD A,0FFh
74 RET
75
76 CONIN: IN A,(CONSTAT)
77 AND RDRF
78 JR Z,CONIN
79 IN A,(CONDATA)
80 AND 7Fh
81 RET
82
83 CONOUT: PUSH AF
84 COUT1: IN A,(CONSTAT)
85 AND TDRE
86 JR Z,COUT1
87 POP AF
88 OUT (CONDATA),A
89 RET
90 ;-----
91 ; PLATFORM END
92 ;-----
93
94 ;-----
95 ; PLATFORM BEGIN: FujiNet I/O on port B
96 ;-----
97 FNOUT: PUSH AF
98 FOUT1: IN A,(FNSTAT)
99 AND TDRE
100 JR Z,FOUT1
101 POP AF
102 OUT (FNSTAT),A
103 RET
104
105 FNST: IN A,(FNSTAT)
106 AND RDRF
107 RET
108
109 FNIN: PUSH BC
110 LD BC,FNTMO
111 FIN1: IN A,(FNSTAT) ; 11
112 AND RDRF ; 7
113 JR NZ,FIN2 ; 7
114 DEC BC ; 6
115 LD A,B ; 4
116 OR C ; 4
117 JR NZ,FIN1 ; 12 = 51 T-states a pass
118 POP BC
119 SCF ; timed out
120 RET
121 FIN2: IN A,(FNSTAT)
122 POP BC
123 OR A ; clear carry: byte received
124 RET
125 ;-----
126 ; PLATFORM END
127 ;-----
128
129 ;-----
130 ; PLATFORM BEGIN: DELAY -- about A x 10 ms, PEXIT -- leave the program

```

```

131 ;-----
132 DELAY:  PUSH  BC
133         PUSH  DE
134         LD    D,A
135 DLY1:   LD    BC,DLY10MS
136 DLY2:   DEC   BC          ; 6
137         LD    A,B        ; 4
138         OR    C          ; 4
139         JR    NZ,DLY2    ; 12 = 26 T-states a pass
140         DEC   D
141         JR    NZ,DLY1
142         POP   DE
143         POP   BC
144         RET
145
146 ; Bare metal: halt with the front-panel lights showing PC. Under CP/M
147 ; assemble with CPM defined (-DCPM) and the program warm-boots instead.
148 PEXIT:
149         IF    CPM
150         JP    0000h
151         ELSE
152         HALT
153         JR    PEXIT
154         ENDIF
155 ;-----
156 ; PLATFORM END
157 ;-----
158

```

FUJIBUS.INC

```

1 ;=====
2 ; FUJIBUS.INC -- the FujiBus (FEP-004) packet layer, in Z80
3 ;
4 ; Builds a command packet, frames it with SLIP, sends it, and collects
5 ; and checks the reply. It touches no hardware itself: bytes go out
6 ; through FNOUT and come in through FNIN (see ALTAIR.INC).
7 ;
8 ; Building a packet:
9 ;   FBNEW  A = device, C = command, B = first descriptor byte
10 ;   FBBYTE A      append one byte (a u8 parameter, an extra
11 ;                descriptor byte, or payload)
12 ;   FBWORD HL      append a 16-bit value, low byte first
13 ;   FBMEM  HL, BC  append BC bytes from (HL)
14 ;   FBSTR  HL, BC  append the NUL-terminated string at (HL), padded
15 ;                with zeroes (or cut) to exactly BC bytes
16 ; Sending it:
17 ;   FBCALL send the packet and wait for the reply, retrying as needed
18 ;         returns carry set -- no good reply after FBTRIES tries
19 ;         carry clear -- reply in RXBUF, and then
20 ;         Z set -- ACK; HL = payload, BC = its length
21 ;         Z clear -- NAK (no payload)
22 ;
23 ; Registers: FBNEW/FBBYTE/FBWORD/FBMEM/FBSTR preserve everything but
24 ; AF (FBMEM and FBSTR also advance HL past what they copied). FBCALL
25 ; destroys AF, BC, DE, HL.
26 ;=====
27
28 SLEND  EQU    0C0h      ; SLIP END
29 SLESC  EQU    0DBh      ; SLIP ESC
30 SLEEND EQU    0DCh      ; ESC + this = a data byte C0h
31 SLEESC EQU    0DDh      ; ESC + this = a data byte DBh
32
33 ACK    EQU    06h      ; reply command: success
34 NAK    EQU    15h      ; reply command: failure
35 RESEND EQU    05h      ; "send your last reply again"
36                ; (fujinet-pc rs232-resend only)
37
38 TXMAX  EQU    320      ; largest packet we build
39 RXMAX  EQU    600      ; largest reply we accept
40 FBTRIES EQU    3       ; attempts before FBCALL gives up
41
42 ; Assemble with USE_RESEND defined non-zero to ask for a garbled reply
43 ; again (command 05h) instead of sending the whole command a second
44 ; time. Only firmware that implements RESEND answers it.
45         IF    !USE_RESEND
46 USE_RESEND EQU    0
47         ENDIF
48
49 ;-----
50 ; FBNEW -- start a packet: device in A, command in C, descriptor in B
51 ;-----
52 FBNEW:  PUSH  HL
53         LD    HL, TXBUF
54         LD    (HL), A      ; +0 device

```

```

55      INC      HL
56      LD       (HL),C          ; +1 command
57      INC      HL
58      LD       (HL),0          ; +2 length, filled in by FBCALL
59      INC      HL
60      LD       (HL),0          ; +3
61      INC      HL
62      LD       (HL),0          ; +4 checksum, filled in by FBCALL
63      INC      HL
64      LD       (HL),B          ; +5 first descriptor
65      INC      HL
66      LD       (TXPTR),HL
67      POP      HL
68      RET
69
70 ;-----
71 ; FBBYTE -- append A; FBWORD -- append HL, low byte first
72 ;-----
73 FBBYTE: PUSH    HL
74      LD       HL,(TXPTR)
75      LD       (HL),A
76      INC      HL
77      LD       (TXPTR),HL
78      POP      HL
79      RET
80
81 FBWORD: LD      A,L
82      CALL    FBBYTE
83      LD      A,H
84      JR      FBBYTE
85
86 ;-----
87 ; FBMEM -- append BC bytes from (HL)
88 ;-----
89 FBMEM: PUSH    BC
90      PUSH    DE
91      LD      DE,(TXPTR)
92      LD      A,B
93      OR      C
94      JR      Z,FBM1
95      LDIR
96 FBM1:  LD      (TXPTR),DE
97      POP     DE
98      POP     BC
99      RET
100
101 ;-----
102 ; FBSTR -- append the string at (HL) as a field exactly BC bytes long
103 ;-----
104 FBSTR: PUSH    BC
105      PUSH    DE
106      LD      DE,(TXPTR)
107 FBS1:  LD      A,B
108      OR      C
109      JR      Z,FBS3          ; field full
110      LD      A,(HL)
111      OR      A
112      JR      Z,FBS2          ; end of string: pad the rest
113      LD      (DE),A
114      INC     HL
115      INC     DE
116      DEC     BC
117      JR      FBS1
118 FBS2:  XOR     A              ; zero-fill to the field size
119      LD      (DE),A
120      INC     DE
121      DEC     BC
122      LD      A,B
123      OR      C
124      JR      NZ,FBS2
125 FBS3:  LD      (TXPTR),DE
126      POP     DE
127      POP     BC
128      RET
129
130 ;-----
131 ; CKSUM -- FujiBus checksum of BC bytes at (HL), result in A
132 ;
133 ; An 8-bit sum with end-around carry: after each add, the carry out of
134 ; bit 7 is added back in.  ADD then ADC 0 is exactly that.
135 ;-----
136 CKSUM: XOR     A
137 CKS1:  ADD     A,(HL)
138      ADC     A,0
139      INC     HL
140      DEC     BC
141      LD      D,A

```

```

142      LD      A,B
143      OR      C
144      LD      A,D
145      JR      NZ,CKS1
146      RET
147
148      ;-----
149      ; FBCALL -- finish the packet, send it, return the reply
150      ;-----
151 FBCALL: LD      HL,(TXPTR)      ; length = TXPTR - TXBUF
152      LD      DE,TXBUF
153      OR      A
154      SBC     HL,DE
155      LD      (TXLEN),HL
156      LD      (TXBUF+2),HL      ; +2/+3 length, little-endian
157      XOR     A
158      LD      (TXBUF+4),A      ; checksum is computed over a zero
159      LD      B,H
160      LD      C,L
161      LD      HL,TXBUF
162      CALL    CKSUM
163      LD      (TXBUF+4),A
164      LD      A,FBTRIES
165      LD      (TRIES),A
166      CALL    FBDRAIN          ; throw away anything stale first
167      CALL    FBSEND
168 FBC1:  CALL    FBRCV          ; a good frame
169      JR      NC,FBC3
170      LD      A,(TRIES)
171      DEC     A
172      LD      (TRIES),A
173      SCF
174      RET     Z               ; out of tries: carry set
175      IF     USE_RESEND
176      LD      A,(RXGOT)      ; did any of the reply arrive?
177      OR      A
178      JR      Z,FBC2          ; no: the command itself was lost
179      CALL    FBRSD          ; yes: ask for the reply again
180      JR      FBC1
181      ENDIF
182 FBC2:  CALL    FBSEND          ; send the whole command again
183      JR      FBC1
184 FBC3:  LD      HL,(RXLEN)      ; payload length = frame length - 6
185      LD      DE,6
186      OR      A
187      SBC     HL,DE
188      LD      B,H
189      LD      C,L
190      LD      HL,RXBUF+6      ; replies carry no parameters
191      LD      A,(RXBUF+1)      ; the reply's command byte
192      CP      ACK             ; Z = ACK, carry clear
193      RET
194
195      ;-----
196      ; FBDRAIN -- discard whatever is already waiting on the FujiNet port
197      ;
198      ; A reply should only ever answer the command just sent. Some commands
199      ; can answer twice (SET_SSID sends a second NAK when the connect fails),
200      ; and a reply abandoned after a timeout may still trickle in; either
201      ; would be taken for the answer to the NEXT command.
202      ;-----
203 FBDRAIN: CALL    FNST
204      RET     Z
205      CALL    FNIN
206      JR      FBDRAIN
207
208      ;-----
209      ; FBSEND -- SLIP-frame TXBUF (TXLEN bytes) out through FNOUT
210      ;-----
211 FBSEND: LD      HL,TXBUF
212      LD      BC,(TXLEN)
213 FBTX:  LD      A,SLEND          ; exactly one END opens the frame
214      CALL    FNOUT
215 FBT1:  LD      A,(HL)
216      CP      SLEND
217      JR      Z,FBT2
218      CP      SLESC
219      JR      Z,FBT3
220      CALL    FNOUT          ; an ordinary byte
221      JR      FBT4
222 FBT2:  LD      A,SLESC          ; C0h -> DB DC
223      CALL    FNOUT
224      LD      A,SLEEND
225      CALL    FNOUT
226      JR      FBT4
227 FBT3:  CALL    FNOUT          ; DBh -> DB DD
228      LD      A,SLEESC

```

```

229      CALL    FNOUT
230 FBT4:  INC    HL
231      DEC    BC
232      LD     A,B
233      OR     C
234      JR     NZ,FBT1
235      LD     A,SLEND          ; and one END closes it
236      JP     FNOUT
237
238      IF     USE_RESEND
239 ;-----
240 ; FBRSEND -- send RESEND (05h) to the device we were talking to
241 ;-----
242 FBRSEND: LD     HL,RSPKT
243          LD     A,(TXBUF)      ; same device as the command
244          LD     (HL),A
245          XOR    A
246          LD     (RSPKT+4),A
247          LD     BC,6
248          CALL   CKSUM
249          LD     (RSPKT+4),A
250          LD     HL,RSPKT
251          LD     BC,6
252          JR     FBTX
253 RSPKT:  DEFB   0,RESEND,6,0,0,0
254      ENDF
255
256 ;-----
257 ; FBRECV -- collect one SLIP frame into RXBUF and check it
258 ; carry clear: good frame, RXLEN = its decoded length
259 ; carry set:  FBERR says why --
260 ;             1 no reply at all      4 shorter than a header
261 ;             2 stopped part-way     5 length field disagrees
262 ;             3 longer than RXMAX    6 bad checksum
263 ;             RXGOT = 0 if not a single byte of it arrived
264 ;-----
265 FBRECV: XOR    A
266          LD     (RXGOT),A
267          LD     (FBERR),A
268          LD     A,(FBWAIT)      ; first byte: allow FBWAIT timeouts
269          LD     E,A
270 FBR1:  CALL   FNIN
271          JR     NC,FBR2
272          DEC    E
273          JR     NZ,FBR1
274          LD     A,1              ; FBERR 1: the device never answered
275          JR     FBFAIL
276 FBR2:  CP     SLEND              ; skip anything before the opening END
277          JR     NZ,FBR1
278          LD     A,1
279          LD     (RXGOT),A
280 FBR3:  LD     HL,RXBUF          ; HL = store pointer
281          LD     BC,0            ; BC = bytes stored
282 FBR4:  CALL   FNIN
283          JR     C,FBSTAL        ; frame stalled part-way
284          CP     SLEND
285          JR     Z,FBR8          ; end of frame
286          CP     SLESC
287          JR     NZ,FBR6
288          CALL   FNIN            ; an escape: take the next byte
289          JR     C,FBSTAL
290          CP     SLEEND
291          LD     A,SLEND
292          JR     Z,FBR6
293          LD     A,SLESC
294 FBR6:  PUSH   HL                ; overlong? keep reading, stop storing
295          LD     HL,RXMAX
296          OR     A
297          SBC   HL,BC
298          POP    HL
299          JR     Z,FBR7          ; BC = RXMAX: buffer full
300          JR     C,FBR7          ; BC > RXMAX: still full
301          LD     (HL),A
302          INC    HL
303 FBR7:  INC    BC                ; count every byte, stored or not
304          JR     FBR4
305 FBR8:  LD     A,B              ; END with nothing before it: that was
306          OR     C              ; an opening END, so start again
307          JR     Z,FBR3
308          LD     (RXLEN),BC
309          LD     HL,RXMAX        ; overlong frames fail
310          OR     A
311          SBC   HL,BC
312          LD     A,3              ; FBERR 3: longer than RXMAX
313          JR     C,FBFAIL
314          LD     A,B              ; shorter than a header?
315          OR     A

```

```

316      JR      NZ,FBR9
317      LD      A,C
318      CP      6
319      LD      A,4          ; FBERR 4: shorter than a header
320      JR      C,FBFAIL
321 FBR9:  LD      HL,(RXBUF+2) ; the length field must match
322      OR      A
323      SBC     HL,BC
324      LD      A,5          ; FBERR 5: length field disagrees
325      JR      NZ,FBFAIL
326      LD      A,(RXBUF+4)  ; and so must the checksum
327      LD      D,A
328      PUSH    DE
329      XOR     A
330      LD      (RXBUF+4),A
331      LD      HL,RXBUF
332      CALL    CKSUM
333      POP     DE
334      CP      D
335      LD      A,6          ; FBERR 6: bad checksum
336      JR      NZ,FBFAIL
337      OR      A          ; good frame: clear carry
338      RET
339 FBSTAL: LD     A,2          ; FBERR 2: the frame stopped part-way
340 FBFAIL: LD     (FBERR),A
341      SCF
342      RET
343
344 ; -----
345 ; Data
346 ; -----
347 FBWAIT: DEFB  40          ; first-byte wait, in FNIN timeouts
348                        ; (40 x 250 ms = 10 s); raise it for
349                        ; slow commands
350 FBERR:  DEFB   0          ; why the last receive failed (1-6)
351 TRIES:  DEFB   0
352 RXGOT:  DEFB   0
353 TXPTR:  DEFW   0
354 TXLEN:  DEFW   0
355 RXLEN:  DEFW   0
356 TXBUF:  DEFS   TXMAX
357 RXBUF:  DEFS   RXMAX
358

```

CONIO.INC

```

1 ;=====
2 ; CONIO.INC -- terminal helpers built on CONIN/CONOUT
3 ;
4 ; PRINT    print the NUL-terminated string that follows the CALL
5 ; PRSTR    print the NUL-terminated string at (HL); HL ends past NUL
6 ; PRSTRN   print at most B characters of the string at (HL)
7 ; CRLF     carriage return + line feed
8 ; SPACE    one space; SPACES B spaces
9 ; PRHEX    A as two hex digits
10 ; PRDEC    HL as an unsigned decimal number (no leading zeroes)
11 ; PRDECW   HL in decimal, right-aligned in a field of B characters
12 ; GETLN    read a line into (HL), at most B characters; BS/DEL rub
13 ;          out, CR ends; on return the line is NUL-terminated and
14 ;          B = its length (C is destroyed). Ctrl-C leaves the
15 ;          line empty.
16 ; GETKEY   CONIN with lower case folded to upper case
17 ; UCASE    fold the character in A to upper case
18 ;
19 ; All preserve BC, DE, HL except where a result is returned in them.
20 ;=====
21
22 CR      EQU    0Dh
23 LF      EQU    0Ah
24 BS      EQU    08h
25 DEL     EQU    7Fh
26 CTRLC   EQU    03h
27
28 PRINT:  EX      (SP),HL    ; HL = the string after the CALL
29         CALL    PRSTR
30         EX      (SP),HL    ; return just past its NUL
31         RET
32
33 PRSTR:  LD      A,(HL)
34         INC     HL
35         OR      A
36         RET     Z
37         CALL    CONOUT
38         JR      PRSTR
39

```

```

40 PRSTRN: PUSH    BC
41 PRN1:  LD      A,B
42        OR      A
43        JR      Z,PRN2
44        LD      A,(HL)
45        OR      A
46        JR      Z,PRN2
47        CALL    CONOUT
48        INC     HL
49        DEC     B
50        JR      PRN1
51 PRN2:  POP      BC
52        RET
53
54 CRLF:  LD      A,CR
55        CALL    CONOUT
56        LD      A,LF
57        JP      CONOUT
58
59 SPACE: LD      A,' '
60        JP      CONOUT
61
62 SPACES: PUSH    BC
63 SPC1:  LD      A,B
64        OR      A
65        JR      Z,SPC2
66        CALL    SPACE
67        DEC     B
68        JR      SPC1
69 SPC2:  POP      BC
70        RET
71
72 PRHEX: PUSH    AF
73        RRCA
74        RRCA
75        RRCA
76        RRCA
77        CALL    PRNIB
78        POP     AF
79 PRNIB: AND     0Fh
80        ADD     A,90h
81        DAA
82        ADC     A,40h
83        DAA
84        JP      CONOUT
85
86 ; PRDEC / PRDECW share the conversion: digits are built backwards in
87 ; DECBUF, then printed (PRDECW pads on the left first).
88 PRDEC: PUSH    BC
89        LD      B,0
90        CALL    PRDECW
91        POP     BC
92        RET
93 PRDECW: PUSH    BC
94        PUSH    DE
95        PUSH    HL
96        LD      DE,DECBUF+5
97        XOR     A
98        LD      (DE),A
99        LD      C,0
100 PRD1:  PUSH    BC
101        LD      C,10
102        CALL    DIV8
103        POP     BC
104        ADD     A,'0'
105        DEC     DE
106        LD      (DE),A
107        INC     C
108        LD      A,H
109        OR      L
110        JR      NZ,PRD1
111        LD      A,B
112        SUB     C
113        JR      C,PRD2
114        JR      Z,PRD2
115        LD      B,A
116        CALL    SPACES
117 PRD2:  EX      DE,HL
118        CALL    PRSTR
119        POP     HL
120        POP     DE
121        POP     BC
122        RET
123
124 ; DIV8 -- HL = HL / C, A = remainder (C non-zero, 8-bit)
125 DIV8:  PUSH    BC
126        XOR     A

```

```

; the classic DAA trick: 0-9 -> '0'-'9',
; A-F -> 'A'-'F'

```

```

; NUL terminator
; C = digits made

```

```

; HL = HL / 10, A = remainder

```

```

; pad: B - C spaces, if positive

```

```

127      LD      B,16
128 DV1:  ADD     HL,HL
129      RLA
130      CP      C
131      JR      C,DV2
132      SUB     C
133      INC     L
134 DV2:  DJNZ    DV1
135      POP     BC
136      RET
137
138 UCASE: CP      'a'
139      RET     C
140      CP      'z'+1
141      RET     NC
142      SUB     20h
143      RET
144
145 GETKEY: CALL    CONIN
146      JR      UCASE
147
148 GETLN: PUSH     DE
149      PUSH     HL
150      LD      D,B          ; D = limit
151      LD      E,0          ; E = count
152 GLN1:  CALL    CONIN
153      CP      CR
154      JR      Z,GLN4
155      CP      CTRLC
156      JR      Z,GLN5
157      CP      BS
158      JR      Z,GLN2
159      CP      DEL
160      JR      Z,GLN2
161      CP      ' '
162      JR      C,GLN1        ; ignore other control characters
163      LD      C,A
164      LD      A,E
165      CP      D
166      JR      NC,GLN1        ; full: ignore
167      LD      A,C
168      LD      (HL),A
169      INC     HL
170      INC     E
171      CALL    CONOUT        ; echo
172      JR      GLN1
173 GLN2:  LD      A,E          ; rub out the last character
174      OR      A
175      JR      Z,GLN1
176      DEC     HL
177      DEC     E
178      CALL    PRINT
179      DEFB    BS,' ',BS,0
180      JR      GLN1
181 GLN5:  POP     HL          ; Ctrl-C: empty line
182      PUSH     HL
183      LD      E,0
184 GLN4:  LD      (HL),0
185      LD      B,E
186      CALL    CRLF
187      POP     HL
188      POP     DE
189      RET
190
191 DECBUF: DEFS    6
192

```

HELLO.ASM

```

1  ;=====
2  ; HELLO.ASM -- first contact: ask the FujiNet for its WiFi status and
3  ; print the eight host slots.  Altair 8800 + 88-2SI0; ORG 0100h.
4  ;=====
5      ORG      0100h
6
7 START: LD      SP,STACK
8      CALL    PINIT
9      CALL    PRINT
10     DEFB    "FUJINET RS-232 HELLO",13,10,0
11
12 ; GET_WIFISTATUS: device 70h, command FAh, no parameters
13     LD      A,70h
14     LD      C,0FAh
15     LD      B,00h          ; descriptor 0: no parameters
16     CALL    FBNEW

```

```

17      CALL    FBCALL
18      JR      C,NOREPLY
19      JR      NZ,GOTNAK
20      CALL    PRINT
21      DEFB    "WIFI STATUS ",0
22      LD      A,(HL)          ; 3 = connected, 6 = disconnected
23      CALL    PRHEX
24      CALL    CRLF
25
26 ; READ_HOST_SLOTS: device 70h, command F4h -> 8 x 32 bytes
27      LD      A,70h
28      LD      C,0F4h
29      LD      B,00h
30      CALL    FBNEW
31      CALL    FBCALL
32      JR      C,NOREPLY
33      JR      NZ,GOTNAK
34      LD      C,'1'          ; slot number
35 HLOOP: LD      A,C
36      CALL    CONOUT
37      CALL    PRINT
38      DEFB    ": ",0
39      PUSH    HL
40      LD      B,32
41      CALL    PRSTRN
42      CALL    CRLF
43      POP     HL
44      LD      DE,32
45      ADD     HL,DE
46      INC     C
47      LD      A,C
48      CP      '9'
49      JR      NZ,HLOOP
50      JP      PEXIT
51
52 NOREPLY:CALL    PRINT
53          DEFB    "NO REPLY FROM FUJINET",13,10,0
54          JP      PEXIT
55 GOTNAK:  CALL    PRINT
56          DEFB    "FUJINET SAID NAK",13,10,0
57          JP      PEXIT
58
59      INCLUDE  "../common/altair.inc"
60      INCLUDE  "../common/conio.inc"
61      INCLUDE  "../common/fujibus.inc"
62
63      DEFS     128
64 STACK:
65

```

APPENDIX C

CONFIG in Z80

Every FujiNet platform has a CONFIG program: the tool that names host slots, browses them, and puts disk images into disk slots. This one is written for a terminal or a Teletype on an Altair 8800 and the 88-2SIO, in plain scrolling text with no cursor addressing, and it uses only the commands of Chapter 7. It is about 700 lines of Z80 and assembles to under 6 KB.

C.1 What it does

CONFIG starts by showing everything at once — the network, the eight host slots and the eight disk slots — and then waits for a one-letter command:

Key	Action
H	Name a host slot (or clear it), then write the host table back.
B	Browse a host: walk its directories, choose an image, choose a disk slot and a mode, and mount it.
E	Eject the image in a disk slot.
W	Scan for WiFi networks and join one.
L	Show everything again.
Q	Quit.

Here is a whole session on the simulated Altair: mounting a CP/M disk from the SD card into disk slot 1, mounting the lobby disk from a TNFS server into slot 2, renaming host slot 6, and ejecting slot 2 again.

A CONFIG SESSION

```
FUJINET CONFIG FOR RS-232

NETWORK  Dummy Cafe  IP 127.0.0.1
FIRMWARE v1.6-670b2ed55

HOST SLOTS
1 SD
2 fujinet.online
3 apps.irata.online
4 fujinet.diller.org
5 FujiNet.Atari8bit.net
6 atarionline.eu
7 fujinet.pl
8 ec.tnfs.io

DISK SLOTS
1 (EMPTY)
2 (EMPTY)
3 (EMPTY)
4 (EMPTY)
5 (EMPTY)
6 (EMPTY)
7 (EMPTY)
8 (EMPTY)

H)OST B)ROWSE E)JECT W)IFI L)IST Q)UIT? B
HOST SLOT (1-8)? 1

HOST 1: /
1 cpm/
2 FujiNet/
3 games/
4 test/
5 --- FujiNet SD Card ---
(END)
NUMBER, N)EXT, U)P, X)IT? 1
```

```

HOST 1: /cpm/
 1 cpm22.dsk
 2 mbasic.dsk
 3 turbo3.dsk
 4 wordstar.dsk
 5 zork1.dsk
(END)
NUMBER, N)EXT, U)P, X)IT? 1
DISK SLOT (1-8)? 1
R)EAD-ONLY OR W)RITE? R
MOUNTED.

NETWORK Dummy Cafe IP 127.0.0.1
FIRMWARE v1.6-670b2ed55

HOST SLOTS
 1 SD
 2 fujinet.online
 3 apps.irata.online
 4 fujinet.diller.org
 5 FujiNet.Atari8bit.net
 6 atarionline.eu
 7 fujinet.pl
 8 ec.tnfs.io

DISK SLOTS
 1 R HOST 1 cpm22.dsk
 2 (EMPTY)
 3 (EMPTY)
 4 (EMPTY)
 5 (EMPTY)
 6 (EMPTY)
 7 (EMPTY)
 8 (EMPTY)

H)OST B)ROWSE E)JECT W)IFI L)IST Q)UIT? B
HOST SLOT (1-8)? 8

HOST 8: /
 1 apple2/
 2 atari/
 3 c64/
 4 coco/
 5 intv/
 6 msdos/
 7 utils/
 8 == New tnfs ==
(END)
NUMBER, N)EXT, U)P, X)IT? 6

HOST 8: /msdos/
 1 fbs-old.img
 2 fbs.img
 3 fcs.img
 4 fujirkle.img
 5 fujitzee - Copy.img
 6 fujitzee.img
 7 lobby.img
 8 texas.img
(END)
NUMBER, N)EXT, U)P, X)IT? 7
DISK SLOT (1-8)? 2
R)EAD-ONLY OR W)RITE? R
MOUNTED.

NETWORK Dummy Cafe IP 127.0.0.1
FIRMWARE v1.6-670b2ed55

HOST SLOTS
 1 SD
 2 fujinet.online
 3 apps.irata.online
 4 fujinet.diller.org
 5 FujiNet.Atari8bit.net
 6 atarionline.eu
 7 fujinet.pl
 8 ec.tnfs.io

```

C.2 Talking to the Fuji device

Most of CONFIG's commands take no parameters or one 8-bit parameter, so two small routines build and send them: FCMD0 for a Fuji command with no parameters, FCMD1 for one with a single byte in `E`. Both fall into CHECK, which turns a failed FBCALL into a message and returns with carry set, so callers can simply `RET C`:

FCMD0, FCMD1 AND CHECK

```

61 FCMD0: LD      C,A
62         LD      A,FUJI
63         LD      B,00h          ; descriptor 0: no parameters
64         CALL    FBNEW
65         JR      FDONE
66
67 ; FCMD1 -- Fuji command A with one u8 parameter in E
68 FCMD1: LD      C,A
69         LD      A,FUJI
70         LD      B,01h          ; descriptor 1: one u8
71         CALL    FBNEW
72         LD      A,E
73         CALL    FBBYTE
74 FDONE:  CALL    FBCALL
75         ; fall into CHECK
76
77 ; CHECK -- report a failed FBCALL. Carry clear (and HL, BC intact) on
78 ; an ACK; otherwise prints why and returns carry set.
79 CHECK:  JR      C,CHK1
80         RET      Z              ; ACK
81         CALL    PRINT
82         DEFB     "FUJINET SAID NAK",13,10,0
83         SCF
84         RET
85 CHK1:   CALL    PRINT
86         DEFB     "NO REPLY FROM FUJINET (ERROR ",0
87         LD      A,(FBERR)
88         CALL    PRHEX
89         CALL    PRINT
90         DEFB     ")",13,10,0
91         SCF
92         RET

```

C.3 The display

SHOWALL makes three requests. GET_ADAPTERCONFIG supplies the network name, the IP address (four bytes at offset 97, printed by PRIP) and the firmware version (offset 125). READ_HOST_SLOTS fills the 256-byte HOSTS table, which CONFIG keeps: the `H` command changes one name in it and writes the whole table back. READ_DEVICE_SLOTS fills SLOTS; for each 38-byte record CONFIG prints the mode (bit 1 of byte 1 is “write”), the host slot, and the file name:

PRINTING THE DISK SLOTS

```

154 ; READ_DEVICE_SLOTS (F2h) -> 8 x 38 bytes:
155 ;   +0 host slot (FFh = none), +1 mode, +2 file name (36 bytes)
156         LD      A,0F2h
157         CALL    FCMD0
158         RET      C
159         LD      DE,SLOTS
160         LD      BC,8*38
161         LDIR
162         CALL    PRINT
163         DEFB     13,10,"DISK SLOTS",13,10,0
164         LD      HL,SLOTS
165         LD      C,1
166 SHD1:   CALL    SLOTNUM
167         PUSH    HL
168         LD      A,(HL)          ; host slot

```

```

169      CP      0FFh
170      JR      Z,SHD2
171      INC     HL
172      INC     HL
173      LD      A,(HL)          ; file name
174      DEC     HL
175      DEC     HL
176      OR      A
177      JR      NZ,SHD3
178 SHD2:  CALL   PRINT
179      DEFB    "(EMPTY)",0
180      JR      SHD5
181 SHD3:  INC     HL          ; mode: bit 1 = write
182      LD      A,(HL)
183      DEC     HL
184      AND     02h
185      LD      A,'R'
186      JR      Z,SHD4
187      LD      A,'W'
188 SHD4:  CALL   CONOUT
189      CALL   PRINT
190      DEFB    " HOST ",0
191      LD      A,(HL)
192      ADD     A,'1'
193      CALL   CONOUT
194      CALL   PRINT
195      DEFB    " ",0
196      INC     HL
197      INC     HL
198      LD      B,36
199      CALL   PRSTRN
200 SHD5:  CALL   CRLF
201      POP     HL
202      LD      DE,38
203      ADD     HL,DE
204      INC     C
205      LD      A,C
206      CP      9
207      JR      NZ,SHD1
208      RET
209
210 ; SLOTNUM -- print " n " for slot number C

```

Note that CONFIG **never** writes SLOTS back: the slot table holds only file names, not paths (Chapter 7), and writing it back would lose every directory. Disk slots are changed one at a time, with SET_DEVICE_FULLPATH.

C.4 Naming a host

DOHOST asks for a slot number and a name, copies the name into the HOSTS table zero-filled to 32 bytes, and sends all 256 bytes with WRITE_HOST_SLOTS:

DOHOST

```

235 DOHOST: LD      HL,PHOST
236      LD      C,8
237      CALL   ASKNUM
238      RET     C
239      LD      (CURH),A
240      CALL   PRINT
241      DEFB    "HOST NAME (RETURN TO CLEAR)? ",0
242      LD      HL,LINE
243      LD      B,31
244      CALL   GETLN
245      LD      A,(CURH)      ; DE = HOSTS + slot x 32
246      LD      L,A
247      LD      H,0

```

```

248      ADD    HL,HL
249      ADD    HL,HL
250      ADD    HL,HL
251      ADD    HL,HL
252      ADD    HL,HL
253      LD     DE,HOSTS
254      ADD    HL,DE
255      EX     DE,HL
256      LD     HL,LINE      ; copy the name, zero-filled to 32
257      LD     B,32
258 DH1:   LD     A,(HL)
259      LD     (DE),A
260      OR     A
261      JR     Z,DH2
262      INC    HL
263 DH2:   INC    DE
264      DJNZ   DH1
265 ; WRITE_HOST_SLOTS (F3h): all 256 bytes in the payload
266      LD     A,FUJI
267      LD     C,0F3h
268      LD     B,00h
269      CALL   FBNEW
270      LD     HL,HOSTS
271      LD     BC,256
272      CALL   FBMEM
273      CALL   FBCALL
274      JP     CHECK

```

C.5 Browsing

Browsing is the heart of CONFIG. DOBROWSE mounts the chosen host and opens its root directory; BROPEN opens whatever directory `PATH` holds. The path is sent with FBSTR, which pads it to the 256 bytes OPEN_DIRECTORY requires:

BROPEN

```

305 ; OPEN_DIRECTORY (F7h): p0 = host slot, payload = path, exactly 256
306 BROPEN: LD     A,FUJI
307      LD     C,0F7h
308      LD     B,01h
309      CALL   FBNEW
310      LD     A,(CURH)
311      CALL   FBBYTE
312      LD     HL,PATH
313      LD     BC,256
314      CALL   FBSTR
315      CALL   FBCALL
316      CALL   CHECK
317      RET    C
318      XOR    A
319      LD     (ATEND),A

```

BRNEXT reads entries one at a time with READ_DIR_ENTRY, asking for 64 bytes each (long enough that the firmware rarely has to shorten a name), and stops at the `7F 7F` end marker or after a page of sixteen. Each name is kept in a page table so that the operator can pick one by number:

BRNEXT: READING ONE ENTRY

```

334 ; READ_DIR_ENTRY (F6h): p0 = length wanted, p1 = flags (0 = name only)
335 BRNEXT: LD     A,(ATEND)
336      OR     A
337      JR     NZ,BRASK
338      PUSH   DE
339      LD     A,FUJI
340      LD     C,0F6h

```

```

341      LD      B,02h          ; descriptor 2: two u8s
342      CALL    FBNEW
343      LD      A,ENTLEN
344      CALL    FBBYTE
345      XOR     A
346      CALL    FBBYTE
347      CALL    FBCALL
348      CALL    CHECK
349      POP     DE
350      JP      C,BRCLOSE
351      LD      A,(HL)          ; 7F 7F = end of directory
352      CP      7Fh
353      JR      NZ,BRN1
354      INC     HL
355      CP      (HL)
356      DEC     HL
357      JR      NZ,BRN1
358      LD      A,1
359      LD      (ATEND),A
360      JR      BRASK

```

A name that ends in `/` is a directory: CONFIG appends it to `PATH` and opens that. `U` climbs back up by cutting `PATH` after its second-to-last `/`. Anything else is a file, and BRFILE mounts it: it closes the directory, asks for a disk slot and a mode, joins `PATH` and the name into `FULL`, and sends SET_DEVICE_FULLPATH followed by MOUNT_IMAGE:

BRFILE: MOUNTING THE CHOSEN IMAGE

```

479 ; SET_DEVICE_FULLPATH (E2h): p0 disk slot, p1 host slot, p2 mode,
480 ; payload = the full path, exactly 256 bytes
481      LD      A,FUJI
482      LD      C,0E2h
483      LD      B,03h          ; descriptor 3: three u8s
484      CALL    FBNEW
485      LD      A,(CURD)
486      CALL    FBBYTE
487      LD      A,(CURH)
488      CALL    FBBYTE
489      LD      A,(MODE)
490      CALL    FBBYTE
491      LD      HL,FULL
492      LD      BC,256
493      CALL    FBSTR
494      CALL    FBCALL
495      CALL    CHECK
496      RET     C
497 ; MOUNT_IMAGE (F8h): p0 disk slot, p1 mode
498      LD      A,FUJI
499      LD      C,0F8h
500      LD      B,02h
501      CALL    FBNEW
502      LD      A,(CURD)
503      CALL    FBBYTE
504      LD      A,(MODE)
505      CALL    FBBYTE
506      CALL    FBCALL
507      CALL    CHECK
508      RET     C
509      CALL    PRINT
510      DEFB    "MOUNTED.",13,10,0
511      RET

```

C.6 Joining a network

DOWIFI scans, lists up to sixteen networks with their signal strength (a signed byte, printed with NEG), asks for one and a password, and sends the 97-byte SET_SSID. Joining takes a while, so it raises FBWAIT to 120 timeouts, about thirty seconds, for that one command:

JOINING A NETWORK

```

588 ; SET_SSID (FBh): payload = SSID (33) + password (64), exactly 97
589     LD     A,FUJI
590     LD     C,0FBh
591     LD     B,00h
592     CALL   FBNEW
593     LD     HL,SSID
594     LD     BC,33
595     CALL   FBSTR
596     LD     HL,PASS
597     LD     BC,64
598     CALL   FBSTR
599     LD     A,120           ; joining can take a while: allow 30 s
600     LD     (FBWAIT),A
601     CALL   FBCALL
602     LD     A,40
603     LD     (FBWAIT),A
604     CALL   CHECK
605     RET     C
606     CALL   PRINT
607     DEFB   "JOINED.",13,10,0
608     RET

```

C.7 Listing

FNCFG.ASM

```

1 ;=====
2 ; FNCFG.ASM -- CONFIG for the RS-232 FujiNet, in Z80 assembler
3 ;
4 ; A Teletype-friendly CONFIG: shows the network, the eight host slots
5 ; and the eight disk slots, then lets the operator
6 ;   H name a host slot      B browse a host and mount a disk
7 ;   E eject a disk slot     W join a WiFi network
8 ;   L list everything again  Q quit
9 ;
10 ; Plain scrolling text, no cursor addressing: it works the same on a
11 ; glass terminal or an ASR-33. Hardware lives in ALTAIR.INC (88-2SIO:
12 ; terminal on port A, FujiNet on port B); the packet layer is
13 ; FUJIBUS.INC. Assemble: z88dk-z80asm -b fncfg.asm
14 ;=====
15
16     ORG     0100h
17
18 FUJI    EQU     70h           ; the Fuji device
19 ENTLEN  EQU     64           ; directory entry length we ask for
20 PAGELEN EQU     16           ; entries shown per page
21
22 START:  LD      SP,STACK
23         CALL    PINIT
24         CALL    PRINT
25         DEFB    13,10,"FUJINET CONFIG FOR RS-232",13,10,0
26 MLOOP:  CALL    SHOWALL
27 MPROMPT:CALL    PRINT
28         DEFB    13,10,"(H)OST (B)ROWSE (E)JECT (W)IFI (L)IST (Q)UIT? ",0
29         CALL    GETKEY
30         PUSH    AF
31         CALL    CONOUT
32         CALL    CRLF
33         POP     AF
34         CP      'L'
35         JR      Z,MLOOP
36         CP      'H'
37         JR      Z,MHOST
38         CP      'B'
39         JR      Z,MBROWSE
40         CP      'E'
41         JR      Z,MEJECT
42         CP      'W'

```

```

43      JR      Z,MWIFI
44      CP      'Q'
45      JP      Z,PEXIT
46      JR      MPROMPT
47 MHOST: CALL  DOHOST
48      JR      MLOOP
49 MBROWSE:CALL DOBROWSE
50      JR      MLOOP
51 MEJECT: CALL  DOEJECT
52      JR      MLOOP
53 MWIFI:  CALL  DOWIFI
54      JR      MLOOP
55
56 ;=====
57 ; Talking to the Fuji device
58 ;=====
59
60 ; FCMD0 -- Fuji command A with no parameters
61 FCMD0: LD     C,A
62      LD     A,FUJI
63      LD     B,00h          ; descriptor 0: no parameters
64      CALL   FBNEW
65      JR      FDONE
66
67 ; FCMD1 -- Fuji command A with one u8 parameter in E
68 FCMD1: LD     C,A
69      LD     A,FUJI
70      LD     B,01h          ; descriptor 1: one u8
71      CALL   FBNEW
72      LD     A,E
73      CALL   FBBYTE
74 FDONE: CALL   FBCALL
75          ; fall into CHECK
76
77 ; CHECK -- report a failed FBCALL. Carry clear (and HL, BC intact) on
78 ; an ACK; otherwise prints why and returns carry set.
79 CHECK: JR     C,CHK1
80      RET     Z          ; ACK
81      CALL   PRINT
82      DEFB   "FUJINET SAID NAK",13,10,0
83      SCF
84      RET
85 CHK1:  CALL   PRINT
86      DEFB   "NO REPLY FROM FUJINET (ERROR ",0
87      LD     A,(FBERR)
88      CALL   PRHEX
89      CALL   PRINT
90      DEFB   ")",13,10,0
91      SCF
92      RET
93
94 ;=====
95 ; SHOWALL -- network, host slots, disk slots
96 ;=====
97 SHOWALL:
98 ; GET_ADAPTERCONFIG (E8h) -> 140 bytes: SSID at +0, IP at +97,
99 ; firmware version at +125
100      LD     A,0E8h
101      CALL   FCMD0
102      RET     C
103      PUSH   HL
104      CALL   PRINT
105      DEFB   13,10,"NETWORK ",0
106      LD     B,32
107      CALL   PRSTRN          ; the SSID
108      CALL   PRINT
109      DEFB   " IP ",0
110      POP    HL
111      PUSH   HL
112      LD     DE,97
113      ADD    HL,DE
114      CALL   PRIP
115      CALL   PRINT
116      DEFB   13,10,"FIRMWARE ",0
117      POP    HL
118      LD     DE,125
119      ADD    HL,DE
120      LD     B,15
121      CALL   PRSTRN
122      CALL   CRLF
123
124 ; READ_HOST_SLOTS (F4h) -> 8 x 32 bytes
125      LD     A,0F4h
126      CALL   FCMD0
127      RET     C
128      LD     DE,HOSTS
129      LD     BC,256

```

```

130      LDIR
131      CALL PRINT
132      DEFB 13,10,"HOST SLOTS",13,10,0
133      LD HL,HOSTS
134      LD C,1
135 SHH1: CALL SLOTNUM
136      PUSH HL
137      LD A,(HL)
138      OR A
139      JR NZ,SHH2
140      CALL PRINT
141      DEFB "(EMPTY)",0
142      JR SHH3
143 SHH2: LD B,32
144      CALL PRSTRN
145 SHH3: CALL CRLF
146      POP HL
147      LD DE,32
148      ADD HL,DE
149      INC C
150      LD A,C
151      CP 9
152      JR NZ,SHH1
153
154 ; READ_DEVICE_SLOTS (F2h) -> 8 x 38 bytes:
155 ; +0 host slot (FFh = none), +1 mode, +2 file name (36 bytes)
156      LD A,0F2h
157      CALL FCMD0
158      RET C
159      LD DE,SLOTS
160      LD BC,8*38
161      LDIR
162      CALL PRINT
163      DEFB 13,10,"DISK SLOTS",13,10,0
164      LD HL,SLOTS
165      LD C,1
166 SHD1: CALL SLOTNUM
167      PUSH HL
168      LD A,(HL) ; host slot
169      CP 0FFh
170      JR Z,SHD2
171      INC HL
172      INC HL
173      LD A,(HL) ; file name
174      DEC HL
175      DEC HL
176      OR A
177      JR NZ,SHD3
178 SHD2: CALL PRINT
179      DEFB "(EMPTY)",0
180      JR SHD5
181 SHD3: INC HL ; mode: bit 1 = write
182      LD A,(HL)
183      DEC HL
184      AND 02h
185      LD A,'R'
186      JR Z,SHD4
187      LD A,'W'
188 SHD4: CALL CONOUT
189      CALL PRINT
190      DEFB " HOST ",0
191      LD A,(HL)
192      ADD A,'1'
193      CALL CONOUT
194      CALL PRINT
195      DEFB " ",0
196      INC HL
197      INC HL
198      LD B,36
199      CALL PRSTRN
200 SHD5: CALL CRLF
201      POP HL
202      LD DE,38
203      ADD HL,DE
204      INC C
205      LD A,C
206      CP 9
207      JR NZ,SHD1
208      RET
209
210 ; SLOTNUM -- print " n " for slot number C
211 SLOTNUM:CALL SPACE
212      LD A,C
213      ADD A,'0'
214      CALL CONOUT
215      CALL SPACE
216      JP SPACE

```

```

217
218 ; PRIP -- print the four bytes at (HL) as a dotted IP address
219 PRIP: LD B,4
220 PRIP1: PUSH HL
221 LD L,(HL)
222 LD H,0
223 CALL PRDEC
224 POP HL
225 INC HL
226 DEC B
227 RET Z
228 LD A,'.'
229 CALL CONOUT
230 JR PRIP1
231
232 ;=====
233 ; H -- name a host slot, then WRITE_HOST_SLOTS
234 ;=====
235 DOHOST: LD HL,PHOST
236 LD C,8
237 CALL ASKNUM
238 RET C
239 LD (CURH),A
240 CALL PRINT
241 DEFB "HOST NAME (RETURN TO CLEAR)? ",0
242 LD HL,LINE
243 LD B,31
244 CALL GETLN
245 LD A,(CURH) ; DE = HOSTS + slot x 32
246 LD L,A
247 LD H,0
248 ADD HL,HL
249 ADD HL,HL
250 ADD HL,HL
251 ADD HL,HL
252 ADD HL,HL
253 LD DE,HOSTS
254 ADD HL,DE
255 EX DE,HL
256 LD HL,LINE ; copy the name, zero-filled to 32
257 LD B,32
258 DH1: LD A,(HL)
259 LD (DE),A
260 OR A
261 JR Z,DH2
262 INC HL
263 DH2: INC DE
264 DJNZ DH1
265 ; WRITE_HOST_SLOTS (F3h): all 256 bytes in the payload
266 LD A,FUJI
267 LD C,0F3h
268 LD B,00h
269 CALL FBNEW
270 LD HL,HOSTS
271 LD BC,256
272 CALL FBMEM
273 CALL FBCALL
274 JP CHECK
275
276 ;=====
277 ; E -- eject (UNMOUNT_IMAGE) a disk slot
278 ;=====
279 DOEJECT: LD HL,PDRIVE
280 LD C,8
281 CALL ASKNUM
282 RET C
283 LD E,A
284 LD A,0E9h ; UNMOUNT_IMAGE, p0 = disk slot
285 JP FCMD1
286
287 ;=====
288 ; B -- browse a host, pick an image, mount it
289 ;=====
290 DOBROWSE:
291 LD HL,PHOST
292 LD C,8
293 CALL ASKNUM
294 RET C
295 LD (CURH),A
296 LD E,A
297 LD A,0F9h ; MOUNT_HOST, p0 = host slot
298 CALL FCMD1
299 RET C
300 LD HL,PATH ; start at the root
301 LD (HL),'/'
302 INC HL
303 LD (HL),0

```

```

304
305 ; OPEN_DIRECTORY (F7h): p0 = host slot, payload = path, exactly 256
306 BROPEN: LD      A,FUJI
307         LD      C,0F7h
308         LD      B,01h
309         CALL    FBNEW
310         LD      A,(CURH)
311         CALL    FBBYTE
312         LD      HL,PATH
313         LD      BC,256
314         CALL    FBSTR
315         CALL    FBCALL
316         CALL    CHECK
317         RET     C
318         XOR     A
319         LD      (ATEND),A
320
321 BRPAGE: CALL    PRINT
322         DEFB    13,10,"HOST ",0
323         LD      A,(CURH)
324         ADD     A,'1'
325         CALL    CONOUT
326         CALL    PRINT
327         DEFB    ":",0
328         LD      HL,PATH
329         CALL    PRSTR
330         CALL    CRLF
331         XOR     A
332         LD      (NENT),A
333         LD      DE,PAGE
334 ; READ_DIR_ENTRY (F6h): p0 = length wanted, p1 = flags (0 = name only)
335 BRNEXT: LD      A,(ATEND)
336         OR      A
337         JR      NZ,BRASK
338         PUSH    DE
339         LD      A,FUJI
340         LD      C,0F6h
341         LD      B,02h           ; descriptor 2: two u8s
342         CALL    FBNEW
343         LD      A,ENTLEN
344         CALL    FBBYTE
345         XOR     A
346         CALL    FBBYTE
347         CALL    FBCALL
348         CALL    CHECK
349         POP     DE
350         JP      C,BRCLOSE
351         LD      A,(HL)           ; 7F 7F = end of directory
352         CP      7Fh
353         JR      NZ,BRN1
354         INC     HL
355         CP      (HL)
356         DEC     HL
357         JR      NZ,BRN1
358         LD      A,1
359         LD      (ATEND),A
360         JR      BRASK
361 BRN1:   PUSH    DE           ; keep the name in the page table
362         LD      BC,ENTLEN
363         LDIR
364         XOR     A
365         LD      (DE),A           ; and make sure it ends
366         POP     HL           ; HL = the stored name
367         LD      DE,ENTLEN+1
368         EX      DE,HL
369         ADD     HL,DE           ; DE = name, HL = next table slot
370         EX      DE,HL
371         PUSH    DE
372         LD      A,(NENT)
373         INC     A
374         LD      (NENT),A
375         PUSH    HL           ; print " nn  name"
376         LD      L,A
377         LD      H,0
378         LD      B,3
379         CALL    PRDECW
380         CALL    SPACE
381         CALL    SPACE
382         POP     HL
383         CALL    PRSTR
384         CALL    CRLF
385         POP     DE
386         LD      A,(NENT)
387         CP      PAGELEN
388         JR      C,BRNEXT
389
390 BRASK:  LD      A,(ATEND)

```

```

391      OR      A
392      JR      Z,BRA1
393      CALL    PRINT
394      DEFB    "(END)",13,10,0
395  BRA1:  CALL    PRINT
396      DEFB    "NUMBER, N)EXT, U)P, X)IT? ",0
397      LD      HL,LINE
398      LD      B,3
399      CALL    GETLN
400      LD      A,(LINE)
401      CALL    UCASE
402      CP      'N'
403      JP      Z,BRPAGE
404      CP      'U'
405      JR      Z,BRUP
406      CP      'X'
407      JR      Z,BRCLOSE
408      OR      A
409      JR      Z,BRCLOSE
410      LD      A,(NENT)
411      LD      C,A
412      CALL    PARSE      ; A = choice - 1
413      JR      C,BRA1
414      LD      L,A      ; HL = PAGE + A x (ENTLEN+1)
415      LD      H,0
416      LD      DE,ENTLEN+1
417      CALL    MUL16
418      LD      DE,PAGE
419      ADD     HL,DE
420      LD      (CHOSEN),HL
421      CALL    STREND      ; a directory ends in '/'
422      DEC     HL
423      LD      A,(HL)
424      CP      '/'
425      JR      NZ,BRFILE
426      LD      HL,PATH      ; descend: PATH = PATH + name
427      CALL    STREND
428      EX      DE,HL
429      LD      HL,(CHOSEN)
430      CALL    STRCPY
431      JP      BROPEN
432
433  BRUP:  LD      HL,PATH      ; climb: drop the last "name/"
434      CALL    STREND
435      DEC     HL      ; the trailing '/'
436  BRU1:  LD      DE,PATH
437      OR      A
438      SBC     HL,DE
439      ADD     HL,DE
440      JP      Z,BROPEN      ; back at the root "/"
441      DEC     HL
442      LD      A,(HL)
443      CP      '/'
444      JR      NZ,BRU1
445      INC     HL
446      LD      (HL),0
447      JP      BROPEN
448
449  BRCLOSE:LD  A,0F5h      ; CLOSE_DIRECTORY
450      JP      FCMD0
451
452 ; A file was chosen: close the directory, choose a slot and a mode,
453 ; SET_DEVICE_FULLPATH, then MOUNT_IMAGE.
454  BRFILE: LD  A,0F5h
455      CALL    FCMD0
456      LD      HL,PDRIVE
457      LD      C,8
458      CALL    ASKNUM
459      RET     C
460      LD      (CURD),A
461      CALL    PRINT
462      DEFB    "R)EAD-ONLY OR W)RITE? ",0
463      CALL    GETKEY
464      PUSH    AF
465      CALL    CONOUT
466      CALL    CRLF
467      POP     AF
468      LD      B,1      ; mode 1 = read
469      CP      'W'
470      JR      NZ,BRF1
471      LD      B,2      ; mode 2 = read/write
472  BRF1:  LD      A,B
473      LD      (MODE),A
474      LD      DE,FULL      ; FULL = PATH + name
475      LD      HL,PATH
476      CALL    STRCPY
477      LD      HL,(CHOSEN)

```

```

478      CALL      STRCPY
479 ; SET_DEVICE_FULLPATH (E2h): p0 disk slot, p1 host slot, p2 mode,
480 ; payload = the full path, exactly 256 bytes
481      LD        A,FUJI
482      LD        C,0E2h
483      LD        B,03h          ; descriptor 3: three u8s
484      CALL      FBNEW
485      LD        A,(CURD)
486      CALL      FBBYTE
487      LD        A,(CURH)
488      CALL      FBBYTE
489      LD        A,(MODE)
490      CALL      FBBYTE
491      LD        HL,FULL
492      LD        BC,256
493      CALL      FBSTR
494      CALL      FBCALL
495      CALL      CHECK
496      RET        C
497 ; MOUNT_IMAGE (F8h): p0 disk slot, p1 mode
498      LD        A,FUJI
499      LD        C,0F8h
500      LD        B,02h
501      CALL      FBNEW
502      LD        A,(CURD)
503      CALL      FBBYTE
504      LD        A,(MODE)
505      CALL      FBBYTE
506      CALL      FBCALL
507      CALL      CHECK
508      RET        C
509      CALL      PRINT
510      DEFB      "MOUNTED.",13,10,0
511      RET
512
513 ;=====
514 ; W -- scan for networks and join one
515 ;=====
516 DOWIFI: CALL      PRINT
517      DEFB      "SCANNING...",13,10,0
518      LD        A,0FDh          ; SCAN_NETWORKS -> 1 byte: count
519      CALL      FCMD0
520      RET        C
521      LD        A,(HL)
522      OR        A
523      RET        Z
524      CP        PAGELEN+1        ; show at most 16
525      JR        C,DW1
526      LD        A,PAGELEN
527 DW1:   LD        (NENT),A
528      LD        C,0
529 DW2:   LD        E,C          ; GET_SCAN_RESULT (FCh), p0 = index
530      PUSH      BC
531      LD        A,0FCh
532      CALL      FCMD1          ; -> 33-byte SSID, then signed RSSI
533      POP       BC
534      RET        C
535      PUSH      BC
536      PUSH      HL
537      LD        L,C
538      INC       L
539      LD        H,0
540      LD        B,3
541      CALL      PRDECW
542      CALL      SPACE
543      CALL      SPACE
544      POP       HL
545      PUSH      HL
546      LD        B,32
547      CALL      PRSTRN
548      CALL      PRINT
549      DEFB      " ",0
550      POP       HL
551      LD        DE,33
552      ADD       HL,DE
553      LD        A,(HL)          ; RSSI in dBm, negative
554      OR        A
555      JP        P,DW3
556      NEG
557      PUSH      AF
558      LD        A,'-'
559      CALL      CONOUT
560      POP       AF
561 DW3:   LD        L,A
562      LD        H,0
563      CALL      PRDEC
564      CALL      PRINT

```

```

565     DEFB     " DBM",13,10,0
566     POP      BC
567     INC       C
568     LD        A,(NENT)
569     CP        C
570     JR        NZ,DW2
571     LD        HL,PNET
572     LD        A,(NENT)
573     LD        C,A
574     CALL      ASKNUM
575     RET       C
576     LD        E,A           ; fetch that entry's SSID again
577     LD        A,0FCh
578     CALL      FCMD1
579     RET       C
580     LD        DE,SSID
581     LD        BC,33
582     LDIR
583     CALL      PRINT
584     DEFB      "PASSWORD? ",0
585     LD        HL,PASS
586     LD        B,63
587     CALL      GETLN
588 ; SET_SSID (FBh): payload = SSID (33) + password (64), exactly 97
589     LD        A,FUJI
590     LD        C,0FBh
591     LD        B,00h
592     CALL      FBNEW
593     LD        HL,SSID
594     LD        BC,33
595     CALL      FBSTR
596     LD        HL,PASS
597     LD        BC,64
598     CALL      FBSTR
599     LD        A,120         ; joining can take a while: allow 30 s
600     LD        (FBWAIT),A
601     CALL      FBCALL
602     LD        A,40
603     LD        (FBWAIT),A
604     CALL      CHECK
605     RET       C
606     CALL      PRINT
607     DEFB      "JOINED.",13,10,0
608     RET
609
610 ;=====
611 ; Small helpers
612 ;=====
613
614 ; ASKNUM -- print the prompt at HL, read a number 1..C;
615 ; A = number - 1 with carry clear, or carry set if blank or invalid
616 ASKNUM: CALL    PRSTR
617         PUSH     BC
618         LD       HL,LINE
619         LD       B,3
620         CALL     GETLN
621         POP      BC
622
623 ; fall into PARSE
624 ; PARSE -- the decimal number in LINE, 1..C -> A = number - 1
625 PARSE:  LD       HL,LINE
626         LD       A,(HL)
627         OR       A
628         SCF
629         RET      Z           ; blank
630         LD       B,0         ; B = value
631 PA1:    LD       A,(HL)
632         OR       A
633         JR       Z,PA2
634         SUB      '0'
635         RET      C           ; not a digit
636         CP       10
637         CCF
638         RET      C
639         LD       D,A         ; B = B x 10 + digit
640         LD       A,B
641         ADD      A,A
642         LD       E,A
643         ADD      A,A
644         LD       A,E
645         ADD      A,D
646         LD       B,A
647         INC      HL
648         JR       PA1
649 PA2:    LD       A,B
650         OR       A
651         SCF

```

```

652      RET      Z          ; zero is not a choice
653      CP      C
654      JR      Z,PA3
655      CCF
656      RET      C          ; bigger than C
657 PA3:  DEC      A
658      OR      A          ; clear carry
659      RET
660
661 ; STREND -- HL = address of the NUL ending the string at HL
662 STREND: LD      A,(HL)
663      OR      A
664      RET      Z
665      INC     HL
666      JR      STREND
667
668 ; STRCPY -- copy the string at HL to DE, including its NUL; on return
669 ; DE points at the copied NUL (so a second STRCPY appends)
670 STRCPY: LD      A,(HL)
671      LD      (DE),A
672      OR      A
673      RET      Z
674      INC     HL
675      INC     DE
676      JR      STRCPY
677
678 ; MUL16 -- HL = HL x DE (low 16 bits)
679 MUL16: PUSH     BC
680      LD      B,H
681      LD      C,L
682      LD      HL,0
683      LD      A,16
684 MU1:  ADD      HL,HL
685      EX      DE,HL
686      ADD      HL,HL
687      EX      DE,HL
688      JR      NC,MU2
689      ADD      HL,BC
690 MU2:  DEC      A
691      JR      NZ,MU1
692      POP     BC
693      RET
694
695 PHOST: DEFB     "HOST SLOT (1-8)? ",0
696 PDRIVE: DEFB    "DISK SLOT (1-8)? ",0
697 PNET:  DEFB     "NETWORK NUMBER? ",0
698
699      INCLUDE  "../common/altair.inc"
700      INCLUDE  "../common/conio.inc"
701      INCLUDE  "../common/fujibus.inc"
702
703 ;=====
704 ; Variables and buffers
705 ;=====
706 CURH:  DEFB     0          ; host slot being worked on (0-7)
707 CURD:  DEFB     0          ; disk slot being worked on (0-7)
708 MODE:  DEFB     0          ; 1 = read, 2 = read/write
709 NENT:  DEFB     0          ; entries on this page
710 ATEND: DEFB     0          ; directory exhausted
711 CHOSEN: DEFW     0          ; the chosen entry's name
712 LINE:  DEFS     32
713 HOSTS: DEFS     256         ; 8 x 32: the host slot names
714 SLOTS: DEFS     8*38        ; 8 x 38: the disk slots
715 PATH:  DEFS     256         ; current directory, ends in '/'
716 FULL:  DEFS     256         ; directory + file name
717 SSID:  DEFS     33
718 PASS:  DEFS     64
719 PAGE:  DEFS     PAGELEN*(ENTLEN+1)
720      DEFS     256
721 STACK:
722

```

APPENDIX D

NETCAT in Z80

NETCAT turns the Altair into a terminal for any TCP or TELNET host: a bulletin board, a MUD, a shell on another machine. It asks for a devicespec, opens it on network unit 1, and then does two things in a loop: keys typed on the terminal are sent with WRITE, and whatever the far end has sent is collected with STATUS and READ and printed. Ctrl-] hangs up.

NETCAT TALKING TO A TEST SERVER

```
NETCAT FOR THE RS-232 FUJINET

CONNECT TO (RETURN TO QUIT)?
> N:TCP://127.0.0.1:7777/
CONNECTED.  CTRL-] HANGS UP.
WELCOME TO THE ECHO SERVER
TYPE A LINE, OR BYE.
YOU SAID: HELLO FROM THE ALTAIR
SO LONG.

DISCONNECTED.

CONNECT TO (RETURN TO QUIT)?
> N:TCP://127.0.0.1:1/
OPEN FAILED, ERROR 200

CONNECT TO (RETURN TO QUIT)?
> N:TELNET://nonexistent.invalid/
OPEN FAILED, ERROR 201

CONNECT TO (RETURN TO QUIT)?
>
```

The typed lines do not appear: NETCAT does not echo, because the far end usually does (a TELNET host always will). The two failed connections show STATUS's error byte after a refused OPEN: **c8** (200) connection refused, and **c9** (201) network unreachable for a host name that does not resolve.

D.1 Opening

The devicespec is sent at its natural length with its NUL: unlike the Fuji device's paths, OPEN's devicespec is a variable-length payload. OPEN can wait for DNS and a TCP or TLS handshake, so NETCAT allows twenty seconds:

OPENING THE CONNECTION

```
34 ; OPEN ('0'): p0 = mode 12 (read/write), p1 = translation 0 (none),
35 ; payload = the devicespec, NUL-terminated
36     LD     A,NET
37     LD     C,'0'
38     LD     B,02h           ; descriptor 2: two u8s
39     CALL   FBNEW
40     LD     A,12
41     CALL   FBBYTE
42     XOR    A
43     CALL   FBBYTE
44     LD     HL,URL
45     CALL   STRLEN           ; BC = length, then the NUL too
46     INC    BC
47     CALL   FBMEM
48     LD     A,80           ; DNS + TLS can be slow: allow 20 s
49     LD     (FBWAIT),A
50     CALL   FBCALL
```

```

51      LD      A,40
52      LD      (FBWAIT),A
53      JP      C,NOREPLY
54      JR      Z,OPENED
55      CALL    STATUS      ; NAK: STATUS says why
56      CALL    PRINT
57      DEFB    "OPEN FAILED, ERROR ",0
58      LD      A,(ERR)
59      LD      L,A
60      LD      H,0
61      CALL    PRDEC
62      CALL    CRLF
63      JP      ASKURL

```

When OPEN answers NAK, NETCAT sends STATUS to learn why.

D.2 The loop

THE TERMINAL LOOP

```

71 TLOOP: CALL    CONST      ; a key?
72      JR      Z,TL2
73      CALL    CONIN
74      CP      HANGUP
75      JR      Z,BYE
76 ; WRITE ('W'): p0 = byte count (u16), payload = the bytes
77      LD      (KEY),A
78      LD      A,NET
79      LD      C,'W'
80      LD      B,05h      ; descriptor 5: one u16
81      CALL    FBNEW
82      LD      HL,1
83      CALL    FBWORD
84      LD      A,(KEY)
85      CALL    FBBYTE
86      CALL    FBCALL
87      JR      C,NOREPLY
88 TL2:  CALL    STATUS
89      JR      C,NOREPLY
90      LD      HL,(AVAIL)
91      LD      A,H
92      OR      L
93      JR      NZ,TL3
94      LD      A,(CONN)      ; nothing waiting: still connected?
95      OR      A
96      JR      NZ,TLOOP
97      CALL    PRINT
98      DEFB    13,10,"DISCONNECTED.",13,10,0
99      JR      CLOSE
100 TL3: LD      DE,CHUNK      ; ask for min(AVAIL, CHUNK)
101      OR      A
102      SBC     HL,DE
103      ADD     HL,DE
104      JR      C,TL4
105      EX      DE,HL
106 ; READ ('R'): p0 = byte count (u16) -> the bytes
107 TL4: PUSH    HL
108      LD      A,NET
109      LD      C,'R'
110      LD      B,05h
111      CALL    FBNEW
112      POP     HL
113      CALL    FBWORD
114      CALL    FBCALL
115      JR      C,NOREPLY
116      JR      NZ,TLOOP

```

```

117 TL5:    LD      A,B          ; print BC bytes from (HL)
118        OR      C
119        JR      Z,TLOOP
120        LD      A,(HL)
121        CALL    CONOUT
122        INC     HL
123        DEC     BC
124        JR      TL5

```

Each pass sends at most one key and collects at most 256 bytes. When nothing is waiting and the connected byte has gone to zero, the far end has hung up. At 9600 baud a STATUS exchange takes about 20 ms, so NETCAT polls some fifty times a second; a key typed while a READ is being printed waits in the 6850's receive register and is sent on the next pass.

NOTE

Printing on a slow terminal holds up the loop, but loses nothing: the remote host's data waits in the FujiNet's buffer until NETCAT asks for it. The same program works on a Teletype at 110 baud — slowly.

What could be added

The RI line (Chapter 2) could replace the polling, if the serial board offers an input to wire it to: STATUS would then be sent only when RI goes off. A line-at-a-time mode would send fewer, larger WRITES; a local-echo switch would help with raw TCP hosts that do not echo.

D.3 Listing

NETCAT.ASM

```

1  ;=====
2  ; NETCAT.ASM -- a dumb terminal to anywhere, through the FujiNet
3  ;
4  ; Asks for a devicespec (N:TCP://host:port/ or N:TELNET://host/),
5  ; opens it on network unit 1, then shuttles bytes: keys typed on the
6  ; terminal go out with WRITE, and whatever the far end sends comes
7  ; back with STATUS + READ and is printed.  Ctrl-] hangs up.
8  ;
9  ; Hardware lives in ALTAIR.INC (88-2SI0: terminal on port A, FujiNet
10 ; on port B); the packet layer is FUJIBUS.INC.
11 ; Assemble:  z88dk-z80asm -b netcat.asm
12 ;=====
13
14         ORG     0100h
15
16 NET      EQU     71h          ; network unit 1 (N1:)
17 HANGUP   EQU     10h          ; Ctrl-]
18 CHUNK    EQU     256          ; most bytes asked for in one READ
19
20 START:   LD      SP,STACK
21         CALL    PINIT
22         CALL    PRINT
23         DEFB     13,10,"NETCAT FOR THE RS-232 FUJINET",13,10,0
24
25 ASKURL:  CALL    PRINT
26         DEFB     13,10,"CONNECT TO (RETURN TO QUIT)?",13,10,"> ",0
27         LD      HL,URL
28         LD      B,250
29         CALL    GETLN
30         LD      A,B
31         OR      A
32         JP      Z,PEXIT
33
34 ; OPEN ('O'): p0 = mode 12 (read/write), p1 = translation 0 (none),
35 ; payload = the devicespec, NUL-terminated
36         LD      A,NET
37         LD      C,'O'
38         LD      B,02h          ; descriptor 2: two u8s
39         CALL    FBNEW
40         LD      A,12
41         CALL    FBBYTE
42         XOR     A
43         CALL    FBBYTE
44         LD      HL,URL
45         CALL    STRLEN          ; BC = length, then the NUL too
46         INC     BC

```

```

47      CALL    FBMEM
48      LD      A,80                ; DNS + TLS can be slow: allow 20 s
49      LD      (FBWAIT),A
50      CALL    FBCALL
51      LD      A,40
52      LD      (FBWAIT),A
53      JP      C,NOREPLY
54      JR      Z,OPENED
55      CALL    STATUS              ; NAK: STATUS says why
56      CALL    PRINT
57      DEFB    "OPEN FAILED, ERROR ",0
58      LD      A,(ERR)
59      LD      L,A
60      LD      H,0
61      CALL    PRDEC
62      CALL    CRLF
63      JP      ASKURL
64
65 OPENED: CALL    PRINT
66      DEFB    "CONNECTED. CTRL-] HANGS UP.",13,10,0
67
68 ;-----
69 ; The terminal loop
70 ;-----
71 TLOOP: CALL    CONST              ; a key?
72      JR      Z,TL2
73      CALL    CONIN
74      CP      HANGUP
75      JR      Z,BYE
76 ; WRITE ('W'): p0 = byte count (u16), payload = the bytes
77      LD      (KEY),A
78      LD      A,NET
79      LD      C,'W'
80      LD      B,05h                ; descriptor 5: one u16
81      CALL    FBNEW
82      LD      HL,1
83      CALL    FBWORD
84      LD      A,(KEY)
85      CALL    FBBYTE
86      CALL    FBCALL
87      JR      C,NOREPLY
88 TL2:   CALL    STATUS
89      JR      C,NOREPLY
90      LD      HL,(AVAIL)
91      LD      A,H
92      OR      L
93      JR      NZ,TL3
94      LD      A,(CONN)              ; nothing waiting: still connected?
95      OR      A
96      JR      NZ,TLOOP
97      CALL    PRINT
98      DEFB    13,10,"DISCONNECTED.",13,10,0
99      JR      CLOSE
100 TL3:  LD      DE,CHUNK              ; ask for min(AVAIL, CHUNK)
101      OR      A
102      SBC     HL,DE
103      ADD     HL,DE
104      JR      C,TL4
105      EX      DE,HL
106 ; READ ('R'): p0 = byte count (u16) -> the bytes
107 TL4:  PUSH    HL
108      LD      A,NET
109      LD      C,'R'
110      LD      B,05h
111      CALL    FBNEW
112      POP     HL
113      CALL    FBWORD
114      CALL    FBCALL
115      JR      C,NOREPLY
116      JR      NZ,TLOOP
117 TL5:  LD      A,B                  ; print BC bytes from (HL)
118      OR      C
119      JR      Z,TLOOP
120      LD      A,(HL)
121      CALL    CONOUT
122      INC     HL
123      DEC     BC
124      JR      TL5
125
126 BYE:  CALL    PRINT
127      DEFB    13,10,"HANGING UP.",13,10,0
128 ; CLOSE ('C'): no parameters
129 CLOSE: LD      A,NET
130      LD      C,'C'
131      LD      B,00h
132      CALL    FBNEW
133      CALL    FBCALL

```

```

134         JP      ASKURL
135
136 NOREPLY:CALL    PRINT
137         DEFB    13,10,"NO REPLY FROM FUJINET.",13,10,0
138         JP      ASKURL
139
140 ;-----
141 ; STATUS ('S') -> 4 bytes: bytes waiting (u16), connected, error
142 ;-----
143 STATUS: LD      A,NET
144         LD      C,'S'
145         LD      B,00h
146         CALL    FBNEW
147         CALL    FBCALL
148         RET     C
149         LD      DE,AVAIL
150         LD      BC,4
151         LDIR
152         OR      A
153         RET
154
155 ; STRLEN -- BC = length of the string at HL
156 STRLEN: PUSH    HL
157         LD      BC,0
158 SL1:     LD      A,(HL)
159         OR      A
160         JR      Z,SL2
161         INC     HL
162         INC     BC
163         JR      SL1
164 SL2:     POP     HL
165         RET
166
167         INCLUDE "../common/altair.inc"
168         INCLUDE "../common/conio.inc"
169         INCLUDE "../common/fujibus.inc"
170
171 AVAIL:   DEFW    0           ; the STATUS reply, as it arrives
172 CONN:   DEFB    0
173 ERR:    DEFB    0
174 KEY:    DEFB    0
175 URL:    DEFS    256
176         DEFS    256
177 STACK:
178

```

APPENDIX E

5 Card Stud in Z80

FujiNet 5 Card Stud is a multiplayer poker game served by the FujiNet Game System: players on any FujiNet machine — an Atari, an Apple II, a CoCo, an IBM PC — sit at the same tables, with bots filling the empty seats. This client puts an Altair at the table. It plays the game in plain text, one line per move, so it works on a Teletype.

A HAND AT A DEVELOPER TABLE

```
FUJINET 5 CARD STUD
YOUR NAME? altair

TABLES
 1 DEV ROOM - 7 BOTS      0 / 1
 2 DEV ROOM - 6 BOTS      0 / 2
 3 DEV ROOM - 5 BOTS      0 / 3
 4 DEV ROOM - 4 BOTS      0 / 4
 5 DEV ROOM - 3 BOTS      0 / 5
 6 DEV ROOM - 2 BOTS      0 / 6
 7 DEV ROOM - 1 BOTS      0 / 7
TABLE NUMBER (RETURN TO QUIT)? 6

ROUND 1   POT 3
ALTAIR    199      0 TH AD
> CLYD BOT 199      0 ?? 8S
JIM BOT   199      0 ?? AC
CLYD BOT   BET      5
JIM BOT    CALL     5
YOUR CARDS TH AD          POT 3
YOUR MOVE: 1 FOLD 2 CALL 3 RAISE 5 Q LEAVE? 2
ALTAIR     CALL         5

ROUND 2   POT 18
> ALTAIR   194      0 TH AD 3C
CLYD BOT   194      0 ?? 8S 5C
JIM BOT    194      0 ?? AC 2H
YOUR CARDS TH AD 3C      POT 18
YOUR MOVE: 1 FOLD 2 CHECK 3 BET 5 Q LEAVE? 2
ALTAIR     CHECK         0
CLYD BOT   CHECK         0
JIM BOT    CHECK         0

ROUND 3   POT 18
> ALTAIR   194      0 TH AD 3C QH
CLYD BOT   194      0 ?? 8S 5C 9D
JIM BOT    194      0 ?? AC 2H TS
YOUR CARDS TH AD 3C QH   POT 18
YOUR MOVE: 1 FOLD 2 CHECK 3 BET 10 Q LEAVE? 2
ALTAIR     CHECK         0
CLYD BOT   CHECK         0
JIM BOT    BET          10
YOUR CARDS TH AD 3C QH   POT 18
YOUR MOVE: 1 FOLD 2 CALL 3 RAISE 10 Q LEAVE? 2
ALTAIR     CALL          10
CLYD BOT   FOLD         0

ROUND 4   POT 38
ALTAIR    184      0 TH AD 3C QH JD
CLYD BOT   194      0 ??
> JIM BOT  184      0 ?? AC 2H TS 2S
JIM BOT    BET          10
YOUR CARDS TH AD 3C QH JD POT 38
YOUR MOVE: 1 FOLD 2 CALL 3 RAISE 10 Q LEAVE? 2
ALTAIR     CALL          10
CLYD BOT   FOLD         0

SHOWDOWN  POT 58
ALTAIR    174     10 TH AD 3C QH JD CALL
CLYD BOT   194      0 ??
JIM BOT    232     10 7C AC 2H TS 2S BET
JIM BOT WON WITH PAIR, TWOS
```

```

ROUND 1   POT 3
  ALTAIR   173   0 6S 5D
> CLYD BOT 193   0 ?? 2D
  JIM BOT  231   0 ?? AC
  CLYD BOT  BET   5
  JIM BOT  RAISE 10
YOUR CARDS 6S 5D           POT 3

```

E.1 The server

The server is at <https://5card.carr-designs.com/>. It keeps no connection open: every action is a single HTTP GET, and the game advances only when some client asks for the state. A client needs four requests:

Request	Purpose
<code>tables?bin=1</code>	The list of tables. (<code>&dev=1</code> lists the developer tables instead, for testing without disturbing real games.)
<code>state?table=T&player=NAME&bin=1</code>	The game as player NAME sees it. The first request joins the table.
<code>move/CC?table=T&player=NAME&bin=1</code>	Make the move whose two-letter code is CC (from the valid-move list), and return the new state.
<code>leave?table=T&player=NAME</code>	Leave the table.

With `bin=1` the server answers in a fixed binary layout instead of JSON, so the client reads fields at known offsets and never parses text. Strings are fixed-width, NUL-padded and lower case; numbers are 16 bits, little-endian.

Offset	Size	Game state field
0	81	last result text, such as <code>jim bot won with pair, twos</code>
81	1	round: 0 waiting for players, 1–4 betting, 5 showdown
82	2	pot
84	1	whose turn (signed): 0 is you, –1 between rounds
85	1	seconds left to move
86	1	1 if the table is full and you are only watching
87	1	number of valid moves
88	65	5 valid moves of 13 bytes: code (2 + NUL), name (9 + NUL)
153	1	number of players
154	33 each	the players, you first

Offset	Size	Player field
0	9	name
9	1	status: 0 waiting for the next game, 1 playing, 2 folded, 3 left
10	2	bet this round
12	8	last move, as text
20	2	purse
22	11	cards, two characters each: value (<code>2–9, t, j, q, k, a</code>) and suit (<code>c, d, h, s</code>); <code>??</code> for a face-down card

The table list is one count byte, then up to ten 36-byte records: table id (9), name (21) and a player count such as `3 / 8` (6).

E.2 Fetching

Every request is the same five FujiBus commands — OPEN in mode `0C`, then STATUS and READ until the end-of-file status `88`, then CLOSE — wrapped in HTTPGET. Note the end test: for HTTP the connected byte is no help (Chapter 9), so HTTPGET stops on status `88` with nothing waiting:

HTTPGET: WAITING FOR THE END OF THE REPLY

```

615 HG1:   LD     A,NET           ; STATUS ('S') -> avail, conn, err
616       LD     C,'S'
617       LD     B,00h
618       CALL   FBNEW
619       CALL   FBCALL
620       JR     C,HGX
621       JR     NZ,HGX
622       LD     E,(HL)           ; DE = bytes waiting
623       INC    HL
624       LD     D,(HL)
625       INC    HL
626       INC    HL
627       LD     A,D
628       OR     E
629       JR     NZ,HG3
630       LD     A,(HL)           ; nothing waiting: 136 = end of file
631       CP     136
632       JR     Z,HGOK
633       LD     A,(POLL)
634       DEC    A
635       LD     (POLL),A
636       JR     Z,HGX
637       LD     A,5               ; give the server 50 ms
638       CALL   DELAY
639       JR     HG1

```

The request strings are built by concatenation into URLBUF: the base address, the request name, and QUERY, which appends the table, the player name (spaces turned into `+`) and `&bin=1`.

E.3 The game loop

GLOOP

```

149 GLOOP: LD     DE,URLBUF
150       LD     HL,BASE
151       CALL   STRCPY
152       LD     A,(PEND)         ; a move to send?
153       OR     A
154       JR     Z,GL1
155       LD     HL,QMOVE         ; "move/" + code
156       CALL   STRCPY
157       LD     HL,(PEND+1)
158       CALL   STRCPY
159       XOR    A
160       LD     (PEND),A
161       JR     GL2
162 GL1:   LD     HL,QSTATE       ; "state"
163       CALL   STRCPY
164 GL2:   CALL   QUERY           ; "?table=..&player=..&bin=1"
165       CALL   HTTPGET
166       JR     NC,GL3
167       CALL   PRINT
168       DEFB   "NETWORK ERROR, TRYING AGAIN",13,10,0
169       JR     GWAIT
170 GL3:   CALL   REPORT
171       LD     A,(RESP+G_ACTV) ; my turn?
172       OR     A
173       JR     NZ,GWAIT
174       LD     A,(RESP+G_VIEW)
175       OR     A
176       JR     NZ,GWAIT
177       LD     A,(RESP+G_NMOVE)
178       OR     A
179       JR     Z,GWAIT
180       CALL   ASKMOVE          ; Q (leave) returns carry set

```

```

181      JR      C, LEAVE
182      JR      GLOOP
183
184  GWAIT: LD      B,100          ; about a second, watching for Q
185  GW1:   CALL   CONST
186      JR      Z, GW2
187      CALL   GETKEY
188      CP      'Q'
189      JR      Z, LEAVE
190  GW2:   LD      A,1
191      CALL   DELAY
192      DJNZ   GW1
193      JP      GLOOP

```

Each pass fetches the state — or, if a move is waiting, sends it, which returns the new state just the same — and reports what has changed. If the active player is 0 (you), you are seated, and there are moves to make, it asks for one; otherwise it waits about a second, watching for **Q**, and asks again.

E.4 Reporting for a Teletype

A screen client redraws the table on every change. On a Teletype at ten characters a second that would take most of a minute each time a bot bets, so **REPORT** prints the whole table only when a new round starts (or someone joins or leaves), and otherwise one line for each player whose bet or move has changed:

REPORT: WHAT CHANGED?

```

228  REPORT: LD      HL,(LASTLEN)  ; first state at this table?
229      LD      A,H
230      OR      L
231      JR      Z, RPFULL
232      LD      A,(LAST+G_ROUND) ; a new round?
233      LD      HL,RESP+G_ROUND
234      CP      (HL)
235      JR      NZ, RPFULL
236      LD      A,(LAST+G_NPLAY) ; someone came or went?
237      LD      HL,RESP+G_NPLAY
238      CP      (HL)
239      JR      NZ, RPFULL
240      OR      A
241      JR      Z, RPRES
242      LD      B,A
243      LD      HL,RESP+G_PLAY
244      LD      DE, LAST+G_PLAY
245  RP1:   PUSH   BC
246      PUSH   HL
247      PUSH   DE
248      LD      BC,P_BET          ; compare bet (2) and move (8)
249      ADD     HL,BC
250      EX      DE,HL
251      ADD     HL,BC
252      EX      DE,HL
253      LD      B,10
254  RP2:   LD      A,(DE)
255      CP      (HL)
256      JR      NZ, RP3
257      INC     HL
258      INC     DE
259      DJNZ   RP2
260      JR      RP4

```

The comparison is against **LAST**, a copy of the previous state. **LAST** has a second use: when the player picks a move, the move's two-letter code is taken from **LAST**, because **RESP** will be overwritten by the very request that sends the move.

E.5 Asking for a move

ASKMOVE prints the player's cards and the pot, then the valid moves numbered from 1, and waits for a key — but no longer than the server's move timer, since the server moves for a player who runs out of time. The wait is the timer's seconds, each a hundred 10 ms DELAYs, with the keyboard checked every time round:

WAITING FOR A MOVE, AGAINST THE CLOCK

```

542 AM3:    LD      D,A           ; D seconds of 100 x 10 ms
543 AM4:    LD      E,100
544 AM5:    CALL   CONST
545         JR      NZ,AM6
546         LD      A,1
547         CALL   DELAY
548         DEC     E
549         JR      NZ,AM5
550         DEC     D
551         JR      NZ,AM4
552         CALL   PRINT           ; out of time: the server moves for us
553         DEFB    "TOO SLOW",13,10,0
554         OR      A
555         RET

```

E.6 Listing

STUD.ASM

```

1  ;=====
2  ; STUD.ASM -- FujiNet 5 Card Stud for a terminal or Teletype
3  ;
4  ; A client for the FujiNet Game System's 5 Card Stud server. Every
5  ; exchange is one HTTPS GET through network unit 1; the server answers
6  ; in its fixed-width binary form (&bin=1), so the state is read by
7  ; offset with no parsing. The table is printed as plain scrolling
8  ; text, and only when something has changed, so it suits an ASR-33 as
9  ; well as a glass terminal.
10 ;
11 ; Hardware lives in ALTAIR.INC (88-2SI0: terminal on port A, FujiNet
12 ; on port B); the packet layer is FUJIBUS.INC.
13 ; Assemble: z88dk-z80asm -b stud.asm
14 ; (add -DDEVTABLES to list the server's hidden developer tables)
15 ;=====
16
17     ORG     0100h
18
19 NET     EQU     71h           ; network unit 1 (N1:)
20 RESPMAX EQU     512          ; largest server reply we keep
21
22 ; Offsets into the binary game state (the server's util.go)
23 G_LAST EQU      0           ; last result text, 80 + NUL
24 G_ROUND EQU     81          ; 0 waiting, 1-4 betting, 5 game over
25 G_POT EQU       82          ; u16
26 G_ACTV EQU      84          ; active player, signed; 0 = you
27 G_TIME EQU      85          ; seconds left to move
28 G_VIEW EQU      86          ; 1 = table full, you are watching
29 G_NMOVE EQU     87          ; valid moves that follow
30 G_MOVES EQU     88          ; 5 x 13: code 2+NUL, name 9+NUL
31 G_NPLAY EQU    153          ; players that follow
32 G_PLAY EQU     154          ; players, 33 bytes each:
33 P_NAME EQU      0           ; name 8 + NUL
34 P_STAT EQU      9           ; 0 waiting, 1 in, 2 folded, 3 left
35 P_BET EQU      10          ; u16
36 P_MOVE EQU     12           ; last move 7 + NUL
37 P_PURSE EQU    20          ; u16
38 P_HAND EQU     22          ; cards, 2 chars each, 10 + NUL
39 P_SIZE EQU     33
40
41 START: LD      SP,STACK
42        CALL   PINIT
43        CALL   PRINT
44        DEFB   13,10,"FUJINET 5 CARD STUD",13,10,0
45        CALL   PRINT
46        DEFB   "YOUR NAME? ",0
47        LD     HL,NAME
48        LD     B,8
49        CALL   GETLN
50        LD     A,B
51        OR     A

```

```

52      JR      NZ,ST1
53      LD      HL,DEFNAME      ; nothing typed: a default
54      LD      DE,NAME
55      CALL    STRCPY
56 ST1:  LD      HL,NAME      ; spaces become '+' in a URL
57 ST2:  LD      A,(HL)
58      OR      A
59      JR      Z,TABLES
60      CP      ' '
61      JR      NZ,ST3
62      LD      (HL),'+'
63 ST3:  INC     HL
64      JR      ST2
65
66 ;=====
67 ; The table list
68 ;=====
69 TABLES: LD    DE,URLBUF      ; base + "tables?bin=1"
70          LD    HL,BASE
71          CALL  STRCPY
72          LD    HL,QTABLES
73          CALL  STRCPY
74          CALL  HTTPGET
75          JP    C,NETFAIL
76          CALL  PRINT
77          DEFB  13,10,"TABLES",13,10,0
78          LD    HL,RESP      ; 1 byte count, then 36-byte records:
79          LD    A,(HL)      ; id 8+NUL, name 20+NUL, "p / m"+NUL
80          OR    A
81          JP    Z,NETFAIL
82          CP    10
83          JR    C,TB0      ; the server sends at most ten
84          LD    A,10
85 TB0:  LD    (NTABLE),A
86      INC    HL
87      LD    C,1
88 TB1:  PUSH   HL
89      LD    L,C
90      LD    H,0
91      LD    B,3
92      CALL  PRDECW
93      POP    HL
94      CALL  SPACE
95      CALL  SPACE
96      PUSH   HL
97      LD    DE,9      ; the name, padded to 22 columns
98      ADD    HL,DE
99      LD    B,20
100     CALL  PRUPN
101     LD    A,22
102     SUB    B
103     LD    B,A
104     CALL  SPACES
105     POP    HL
106     PUSH   HL
107     LD    DE,30      ; the player count
108     ADD    HL,DE
109     LD    B,6
110     CALL  PRUPN
111     CALL  CRLF
112     POP    HL
113     LD    DE,36
114     ADD    HL,DE
115     LD    A,(NTABLE)
116     CP    C
117     JR    Z,TB2
118     INC    C
119     JR    TB1
120 TB2:  CALL  PRINT
121     DEFB  "TABLE NUMBER (RETURN TO QUIT)? ",0
122     LD    HL,LINE
123     LD    B,3
124     CALL  GETLN
125     LD    A,B
126     OR    A
127     JP    Z,PEXIT
128     LD    A,(NTABLE)
129     LD    C,A
130     CALL  PARSE
131     JR    C,TB2
132     LD    L,A      ; TABLE = the chosen record's id
133     LD    H,0
134     LD    DE,36
135     CALL  MUL16
136     LD    DE,RESP+1
137     ADD    HL,DE
138     LD    DE,TABLE

```

```

139      CALL    STRCPY
140      XOR     A
141      LD      (PEND),A      ; no move waiting to be sent
142      LD      (LASTLEN),A
143      LD      (LASTLEN+1),A
144
145      ;=====
146      ; The game loop: fetch the state (or send a move), show it if it has
147      ; changed, and either ask for a move or wait a second.
148      ;=====
149  GLOOP:  LD      DE,URLBUF
150          LD      HL,BASE
151          CALL    STRCPY
152          LD      A,(PEND)      ; a move to send?
153          OR     A
154          JR     Z,GL1
155          LD      HL,QMOVE      ; "move/" + code
156          CALL    STRCPY
157          LD      HL,(PEND+1)
158          CALL    STRCPY
159          XOR     A
160          LD      (PEND),A
161          JR     GL2
162  GL1:    LD      HL,QSTATE      ; "state"
163          CALL    STRCPY
164  GL2:    CALL    QUERY          ; "?table=..&player=..&bin=1"
165          CALL    HTTPGET
166          JR     NC,GL3
167          CALL    PRINT
168          DEFB    "NETWORK ERROR, TRYING AGAIN",13,10,0
169          JR     WAIT
170  GL3:    CALL    REPORT
171          LD      A,(RESP+G_ACTV) ; my turn?
172          OR     A
173          JR     NZ,GWAIT
174          LD      A,(RESP+G_VIEW)
175          OR     A
176          JR     NZ,GWAIT
177          LD      A,(RESP+G_NMOVE)
178          OR     A
179          JR     Z,GWAIT
180          CALL    ASKMOVE      ; Q (leave) returns carry set
181          JR     C,LEAVE
182          JR     GLOOP
183
184  GWAIT:   LD      B,100      ; about a second, watching for Q
185  GW1:    CALL    CONST
186          JR     Z,GW2
187          CALL    GETKEY
188          CP     'Q'
189          JR     Z,LEAVE
190  GW2:    LD      A,1
191          CALL    DELAY
192          DJNZ   GW1
193          JP     GLOOP
194
195  LEAVE:   LD      DE,URLBUF      ; base + "leave" + query
196          LD      HL,BASE
197          CALL    STRCPY
198          LD      HL,QLEAVE
199          CALL    STRCPY
200          CALL    QUERY
201          CALL    HTTPGET
202          CALL    PRINT
203          DEFB    "YOU LEAVE THE TABLE.",13,10,0
204          JP     TABLES
205
206  NETFAIL:CALL    PRINT
207          DEFB    "CANNOT REACH THE SERVER.",13,10,0
208          JP     PEXIT
209
210      ; QUERY -- append "?table=<TABLE>&player=<NAME>&bin=1" at DE
211  QUERY:   LD      HL,QTABLE
212          CALL    STRCPY
213          LD      HL,TABLE
214          CALL    STRCPY
215          LD      HL,QPLAYER
216          CALL    STRCPY
217          LD      HL,NAME
218          CALL    STRCPY
219          LD      HL,QBIN
220          JP     STRCPY
221
222      ;=====
223      ; REPORT -- tell the player what has happened since the last state.
224      ; A new round (or the first look at the table) prints the whole table;
225      ; otherwise each player whose bet or move changed gets one line, so a

```

```

226 ; Teletype is not kept busy reprinting the table after every bot move.
227 ;=====
228 REPORT: LD      HL,(LASTLEN)    ; first state at this table?
229         LD      A,H
230         OR      L
231         JR      Z,RPFULL
232         LD      A,(LAST+G_ROUND) ; a new round?
233         LD      HL,RESP+G_ROUND
234         CP      (HL)
235         JR      NZ,RPFULL
236         LD      A,(LAST+G_NPLAY) ; someone came or went?
237         LD      HL,RESP+G_NPLAY
238         CP      (HL)
239         JR      NZ,RPFULL
240         OR      A
241         JR      Z,RPRES
242         LD      B,A
243         LD      HL,RESP+G_PLAY
244         LD      DE,LAST+G_PLAY
245 RP1:    PUSH     BC
246         PUSH     HL
247         PUSH     DE
248         LD      BC,P_BET          ; compare bet (2) and move (8)
249         ADD     HL,BC
250         EX      DE,HL
251         ADD     HL,BC
252         EX      DE,HL
253         LD      B,10
254 RP2:    LD      A,(DE)
255         CP      (HL)
256         JR      NZ,RP3
257         INC     HL
258         INC     DE
259         DJNZ    RP2
260         JR      RP4
261 RP3:    POP      DE                ; changed: one line for this player
262         POP      HL
263         PUSH     HL
264         PUSH     DE
265         CALL    EVENT
266 RP4:    POP      DE
267         POP      HL
268         LD      BC,P_SIZE
269         ADD     HL,BC
270         EX      DE,HL
271         ADD     HL,BC
272         EX      DE,HL
273         POP      BC
274         DJNZ    RP1
275 RPRES: LD      HL,RESP+G_LAST    ; a new result line?
276         LD      DE,LAST+G_LAST
277         LD      B,81
278 RP5:    LD      A,(DE)
279         CP      (HL)
280         JR      NZ,RP6
281         INC     HL
282         INC     DE
283         DJNZ    RP5
284         JR      RPCOPY
285 RP6:    LD      A,(RESP+G_LAST)
286         OR      A
287         JR      Z,RPCOPY
288         LD      HL,RESP+G_LAST
289         LD      B,80
290         CALL    PRUPN
291         CALL    CRLF
292 RPCOPY: LD      HL,RESP          ; this state becomes the last one
293         LD      DE,LAST
294         LD      BC,(RESPLEN)
295         LD      (LASTLEN),BC
296         LDIR
297         RET
298 RPFULL: CALL    SHOW
299         JR      RPCOPY
300
301 ; EVENT -- one line for the player record at HL: name, move, bet
302 EVENT:  PUSH     HL
303         LD      DE,P_MOVE
304         ADD     HL,DE
305         LD      A,(HL)          ; no move (a new round clears them)
306         POP      HL
307         OR      A
308         RET     Z
309         CALL    SPACE
310         CALL    SPACE
311         LD      B,8
312         CALL    PRUPN

```

```

313      LD      A,10
314      SUB     B
315      LD      B,A
316      CALL    SPACES
317      PUSH    HL
318      LD      DE,P_MOVE
319      ADD     HL,DE
320      LD      B,7
321      CALL    PRUPN
322      LD      A,7          ; then the bet, right-aligned
323      SUB     B
324      LD      B,A
325      CALL    SPACES
326      LD      DE,P_BET-P_MOVE
327      ADD     HL,DE
328      LD      E,(HL)
329      INC     HL
330      LD      D,(HL)
331      POP     HL
332      EX      DE,HL
333      LD      B,5
334      CALL    PRDECW
335      JP      CRLF
336
337      ;=====
338      ; SHOW -- print the table
339      ;=====
340  SHOW:  CALL    CRLF
341          LD      A,(RESP+G_ROUND)
342          OR      A
343          JR      NZ,SH1
344          CALL    PRINT          ; round 0: waiting for players
345          DEFB    "WAITING FOR MORE PLAYERS",13,10,0
346          JR      SH2
347  SH1:   CP      5
348          JR      NZ,SH1A
349          CALL    PRINT          ; round 5: the cards are turned over
350          DEFB    "SHOWDOWN",0
351          JR      SH1B
352  SH1A:  CALL    PRINT
353          DEFB    "ROUND ",0
354          LD      A,(RESP+G_ROUND)
355          LD      L,A
356          LD      H,0
357          CALL    PRDEC
358  SH1B:  CALL    PRINT
359          DEFB    "    POT ",0
360          LD      HL,(RESP+G_POT)
361          CALL    PRDEC
362          CALL    CRLF
363  SH2:   LD      A,(RESP+G_NPLAY)
364          OR      A
365          JP      Z,SH9
366          LD      B,A
367          LD      C,0          ; C = player index
368          LD      HL,RESP+G_PLAY
369  SH3:   PUSH    BC
370          PUSH    HL
371          LD      A,(RESP+G_ACTV) ; '>' marks whose turn it is
372          CP      C
373          LD      A,' '
374          JR      NZ,SH4
375          LD      A,'>'
376  SH4:   CALL    CONOUT
377          CALL    SPACE
378          LD      B,8          ; name, 10 columns
379          CALL    PRUPN
380          LD      A,10
381          SUB     B
382          LD      B,A
383          CALL    SPACES
384          POP     HL
385          PUSH    HL
386          LD      DE,P_PURSE    ; purse
387          ADD     HL,DE
388          LD      E,(HL)
389          INC     HL
390          LD      D,(HL)
391          EX      DE,HL
392          LD      B,5
393          CALL    PRDECW
394          POP     HL
395          PUSH    HL
396          LD      DE,P_BET      ; bet
397          ADD     HL,DE
398          LD      E,(HL)
399          INC     HL

```

```

400      LD      D,(HL)
401      EX      DE,HL
402      LD      B,5
403      CALL    PRDECW
404      CALL    SPACE
405      CALL    SPACE
406      POP     HL
407      PUSH    HL
408      LD      DE,P_HAND      ; the cards, "KS 9H ?? "
409      ADD     HL,DE
410      CALL    PRHAND
411      POP     HL
412      PUSH    HL
413      LD      DE,P_MOVE      ; last move, or the player's status
414      ADD     HL,DE
415      LD      A,(HL)
416      OR      A
417      JR      Z,SH5
418      LD      B,7
419      CALL    PRUPN
420      JR      SH7
421 SH5:    POP     HL
422      PUSH    HL
423      LD      DE,P_STAT
424      ADD     HL,DE
425      LD      A,(HL)
426      OR      A
427      JR      NZ,SH6
428      CALL    PRINT
429      DEFB    "WAITING",0
430      JR      SH7
431 SH6:    CP      3
432      JR      NZ,SH7
433      CALL    PRINT
434      DEFB    "LEFT",0
435 SH7:    CALL    CRLF
436      POP     HL
437      LD      DE,P_SIZE
438      ADD     HL,DE
439      POP     BC
440      INC     C
441      DEC     B
442      JP      NZ,SH3
443 SH9:    LD      A,(RESP+G_ROUND) ; the result line belongs to round 5
444      OR      A                ; (and to round 0, where it says who
445      JR      Z,SH10           ; is still awaited)
446      CP      5
447      RET     NZ
448 SH10:   LD      A,(RESP+G_LAST)
449      OR      A
450      RET     Z
451      LD      HL,RESP+G_LAST
452      LD      B,80
453      CALL    PRUPN
454      JP      CRLF
455
456 ; PRHAND -- print the hand at HL as "KS 9H ?? ", padded to 16 columns
457 PRHAND: LD      B,16
458 PH1:    LD      A,(HL)
459      OR      A
460      JR      Z,PH2
461      CALL    UCASE
462      CALL    CONOUT
463      INC     HL
464      LD      A,(HL)
465      CALL    UCASE
466      CALL    CONOUT
467      INC     HL
468      CALL    SPACE
469      DEC     B
470      DEC     B
471      DEC     B
472      JR      PH1
473 PH2:    JP      SPACES
474
475 ; PRUPN -- print at most B characters of the string at HL in upper
476 ; case; on return B = the number printed (C, DE, HL preserved)
477 PRUPN:  PUSH    HL
478      PUSH    DE
479      LD      D,B                ; D = characters still allowed
480      LD      B,0
481 PU1:    LD      A,D
482      OR      A
483      JR      Z,PU2
484      LD      A,(HL)
485      OR      A
486      JR      Z,PU2

```

```

487      CALL    UCASE
488      CALL    CONOUT
489      INC     HL
490      INC     B
491      DEC     D
492      JR      PU1
493 PU2:   POP     DE
494      POP     HL
495      RET
496
497 ;=====
498 ; ASKMOVE -- list the valid moves and wait for a choice, no longer
499 ; than the server's move timer. Carry set if the player typed Q.
500 ;=====
501 ASKMOVE:CALL    PRINT
502      DEFB     "YOUR CARDS ",0
503      LD       HL,RESP+G_PLAY+P_HAND
504      CALL     PRHAND
505      CALL     PRINT
506      DEFB     "POT ",0
507      LD       HL,(RESP+G_POT)
508      CALL     PRDEC
509      CALL     PRINT
510      DEFB     13,10,"YOUR MOVE:",0
511      LD       A,(RESP+G_NMOVE)
512      CP       6
513      JR      C,AM1
514      LD       A,5
515 AM1:   LD       B,A
516      LD       C,'1'
517      LD       HL,RESP+G_MOVES
518 AM2:   CALL     SPACE
519      CALL     SPACE
520      LD       A,C
521      CALL     CONOUT
522      CALL     SPACE
523      PUSH     HL
524      PUSH     BC
525      INC     HL           ; the name follows the 3-byte code
526      INC     HL
527      INC     HL
528      LD       B,9
529      CALL     PRUPN
530      POP      BC
531      POP      HL
532      LD       DE,13
533      ADD      HL,DE
534      INC      C
535      DJNZ     AM2
536      CALL     PRINT
537      DEFB     " Q LEAVE? ",0
538      LD       A,(RESP+G_TIME) ; wait up to the move timer
539      OR       A
540      JR      NZ,AM3
541      LD       A,1
542 AM3:   LD       D,A           ; D seconds of 100 x 10 ms
543 AM4:   LD       E,100
544 AM5:   CALL     CONST
545      JR      NZ,AM6
546      LD       A,1
547      CALL     DELAY
548      DEC     E
549      JR      NZ,AM5
550      DEC     D
551      JR      NZ,AM4
552      CALL     PRINT           ; out of time: the server moves for us
553      DEFB     "TOO SLOW",13,10,0
554      OR      A
555      RET
556 AM6:   CALL     GETKEY
557      CP       'Q'
558      JR      NZ,AM7
559      CALL     CONOUT
560      CALL     CRLF
561      SCF
562      RET
563 AM7:   SUB     '1'           ; a move number?
564      JR      C,AM5
565      LD       HL,RESP+G_NMOVE
566      CP       (HL)
567      JR      NC,AM5
568      PUSH     AF
569      ADD      A,'1'
570      CALL     CONOUT
571      CALL     CRLF
572      POP      AF
573      LD       L,A           ; PEND+1 = address of that move's code

```

```

574      LD      H,0
575      LD      DE,13
576      CALL    MUL16
577      LD      DE, LAST+G_MOVES ; (REPORT copied this state to LAST;
578      ADD     HL, DE           ; RESP is overwritten by the fetch)
579      LD      (PEND+1), HL
580      LD      A, 1
581      LD      (PEND), A
582      OR      A
583      RET
584
585 ;=====
586 ; HTTPGET -- GET the devicespec in URLBUF into RESP (RESPLEN bytes).
587 ; OPEN, then STATUS / READ until the server's reply is used up (EOF),
588 ; then CLOSE. Carry set on failure.
589 ;=====
590 HTTPGET: LD      HL, 0
591          LD      (RESPLEN), HL
592 ; OPEN ('O'): p0 = 12 (HTTP GET), p1 = 0 (no translation)
593          LD      A, NET
594          LD      C, 'O'
595          LD      B, 02h
596          CALL    FBNEW
597          LD      A, 12
598          CALL    FBBYTE
599          XOR     A
600          CALL    FBBYTE
601          LD      HL, URLBUF
602          CALL    STRLEN
603          INC     BC           ; and the NUL
604          CALL    FBMEM
605          LD      A, 80        ; DNS + TLS: allow 20 s
606          LD      (FBWAIT), A
607          CALL    FBCALL
608          LD      A, 40
609          LD      (FBWAIT), A
610          RET      C
611          SCF
612          RET      NZ          ; NAK
613          LD      A, 200       ; at most ~200 empty polls
614          LD      (POLL), A
615 HG1:     LD      A, NET        ; STATUS ('S') -> avail, conn, err
616          LD      C, 'S'
617          LD      B, 00h
618          CALL    FBNEW
619          CALL    FBCALL
620          JR      C, HGX
621          JR      NZ, HGX
622          LD      E, (HL)      ; DE = bytes waiting
623          INC     HL
624          LD      D, (HL)
625          INC     HL
626          INC     HL
627          LD      A, D
628          OR      E
629          JR      NZ, HG3
630          LD      A, (HL)      ; nothing waiting: 136 = end of file
631          CP      136
632          JR      Z, HGOK
633          LD      A, (POLL)
634          DEC     A
635          LD      (POLL), A
636          JR      Z, HGX
637          LD      A, 5         ; give the server 50 ms
638          CALL    DELAY
639          JR      HG1
640 HG3:     LD      HL, RESPMAX   ; room left = RESPMAX - RESPLEN
641          LD      BC, (RESPLEN)
642          OR      A
643          SBC     HL, BC
644          JR      Z, HGX        ; full: the reply is too big
645          OR      A            ; ask for min(waiting, room)
646          SBC     HL, DE
647          ADD     HL, DE
648          JR      NC, HG4
649          EX      DE, HL        ; room < waiting
650 HG4:     PUSH    DE
651          LD      A, NET        ; READ ('R'): p0 = count (u16)
652          LD      C, 'R'
653          LD      B, 05h
654          CALL    FBNEW
655          POP     HL
656          CALL    FBWORD
657          CALL    FBCALL
658          JR      C, HGX
659          JR      NZ, HGX
660          PUSH    BC           ; append the bytes to RESP

```

```

661      LD      DE,(RESPLEN)
662      PUSH    HL
663      LD      HL,RESP
664      ADD     HL,DE
665      EX      DE,HL
666      POP     HL
667      LD      A,B
668      OR      C
669      JR      Z,HG5
670      LDIR
671 HG5:    POP     BC
672      LD      HL,(RESPLEN)
673      ADD     HL,BC
674      LD      (RESPLEN),HL
675      JR      HG1
676 HGOK:   CALL   HCLOSE
677      OR      A
678      RET
679 HGX:    CALL   HCLOSE
680      SCF
681      RET
682 HCLOSE: LD      A,NET          ; CLOSE ('C')
683      LD      C,'C'
684      LD      B,00h
685      CALL   FBNEW
686      JP     FBCALL
687
688 ;=====
689 ; String helpers
690 ;=====
691
692 ; STRCPY -- copy the string at HL to DE with its NUL; DE is left on
693 ; the copied NUL, so a second STRCPY appends
694 STRCPY: LD      A,(HL)
695      LD      (DE),A
696      OR      A
697      RET     Z
698      INC     HL
699      INC     DE
700      JR      STRCPY
701
702 ; STRLEN -- BC = length of the string at HL
703 STRLEN: PUSH    HL
704      LD      BC,0
705 SL1:    LD      A,(HL)
706      OR      A
707      JR      Z,SL2
708      INC     HL
709      INC     BC
710      JR      SL1
711 SL2:    POP     HL
712      RET
713
714 ; PARSE -- the decimal number in LINE, 1..C -> A = number - 1;
715 ; carry set if blank, not a number, or out of range
716 PARSE: LD      HL,LINE
717      LD      B,0
718 PA1:    LD      A,(HL)
719      OR      A
720      JR      Z,PA2
721      SUB     '0'
722      RET     C
723      CP      10
724      CCF
725      RET     C
726      LD      D,A
727      LD      A,B
728      ADD     A,A
729      LD      E,A
730      ADD     A,A
731      ADD     A,A
732      ADD     A,E
733      ADD     A,D
734      LD      B,A
735      INC     HL
736      JR      PA1
737 PA2:    LD      A,B
738      OR      A
739      SCF
740      RET     Z
741      CP      C
742      JR      Z,PA3
743      CCF
744      RET     C
745 PA3:    DEC     A
746      OR      A
747      RET

```

```

748
749 ; MUL16 -- HL = HL x DE
750 MUL16:  PUSH    BC
751         LD      B,H
752         LD      C,L
753         LD      HL,0
754         LD      A,16
755 MU1:     ADD     HL,HL
756         EX      DE,HL
757         ADD     HL,HL
758         EX      DE,HL
759         JR      NC,MU2
760         ADD     HL,BC
761 MU2:     DEC     A
762         JR      NZ,MU1
763         POP     BC
764         RET
765
766 ;=====
767 ; The server and its requests
768 ;=====
769 BASE:    DEFB    "N:HTTPS://Scard.carr-designs.com/",0
770         IF      DEVTABLES
771 QTABLES: DEFB    "tables?bin=1&dev=1",0
772         ELSE
773 QTABLES: DEFB    "tables?bin=1",0
774         ENDIF
775 QSTATE:  DEFB    "state",0
776 QMOVE:   DEFB    "move/",0
777 QLEAVE:  DEFB    "leave",0
778 QTABLE:  DEFB    "?table=",0
779 QPLAYER: DEFB    "&player=",0
780 QBIN:    DEFB    "&bin=1",0
781 DEFNAME: DEFB    "ALTAIR",0
782
783         INCLUDE  "../common/altair.inc"
784         INCLUDE  "../common/conio.inc"
785         INCLUDE  "../common/fujibus.inc"
786
787 ;=====
788 ; Variables and buffers
789 ;=====
790 NTABLE:  DEFB    0
791 POLLS:   DEFB    0
792 PEND:    DEFB    0           ; 1 = a move is waiting to be sent,
793         DEFW    0           ;   and the address of its code
794 NAME:    DEFS    9
795 TABLE:  DEFS    9
796 LINE:    DEFS    4
797 RESPLEN: DEFW    0
798 LASTLEN: DEFW    0
799 URLBUF:  DEFS    128
800 RESP:    DEFS    RESPMAX
801 LAST:    DEFS    RESPMAX
802         DEFS    256
803 STACK:
804

```

APPENDIX F

Running the Examples in SIMH

Every program in this book was assembled and run on the AltairZ80 simulator from SIMH, with its 88-2SIO model connected to fujinet-pc. That arrangement needs no hardware at all, and it is a good place to develop: when something goes wrong, the fujinet-pc log shows every packet it received.



Figure 4. The test rig. Both 88-2SIO ports listen on TCP; a bridge connects port B to fujinet-pc's Bus-over-IP port.

F.1 fujinet-pc

Build fujinet-pc for the RS-232 target from the firmware tree, and give it a configuration with Bus over IP turned on:

```
cd fujinet-firmware
./build.sh -p RS232          # or: cmake -DFUJINET_TARGET=RS232 ...
cd build/dist
cat >> fnconfig.ini <<EOF
[BOIP]
enabled=1
host=localhost
port=1985
EOF
./fujinet -c fnconfig.ini -s SD -u http://127.0.0.1:8000
```

F.2 The simulator

AltairZ80 is part of SIMH (`BIN/altairz80` after building). Its 88-2SIO is two devices, `M2SIO0` (port A, `10h`) and `M2SIO1` (port B, `12h`), disabled until asked for. The script used for this book:

SIM/ALTAIR.INI

```
; SIMH AltairZ80: run a FujiNet example on a simulated 88-2SIO.
; port A (10h/11h) = terminal: connect to TCP <term-port> (nc)
; port B (12h/13h) = FujiNet: listens on TCP <fujinet-port>; bridge
;                      fujinet-pc's BoIP port with
;                      socat TCP:127.0.0.1:2301 TCP:127.0.0.1:1985
; usage: altairz80 altair.ini <program.bin> <term-port> <fujinet-port>
; e.g. altairz80 altair.ini netcat.bin 2300 2301
set cpu z80
set cpu 64k
; run the Z80 at 2 MHz (CLOCK is in kHz), so the software timeouts
; in ALTAIR.INC mean what they say
deposit clock 2000
set m2sio0 enabled
set m2sio1 enabled
set m2sio0 noconsole
; the 6850 drives RTS; let DTR follow it so TMXR answers the call
set m2sio0 dtr
set m2sio1 dtr
attach m2sio0 %2;notelnet
attach m2sio1 %3;notelnet
load %1 100
go 100
exit
```

Four details matter, each learned the hard way:

1. **deposit clock 2000**. Left alone, AltairZ80 runs as fast as the host computer can, and the program's timeouts — counted in instructions — become microseconds. **CLOCK** sets the simulated speed in kilohertz. (**SET THROTTLE** does not affect the Z80 timing here.)
2. **set m2sio0 dtr**. The 88-2SIO model passes modem signals through, and a listening port does not accept a connection until the simulated computer raises DTR. With **DTR**, DTR follows the 6850's RTS output, which **PINIT** turns on.
3. **notelnet**. Without it, SIMH speaks TELNET on the port and doubles every **FF** byte — fatal to binary packets.
4. **Listen on both ports**. Making port B connect out to fujinet-pc (**attach m2sio1 connect=...**) crashed the simulator build used here, so both ports listen and **socat** makes the connection to fujinet-pc.

F.3 Running a program

sim/run.sh puts it all together: it starts the simulator on two fresh TCP ports (SIMH cannot reuse a port still in **TIME_WAIT** from the last run), bridges port B to fujinet-pc, and drives port A with **term.py**, which can type a script of keys:

```
cd sim
./run.sh ../listings/hello/hello.bin
./run.sh ../listings/config/fncfg.bin --keys "{3}B1\r{2}1\r" --wait 5
FAULT=3 ./run.sh ../listings/config/fncfg.bin      # damage every 3rd reply
```

In **--keys**, **{n}** waits **n** seconds and **\r** is Return. For an interactive session, run the simulator by hand and connect to port A with **nc localhost <port>**.

F.4 On real hardware

On a real Altair the same binaries run unchanged at **0100h**. Strap 88-2SIO port B for the baud rate in **fnconfig.ini** (9600 in the example of Chapter 3), connect it to the FujiNet with a straight-through cable — or a null-modem adapter if the board is wired as DCE — and load the program with your usual monitor or from CP/M. If the CPU runs at 4 MHz, double **FNTMO** and **DLY10MS** in **ALTAIR.INC** first.

APPENDIX G

Firmware Behaviour and Errata

These are the places where the RS-232 firmware, at commit `670b2ed55`, does something a programmer would not expect from the command's name or from the rest of the protocol. Each was confirmed on a running adapter. The source references are to `fujinet-firmware`.

Command	Behaviour and source
any	A command missing a parameter its handler reads throws an uncaught exception: fujinet-pc aborts, the hardware restarts. <code>lib/bus/rs232/FujiBusPacket.h:76</code>
any	A bad checksum, a wrong length or a stalled frame gets no reply at all, not a NAK. <code>lib/bus/rs232/rs232.cpp:114</code>
any	A frame opened with two ENDS is lost: the empty frame is discarded and the real one handed to the modem as stray bytes. <code>lib/bus/rs232/rs232.cpp:107</code>
any	A short fixed-size payload is accepted, leaving the rest of the buffer uninitialised (some builds reject it instead). <code>lib/bus/rs232/rs232.cpp:78</code>
unknown device	Answered with a NAK whose device byte is <code>00</code> .
Fuji <code>FF</code> RESET	Restarts without an ACK; the ACK-then-restart routine is unreachable. <code>lib/device/fujiDevice/fujiDevice.cpp:55</code>
Fuji <code>FE</code> GET_SSID	Returns the stored WiFi password. <code>lib/device/fujiDevice/fujiDevice.cpp:438</code>
Fuji <code>FB</code> SET_SSID	A failed join sends two NAKs. <code>lib/device/fujiDevice/fujiDevice.cpp:555</code>
Fuji <code>F2</code> / <code>F1</code>	READ_DEVICE_SLOTS returns file names without their directories; writing them back with WRITE_DEVICE_SLOTS loses the paths. <code>lib/device/fujiDevice/fujiDevice.cpp:1575</code> , <code>:1613</code>
Fuji <code>E7</code> NEW_DISK	Always NAKs (image creation not implemented), after creating or truncating the file and changing the slot. <code>lib/media/rs232/diskTypeImg.cpp:167</code>
Fuji <code>E0</code> GET_HOST_PREFIX	Returns leftover memory after the prefix, and entirely when no prefix was ever set. <code>lib/device/fujiDevice/fujiDevice.cpp:1393</code>
Fuji <code>E6</code> UNMOUNT_HOST	NAK for the SD card host on fujinet-pc. <code>lib/device/fujiDevice/fujiDevice.cpp:1563</code>
Fuji <code>53</code> STATUS	Always four zero bytes; the RS-232 mount-time status routine is shadowed by the shared handler. <code>lib/device/rs232/rs232Fuji.cpp:61</code>
Fuji <code>DD</code> READ_APPKEY	Answers ACK with 64 zero bytes on any failure. <code>lib/device/fujiDevice/AppKeyMixin.cpp:83</code>
Fuji <code>CE</code> B64 ENCODE_LENGTH	Counts a trailing NUL. <code>lib/device/fujiDevice/Base64Mixin.cpp:45</code>
Disk <code>53</code> STATUS	Wired to WRITE. <code>lib/device/rs232/disk.cpp:386</code>
Disk <code>21</code> FORMAT	Returns a 128-byte Atari bad-sector list and leaves the image alone. <code>lib/media/rs232/diskTypeImg.cpp</code>
Disk mount of a ROM	Pushes the ROM to device <code>FF</code> and waits for replies that an ordinary computer never sends. <code>lib/device/rs232/disk.cpp:313</code>
Network <code>53</code> STATUS	For HTTP the connected byte is 0 throughout; for UDP on fujinet-pc the waiting count stays 0. <code>lib/network-protocol/UDP.cpp:166</code>
Network <code>63</code> CLOSE_CLIENT	Answer inverted: NAK after a disconnect, ACK when there was no client. <code>lib/network-protocol/TCP.cpp:337</code>

Command	Behaviour and source
Network 72 GET_REMOTE	Text ends with 9B , not NUL; the rest of the 256 bytes is leftover memory. lib/network-protocol/UDP.cpp:230
Network 44 SET_DESTINATION	Needs a prefix before the host and a terminating NUL. lib/network-protocol/UDP.cpp:199
Network 26 TELL	Leftover memory on a unit that is not open. lib/device/NDevice/NDevice.cpp:697
Clock 54	Named SETT2_ALT2, but a read on RS-232. lib/device/fujiClock/fujiClock.cpp:385
Clock 99 / 74	ACK before validation; a bad zone is ignored silently. lib/device/fujiClock/fujiClock.cpp:319
Printer 57 WRITE	More than 255 bytes is cut short; more than 320 overruns the buffer. lib/device/rs232/printer.cpp:46
Printer 53 STATUS	Byte 1 is never set. lib/device/rs232/printer.cpp:96
Modem	Replies are never framed; immediate AT responses are suppressed. lib/bus/rs232/rs232.cpp:376 , lib/device/rs232/modem.cpp:105
Modem ATCPM	Dereferences the absent CP/M device and crashes. lib/device/rs232/modem.cpp:1254
Modem AT+TERM	ANSI and DUMB are swapped. lib/device/rs232/modem.cpp:1023
CP/M 5A	Never added to the bus. src/main.cpp:364
[PrinterN]	Every printer section loads into slot 0. lib/config/fnc_util.cpp:47
[Tape1]	Loads into the disk-slot table. lib/config/fnc_mounts.cpp:156
[Serial] proceed	Not written back on the RS-232 build of fujinet-pc. lib/config/fnc_save.cpp:218
RTS/CTS	Flow control requested but the pins are not routed to the UART. lib/hardware/ESP32UARTChannel.cpp:57